Student Project Management Platform at the College of AI Engineering

Project Report (Graduation 2)

In advance following the completion of obtaining the Bachelor's degree in Engineering

Artificial Intelligence Engineering - Software Engineering and Intelligent Information Systems

Prepared by

Nader Al-Rifai Osama Al-Omari

Supervised by

ENG. Shadi Al-Bleidi

The academic year

2026-2025

Dedication

Abstract

Graduation-project management represented a complex academic process that required continuous coordination among Students, Supervisors, and Administrators, beginning with project publication, idea submission, and team formation and continuing through milestone and submission monitoring. Fragmented procedures and multiple communication channels caused difficulties in tracking project status, scattered data and documents, delayed request processing, and unclear responsibilities among the involved parties.

The project aimed to develop a centralized electronic platform for managing the graduation-project lifecycle within the Faculty of Artificial Intelligence Engineering at the Syrian Private University. It sought to improve the organization of academic processes, facilitate communication and monitoring, and provide clear permissions for Students, Supervisors, and Administrators. It also aimed to assist Students in preparing more structured project proposals and identifying semantically similar ideas before final submission.

An iterative and incremental development methodology was followed. The work began by examining the existing graduation-project management procedures and analyzing their principal limitations. Requirements were then collected and validated in cooperation with the relevant stakeholders. The system processes, structure, data, and interfaces were subsequently modeled and designed. Functionalities were implemented through successive development stages, with each stage reviewed and improved according to feedback and newly identified needs.

The project resulted in a unified platform that supported user and project management, project join requests, new project ideas, team formation, milestone and progress monitoring, submission upload and review, messaging, announcements, notifications, and activity monitoring. The platform also provided advisory assistance for project-proposal preparation and semantic-similarity analysis while preserving acceptance and rejection decisions as the responsibility of academic Supervisors.

The project concluded that the platform improved the organization of graduation-project management, reduced information fragmentation, connected academic activities through a clear and integrated workflow, and provided a foundation that could be extended in future development.

Keywords: Graduation Project Management, Academic Workflow, Project Management, Access Control, Artificial Intelligence, Semantic Similarity.

الملخص

مثّلت إدارة مشاريع التخرج عملية أكاديمية معقدة تطلبت تنسيقًا مستمرًا بين الطلاب والمشرفين والإدارة، بدءًا من طرح المشاريع وتقديم الأفكار وتشكيل الفرق، وصولًا إلى متابعة المراحل والتسليمات. وأدت الإجراءات المتفرقة وتعدد قنوات التواصل إلى صعوبة متابعة حالة المشاريع، وتشتت البيانات والوثائق، وتأخر معالجة الطلبات، وضعف وضوح المسؤوليات بين الأطراف المعنية.

هدف المشروع إلى تطوير منصة إلكترونية مركزية لإدارة دورة حياة مشاريع التخرج ضمن كلية هندسة الذكاء الاصطناعي في الجامعة السورية الخاصة، وتحسين تنظيم العمليات الأكاديمية، وتسهيل التواصل والمتابعة، وتوفير صلاحيات واضحة للطلاب والمشرفين ومديري النظام. كما هدف إلى مساعدة الطلاب في إعداد مقترحات مشاريع أكثر تنظيمًا والكشف عن الأفكار المتشابهة دلاليًا قبل تقديمها.

اتُّبعت منهجية تطوير تكرارية وتزايدية، بدأت بدراسة إجراءات إدارة مشاريع التخرج وتحليل المشكلات القائمة، ثم جمع المتطلبات والتحقق منها بالتعاون مع أصحاب المصلحة. بعد ذلك جرى نمذجة عمليات النظام، وتصميم بنيته وبياناته وواجهاته، وتنفيذ الوظائف على مراحل متتابعة، مع مراجعة كل مرحلة وتحسينها وفق الملاحظات والاحتياجات التي ظهرت أثناء التطوير.

أسفر المشروع عن منصة موحدة دعمت إدارة المستخدمين والمشاريع، وتقديم طلبات الانضمام والأفكار الجديدة، وتشكيل فرق المشاريع، ومتابعة المراحل والتقدم، ورفع التسليمات ومراجعتها، وتبادل الرسائل والإعلانات والإشعارات، ومراقبة الأنشطة. كما وفرت المنصة مساعدة استشارية لإعداد مقترحات المشاريع وتحليل التشابه الدلالي، مع إبقاء قرارات القبول والرفض من مسؤولية المشرفين.

خلص المشروع إلى أن المنصة حسّنت تنظيم إدارة مشاريع التخرج، وقللت تشتت المعلومات، وربطت العمليات الأكاديمية ضمن سير عمل واضح ومتكامل، ووفرت أساسًا قابلًا للتطوير مستقبلًا.

الكلمات المفتاحية :إدارة مشاريع التخرج، سير العمل الأكاديمي، إدارة المشاريع، التحكم بالصلاحيات، الذكاء الاصطناعي، التشابه الدلالي.

Table of Contents

Table of Tables

Table of Figure

Chapter 1: Introduction

1.1 Project Overview and Background

Graduation projects represent one of the most significant academic requirements in software engineering education, as they provide students with the opportunity to integrate the theoretical knowledge and practical skills acquired throughout their academic studies into a comprehensive software solution that addresses real-world problems. Successfully managing graduation projects requires continuous coordination among students, academic supervisors, and administrative staff throughout multiple stages, including project proposal submission, project selection, team formation, supervision, progress monitoring, milestone management, and final project evaluation.

Despite the importance of this process, many educational institutions continue to rely on fragmented administrative procedures, manual documentation, and multiple communication channels to manage graduation projects. Such approaches often result in duplicated administrative work, inconsistent project records, delayed communication, limited visibility into project progress, and difficulties in tracking project requests, approvals, and academic activities. These challenges affect all stakeholders involved in the graduation project lifecycle and reduce the overall efficiency and transparency of project management.

To address these challenges, the **Graduation Project Management Platform** was developed as a centralized web-based information system that manages the complete graduation project lifecycle within the Faculty of Artificial Intelligence Engineering at the Syrian Private University. The platform provides a unified environment that enables administrators, supervisors, and students to perform their respective responsibilities through structured workflows and integrated system services.

The platform supports project publication, project enrollment requests, student-proposed graduation ideas, supervisor review and approval workflows, milestone management, document submission, announcements, notifications, secure messaging, activity monitoring, and administrative management through dedicated interfaces for each user role. In addition, the platform incorporates advisory Artificial Intelligence capabilities that assist students during the project proposal process. These capabilities include an **AI Proposal Assistant**, which helps students organize and improve project proposals, and a **Semantic Similarity Assistant**, which analyzes the semantic similarity between newly proposed ideas and previously accepted graduation projects or existing proposals to reduce unnecessary duplication while preserving human academic decision-making.

The system was implemented using the Laravel framework following the Model–View–Controller (MVC) architectural pattern and adopts a modular architecture that emphasizes maintainability, scalability, security, and extensibility. Authentication is performed using university numbers, while authorization is enforced through role-based access control to ensure that each user can access only the functionalities associated with their academic responsibilities.

By integrating project management workflows, communication services, document management, administrative functions, and advisory Artificial Intelligence features into a single platform, the proposed system provides a comprehensive solution that improves the organization, efficiency, and transparency of graduation project management while supporting both academic staff and students throughout the entire project lifecycle.

1.2 Problem Statement

The management of graduation projects is a multidisciplinary academic process that requires continuous coordination among students, supervisors, and administrative staff from the initial project proposal stage to the final project evaluation. In many higher education institutions, this process is still performed using fragmented administrative procedures, manual record keeping, and multiple independent communication channels. Consequently, project-related information becomes scattered across different platforms, making it difficult to maintain consistency, monitor project progress, and ensure efficient collaboration among all stakeholders.

The problem lies in the fact that existing graduation project management practices do not provide a unified environment capable of supporting the entire project lifecycle. This limitation leads to delays in processing project requests, difficulties in tracking project milestones, inefficient communication between students and supervisors, duplicated administrative effort, and limited visibility into project status. Furthermore, the absence of centralized workflow management increases the possibility of inconsistent decision-making and reduces the overall efficiency of academic administration.

Students also encounter additional challenges during the project proposal stage. Preparing a well-structured graduation project proposal requires organizing ideas into a clear academic format that includes the project motivation, objectives, scope, methodology, expected outcomes, and functional requirements. Many students find this process difficult, resulting in proposals that vary considerably in quality and often require multiple rounds of revision before evaluation.

Another significant challenge is the possibility of submitting project ideas that are conceptually similar to previously accepted graduation projects or existing project proposals. Since similarity assessment is traditionally performed through manual review, identifying semantically related ideas becomes time-consuming and highly dependent on the experience of academic supervisors. This situation may increase review effort and allow similar project ideas to pass unnoticed during the early stages of proposal evaluation.

To address these challenges, the proposed **Graduation Project Management Platform** provides a centralized web-based environment that integrates graduation project administration, structured workflow management, stakeholder collaboration, project monitoring, document management, and secure communication into a single platform. In addition, the system incorporates two advisory

Artificial Intelligence services. The **AI Proposal Assistant** supports students in preparing structured project proposals, while the **Semantic Similarity Assistant** analyzes newly proposed ideas against existing graduation projects and previously accepted proposals to provide advisory similarity feedback before submission. These AI services are designed to support academic decision-making without replacing the responsibility of supervisors or administrators.

By providing an integrated platform that combines administrative management, project collaboration, workflow automation, and advisory Artificial Intelligence capabilities, the proposed system aims to improve efficiency, transparency, proposal quality, and overall graduation project management within the university environment.

1.3 Project Objectives

The primary objective of the **Graduation Project Management Platform** is to develop a secure, centralized, and scalable web-based platform that effectively manages the complete graduation project lifecycle while improving collaboration, transparency, and administrative efficiency within the Faculty of Artificial Intelligence Engineering at the Syrian Private University.

To accomplish this objective, the project aims to achieve the following specific objectives:

1. **To centralize graduation project management** by providing an integrated platform that manages project publication, student enrollment, project requests, proposal submission, milestone tracking, and project completion.
2. **To enhance collaboration among stakeholders** by facilitating structured communication and coordination between students, supervisors, and administrators through dedicated role-based interfaces.
3. **To improve the quality of graduation project proposals** by integrating an advisory **AI Proposal Assistant** that assists students in preparing well-structured project proposals before submission.
4. **To reduce unnecessary duplication of project ideas** by implementing a **Semantic Similarity Assistant** that evaluates the semantic similarity between proposed ideas and previously accepted projects or existing proposals.
5. **To automate administrative workflows** related to project requests, proposal evaluation, project approval, notifications, and milestone management in order to reduce manual effort and improve operational efficiency.
6. **To provide secure and controlled access** through university number authentication and role-based authorization mechanisms that protect system resources and academic information.

7. **To support effective project monitoring** by enabling supervisors and administrators to track project progress, monitor student activities, and manage project milestones throughout the development lifecycle.
8. **To develop a maintainable and extensible software solution** by adopting the Model–View–Controller (MVC) architectural pattern, modular software design principles, and modern web development technologies.
9. **To contribute to the digital transformation of graduation project management** by replacing fragmented manual procedures with an integrated academic information system that improves efficiency, transparency, and decision support.

1.4 Project Scope, Constraints, and Assumptions

1.4.1 Project Scope

The **Graduation Project Management Platform** is designed to support the complete lifecycle of graduation project management within the Faculty of Artificial Intelligence Engineering at the Syrian Private University. The platform provides a centralized web-based environment that integrates academic, administrative, and communication processes into a single information system.

The implemented system supports three primary categories of users: **Administrators**, **Supervisors**, and **Students**, each with dedicated responsibilities and role-based permissions. The platform enables project publication, project enrollment requests, student-proposed graduation ideas, proposal evaluation workflows, milestone management, secure messaging, announcements, notifications, document submission, project monitoring, and administrative management through structured workflows.

The project also incorporates two advisory Artificial Intelligence services that assist students during the proposal preparation phase. The **AI Proposal Assistant** supports students in generating well-structured project proposals, while the **Semantic Similarity Assistant** evaluates the semantic similarity between newly proposed ideas and previously accepted graduation projects or existing project proposals before submission. These services operate as advisory tools and do not perform automatic academic decision-making.

1.4.2 Project Constraints

The implementation of the proposed platform is subject to the following constraints:

- The system is developed as a **web-based application** and does not include native mobile applications.

- User authentication is based exclusively on **university numbers** assigned by the university.
- Artificial Intelligence services provide **advisory assistance only** and do not replace academic decision-making by supervisors or administrators.
- The platform operates within the academic environment of the Faculty of Artificial Intelligence Engineering and is designed according to its organizational workflow.
- The implementation is based on the Laravel framework and follows the **Model–View–Controller (MVC)** architectural pattern.
- External integration with university enterprise systems is outside the scope of the current project.

1.4.3 Project Assumptions

The project is developed based on the following assumptions:

- Each student possesses a valid university number and an active university account.
- Supervisors and administrators are responsible for maintaining accurate academic information within the system.
- Users have access to standard web browsers and a stable network connection.
- Artificial Intelligence services are available within the deployment environment and function as decision-support components.
- All academic decisions, including proposal approval, project acceptance, and evaluation, remain the responsibility of authorized university personnel.

1.5 Project Contribution

The **Graduation Project Management Platform** contributes to the digital transformation of graduation project management by replacing fragmented manual procedures with an integrated web-based information system that supports the complete graduation project lifecycle within a unified academic environment.

The platform provides a structured framework that improves collaboration among students, supervisors, and administrators by centralizing project-related activities, including project publication, enrollment requests, project proposal submission, milestone management, document handling, notifications, and communication. This integration reduces administrative overhead, improves workflow transparency, and enables more efficient monitoring of graduation projects throughout their lifecycle.

A significant contribution of the proposed system is the integration of advisory Artificial Intelligence services into the graduation project proposal process. The **AI Proposal Assistant** assists students in preparing structured project proposals, while the **Semantic Similarity Assistant** provides semantic similarity analysis between newly proposed ideas and existing graduation

projects or previously accepted proposals. Unlike fully automated decision-making systems, these services function as decision-support tools that assist users while preserving the authority of supervisors and administrators over all academic decisions.

Finally, the project provides a scalable and extensible foundation that can support future enhancements, including integration with additional university information systems, advanced analytics, mobile applications, and further Artificial Intelligence services, making it a sustainable solution for long-term academic use.

1.6 Research Methodology

The development of the **Graduation Project Management Platform** followed a systematic research methodology that combines analytical investigation with practical software engineering practices. The adopted methodology was designed to ensure that the proposed solution addresses real academic needs while maintaining consistency with established software development principles.

The research began with an analysis of the existing graduation project management process to identify its limitations, stakeholder requirements, and operational challenges. Information was collected through discussions with academic supervisors, analysis of current administrative procedures, and examination of existing graduation project management practices. This stage provided the foundation for defining both the functional and non-functional requirements of the proposed platform.

1.7 Report Organization

This thesis is organized into eight chapters, each addressing a specific aspect of the proposed **Graduation Project Management Platform**.

Chapter One introduces the project by presenting its background, problem statement, objectives, scope, contributions, research methodology, report organization, and fundamental concepts.

Chapter Two describes the development methodology and project management practices adopted throughout the project. It presents the project planning process, work breakdown structure, schedule management, risk management, quality assurance activities, software configuration management, and stakeholder analysis.

Chapter Three provides the theoretical background and reviews related work relevant to graduation project management systems, web-based information systems, workflow management, and Artificial Intelligence technologies employed in the project.

Chapter Four presents the system requirements analysis and modeling. It identifies functional and non-functional requirements, analyzes stakeholders, and documents the system behavior using standard software engineering models such as use case diagrams, activity diagrams, sequence diagrams, and requirements traceability.

Chapter Five discusses the architectural and detailed design of the proposed system. It explains the overall software architecture, database design, class design, security mechanisms, Artificial Intelligence integration, and application programming interfaces.

Chapter Six describes the implementation of the proposed platform, including the development environment, technologies, software components, database implementation, user interface development, and implementation of the system modules.

Chapter Seven presents the testing and evaluation activities conducted to validate the implemented system. It describes the testing strategy, test cases, testing results, performance evaluation, and verification of the implemented requirements.

Chapter Eight summarizes the project outcomes by presenting the obtained results, evaluating the effectiveness of the proposed solution, discussing project limitations, and outlining recommendations and future improvements.

The thesis concludes with the references used throughout the study and the appendices, which include supplementary materials that support the implementation and documentation of the project.

1.8 Definitions and Basic Concepts

To facilitate understanding of the terminology used throughout this thesis, the following definitions present the fundamental concepts related to the proposed **Graduation Project Management Platform**.

2 Graduation Project

A graduation project is a comprehensive academic project undertaken by undergraduate students to demonstrate the knowledge and practical skills acquired throughout their academic program. It represents the final stage of the study plan and requires students to analyze, design, implement, and evaluate a software solution under the supervision of academic staff.

3 Graduation Project Management Platform

The Graduation Project Management Platform is a centralized web-based information system developed to manage the complete graduation project lifecycle. It supports project publication, project enrollment, proposal submission, project supervision, milestone management,

communication, document management, and administrative activities through an integrated environment.

4 Administrator

An Administrator is an authorized user responsible for managing system configuration, user accounts, academic information, project data, and overall platform administration. Administrators supervise system operations and ensure that academic workflows are executed correctly.

5 Supervisor

A Supervisor is an academic staff member responsible for publishing graduation projects, reviewing project requests and proposed ideas, supervising student teams, monitoring project progress, and evaluating submitted milestones throughout the project lifecycle.

6 Student

A Student is an authorized user who interacts with the platform to browse available graduation projects, submit project enrollment requests, propose new project ideas, communicate with supervisors, manage project deliverables, and monitor project progress.

7 Artificial Intelligence Proposal Assistant

The AI Proposal Assistant is an advisory service integrated into the platform that assists students in transforming initial project ideas into structured graduation project proposals. The generated content serves as academic guidance and remains subject to student modification and supervisor evaluation.

8 Semantic Similarity Assistant

The Semantic Similarity Assistant is an advisory Artificial Intelligence service that compares newly proposed graduation project ideas with existing projects and previously accepted proposals using semantic similarity analysis. Its purpose is to assist users in identifying conceptually similar ideas before submission without making automatic acceptance or rejection decisions.

9 Role-Based Access Control (RBAC)

Role-Based Access Control is a security mechanism that restricts access to system resources according to predefined user roles and permissions. Each user can access only the functions associated with their assigned responsibilities within the platform.

10 University Number Authentication

University Number Authentication is the authentication mechanism adopted by the platform, where each user accesses the system using an officially assigned university number together with secure authentication credentials.

11 Workflow

A workflow represents the predefined sequence of activities and decision points through which graduation project requests, proposals, approvals, milestone submissions, and other academic processes progress within the platform.

Chapter 2: Development Methodology & Project Management

2.1 Selected Development Methodology and Justifications

The development of the **Graduation Project Management Platform** adopted an **iterative and incremental software development methodology**. This approach was selected because the project was developed over two academic semesters, allowing continuous refinement of the system based on implementation progress, technical discoveries, and supervisor feedback. Rather than implementing the entire system in a single development cycle, the project evolved through multiple iterations, where each iteration introduced new functionality, improved existing components, and addressed limitations identified during previous stages.

During the initial phase of the project, emphasis was placed on understanding the graduation project management process, identifying stakeholder requirements, studying related systems, and preparing the initial software architecture. A preliminary implementation was then developed to validate the proposed concepts and establish the core structure of the platform.

In the subsequent development phase, the project underwent significant technical enhancements. These included restructuring the relational database, improving the software architecture, upgrading the application framework to newer Laravel releases, ensuring compatibility with PHP 8.5, strengthening authentication and authorization mechanisms, implementing additional system modules, integrating advisory Artificial Intelligence services, and performing comprehensive testing and validation. Each enhancement was incorporated incrementally while preserving the stability of the existing system.

The iterative and incremental approach proved appropriate for this project because it enabled continuous evaluation and improvement throughout the development lifecycle. Design decisions could be revisited whenever necessary, implementation risks were reduced by introducing features gradually, and feedback received during development could be incorporated without requiring fundamental redesign of the entire system. This methodology ultimately contributed to the development of a stable, maintainable, and extensible graduation project management platform.

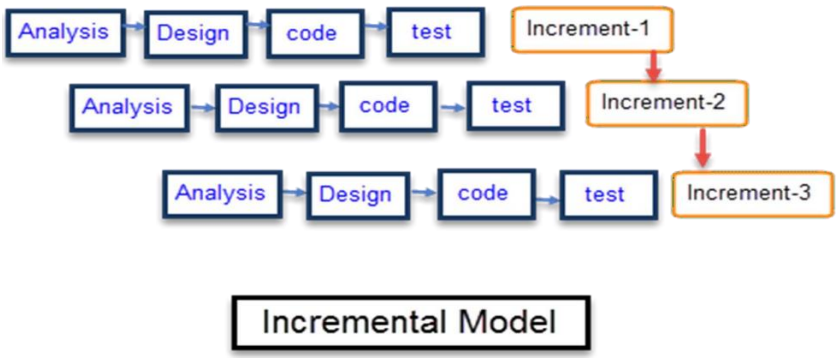


Figure 1 iterative and incremental software development methodology

2.2 Project Timeline (Gantt Chart)

The development of the **Graduation Project Management Platform** was planned and executed over two consecutive academic semesters. The project schedule was organized into a series of interconnected development activities that followed the selected iterative and incremental methodology. Rather than completing each phase independently, several activities overlapped to allow continuous refinement, validation, and integration throughout the project lifecycle.

During the first academic semester, the project focused on identifying the problem domain, reviewing related studies, eliciting and analyzing system requirements, designing the software architecture and database, and developing the initial functional prototype. The outcomes of this phase established the technical foundation required for the subsequent implementation activities.

The second academic semester concentrated on transforming the prototype into a complete and production-ready academic system. Major activities included refining the database schema, improving authentication and authorization mechanisms, completing the remaining functional modules, integrating Artificial Intelligence services, conducting comprehensive testing and validation, and preparing the final technical documentation. Continuous supervisor feedback and periodic evaluations were incorporated throughout this phase to ensure that the implemented system satisfied the specified project objectives and quality requirements.

The overall project schedule is illustrated in **Figure 2**, which presents the chronological distribution of the principal development activities throughout the two-semester development period.

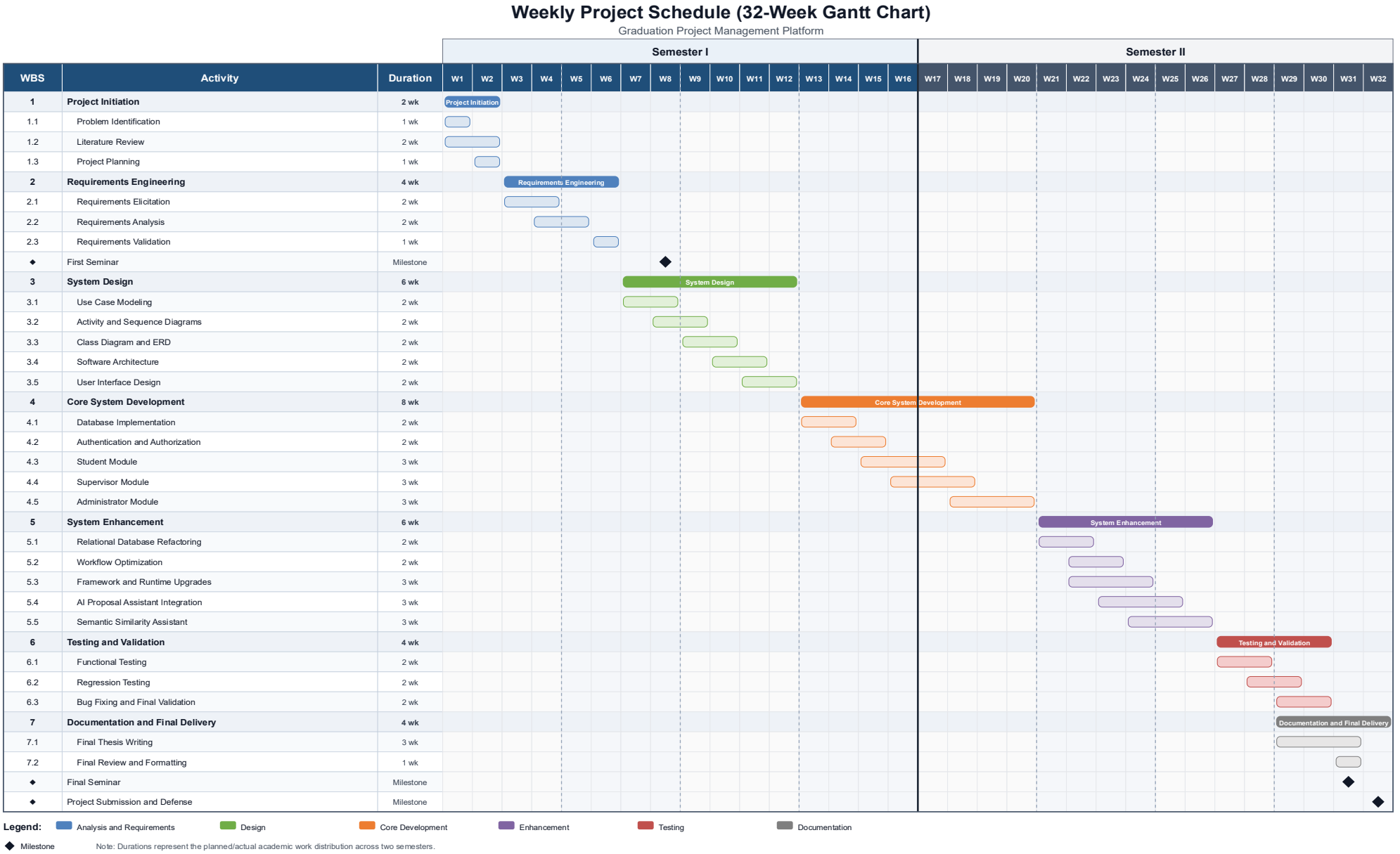


Figure 2 Project Timeline Gant

2.3 Risk Management

Software development projects are inherently associated with technical and managerial risks that may affect project quality, implementation progress, and overall system stability. Throughout the development of the **Graduation Project Management Platform**, potential risks were continuously identified, evaluated, and mitigated using preventive and corrective measures. Since the project followed an iterative and incremental development methodology, risk management was integrated into the development process rather than treated as a separate activity.

Table 1 summarizes the principal risks identified during the project together with their corresponding mitigation strategies.

Table 1 principal risks

Risk ID	Risk	Probability	Impact	Mitigation Strategy
R1	Database schema modifications	High	High	Controlled refactoring using Laravel migrations and validation tests.
R2	Framework and PHP upgrades	Medium	High	Incremental upgrades with regression testing.
R3	Requirement evolution	Medium	Medium	Iterative planning and continuous refinement.
R4	AI service integration	Medium	Medium	Independent integration and validation before deployment.
R5	Regression after major modifications	Medium	High	Regression testing after each major change.
R6	Authentication and authorization changes	Low	High	RBAC, middleware validation, and functional testing.

2.4 Team Structure

The development of the **Graduation Project Management Platform** was carried out by a two-member software engineering team under the continuous supervision of the academic project supervisor. Each team member assumed specific technical responsibilities while collaborating throughout all phases of the software development lifecycle, including analysis, design, implementation, testing, documentation, and project evaluation.

The overall project team structure is illustrated in **Figure 3**.

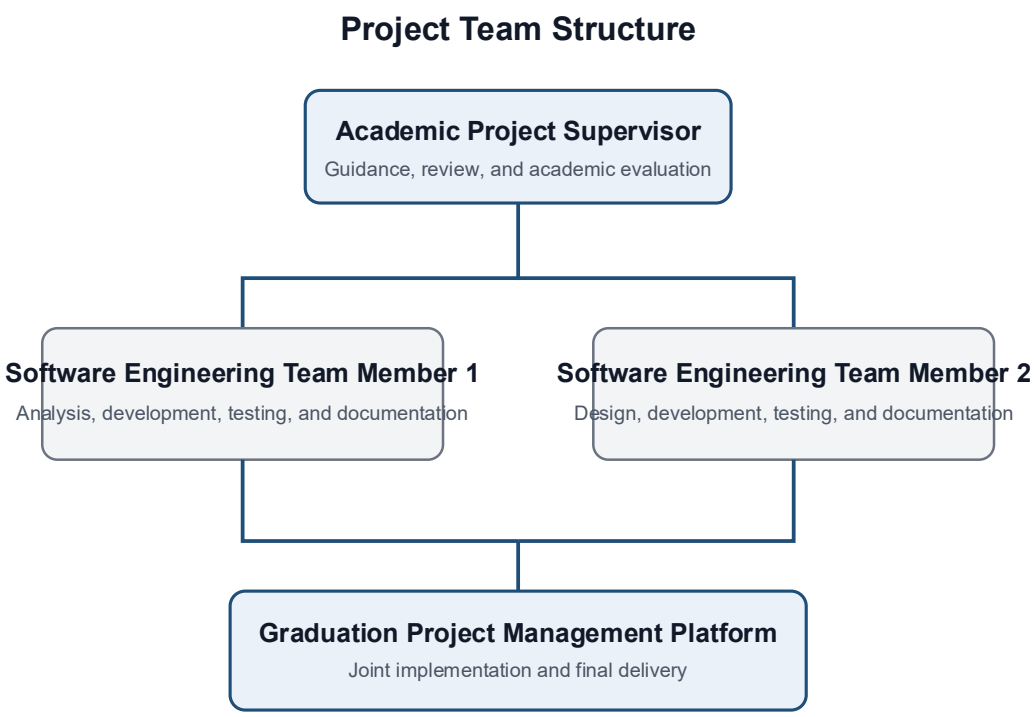


Figure 3 Project Team Structure

2.5 Quality Management

Software quality management was considered a fundamental aspect throughout the development of the **Graduation Project Management Platform**. The primary objective was to ensure that the developed system satisfied the specified functional and non-functional requirements while maintaining reliability, maintainability, security, and usability. Quality management activities were integrated into every stage of the software development lifecycle rather than being limited to the final testing phase.

To achieve these objectives, continuous verification and validation activities were performed during system development. Each completed software component was evaluated before proceeding to subsequent implementation stages, ensuring that newly introduced features did not negatively affect previously implemented functionality. This approach contributed to maintaining system consistency and reducing the likelihood of defects accumulating during development.

2.5.1 Quality Assurance Strategies

Several quality assurance strategies were adopted throughout the project to improve software reliability and implementation quality.

Continuous Testing:
System functionality was verified continuously during development. Individual modules were tested immediately after implementation, followed by regression testing whenever significant architectural or functional modifications were introduced.

Manual Functional Testing:
Manual testing was performed to validate the correctness of user interfaces, authentication procedures, authorization rules, project workflows, database operations, and user interactions. Test scenarios were executed using different user roles to ensure that system behavior matched the specified requirements.

Automated Testing:
Laravel's testing framework was utilized to verify critical system functionality. Automated feature tests were executed throughout development to validate application behavior after major software modifications and framework upgrades.

Version Control Verification:
Git and GitHub were employed to track software evolution and support controlled implementation changes. Each completed modification was committed separately, enabling traceability and simplifying rollback if unexpected issues were identified.

Supervisor Review and Continuous Feedback:
Periodic discussions with the academic supervisor provided continuous evaluation of project progress, implementation decisions, and documentation quality. Feedback received during these reviews was incorporated into subsequent development iterations to improve the overall quality of the final system.

Collectively, these quality assurance strategies contributed to the development of a stable, maintainable, and reliable graduation project management platform that satisfies both the defined project objectives and the academic quality requirements.

2.6 Project Management Tools Used

The development of the **Graduation Project Management Platform** was supported by a collection of software development and project management tools that facilitated implementation, version control, testing, documentation, and collaboration throughout the project lifecycle. Each tool was selected according to its role in improving development efficiency, software quality, and project organization.

The primary development environment consisted of **Visual Studio ,** which were used for source code development, debugging. The Laravel framework served as the core application framework, while **Composer** was utilized for dependency management and package installation.

Version control and software evolution were managed using **Git** and **GitHub**, enabling systematic tracking of implementation changes, branch management, and project version history throughout the development process.

Database development and management were performed using **SQLite**, supported by Laravel migrations and the Eloquent ORM to ensure consistent schema evolution and simplified database operations.

API testing and functional verification were conducted using **Postman**, while the built-in Laravel testing framework was employed for automated feature testing and regression validation.

Project documentation, software diagrams, and system models were prepared using **Microsoft Word**, **draw.io**, and **Visual Paradigm**, enabling the production of standardized software engineering documentation and UML diagrams.

The software development and project management tools used throughout the project are summarized in **Table 2**.

Table 2 Project Management and Development Tools

Tool	Purpose
Visual Studio Code	Source code editing and debugging
Laravel	Web application framework
Composer	Dependency management
Git	Version control
GitHub	Source code hosting and collaboration
Postman	API testing
Microsoft Word	Thesis documentation
draw.io	Software diagrams
Visual Paradigm	UML modeling

2.7 Monitoring and Continuous Evaluation Mechanisms

Continuous monitoring and evaluation were performed throughout the development of the **Graduation Project Management Platform** to ensure that project activities remained aligned with the defined objectives, technical requirements, and academic milestones. Progress was regularly assessed through implementation reviews, functional verification, and supervisor feedback, enabling the early identification and correction of issues before they affected subsequent development stages.

The iterative and incremental development methodology facilitated continuous monitoring by dividing the project into manageable development iterations. At the end of each iteration, completed functionalities were reviewed to verify that they satisfied the intended requirements and maintained compatibility with previously implemented modules.

Project evaluation was also supported by systematic version control practices. Source code changes were committed incrementally, allowing development progress to be tracked and reviewed throughout the project lifecycle. This approach simplified the identification of implementation changes and provided a documented history of software evolution.

Regular meetings with the academic supervisor constituted an essential component of the evaluation process. During these meetings, project progress, implementation decisions, documentation quality, and completed functionalities were discussed. Feedback obtained from these reviews was incorporated into subsequent development iterations, contributing to continuous improvement of both the software product and the accompanying technical documentation.

In addition, functional verification and regression testing were performed following major architectural modifications, framework upgrades, and feature integrations. These evaluation activities ensured that previously implemented functionality remained operational while newly introduced features satisfied the specified requirements.

2.8 Software Configuration Management(SCM)

Software Configuration Management (SCM) was applied throughout the development of the **Graduation Project Management Platform** to maintain software integrity, support controlled evolution, and ensure traceability of implementation changes. Configuration management practices were integrated into the development workflow to minimize inconsistencies, preserve system stability, and facilitate collaborative software development.

The project utilized **Git** as the distributed version control system and **GitHub** as the centralized source code repository. All significant implementation changes were recorded through incremental commits, providing a documented history of software evolution and enabling the development team to track modifications throughout the project lifecycle.

Branches were employed to isolate major development activities and experimental modifications from the primary development line. This branching strategy allowed new features, framework upgrades, and architectural improvements to be implemented and validated independently before being incorporated into the main project version. Such an approach reduced the risk of introducing unstable changes into the working system while supporting continuous software evolution.

Configuration management also included the use of Laravel migration files to maintain consistent database schema evolution. Rather than modifying the database structure manually, all schema changes were implemented through version-controlled migrations, ensuring reproducibility and simplifying database synchronization across development environments.

Dependency management was performed using **Composer**, allowing external packages and framework dependencies to be maintained consistently throughout the project. This ensured compatibility between software components and simplified framework upgrades during later development stages.

Collectively, these configuration management practices enhanced software maintainability, improved traceability, facilitated controlled project evolution, and contributed to the successful completion of a reliable and well-documented software system.

Chapter 3: Theoretical Framework and Related Studies

3.1 Previous Studies in the Field of Intelligent and Similar Systems

3.1.1 Studies on Graduation Project Management Systems

Study 1: Design and Research of University Students' Graduation Project Management System Based on Outcomes-Based Education (2024)

Su et al. (2024) proposed a web-based graduation project management system based on the Outcomes-Based Education (OBE) approach. The system supports project proposal submission, project selection, supervision, and report management to improve project administration and student learning outcomes. However, the study focuses on workflow management and does not integrate Artificial Intelligence techniques such as semantic similarity analysis or AI-assisted proposal preparation.

Study 2: Design of University Graduation Project Management System Based on Microservice Architecture (2021)

Li et al. (2021) developed a graduation project management system using a microservice architecture with Spring Boot and MySQL. The system automates major academic processes, including project declaration, proposal approval, thesis submission, defense management, and document archiving, while improving scalability and maintainability. Nevertheless, the study emphasizes administrative workflow automation without providing intelligent decision support or AI-based semantic analysis.

Study 3: Cloud Based System for Streamlined Final Year Project Allocation and Tracking With Deduplication (2024)

Abinaya and Meenatchi (2024) introduced a cloud-based platform for final-year project allocation and tracking. The system supports project recommendation, team matching, progress monitoring, and duplicate project detection to improve collaboration among students, supervisors, and coordinators. Although it includes project deduplication, it does not employ semantic similarity techniques or AI-assisted proposal evaluation.

Study 4: A Web-Based FSKTM Final Year Project Management System (2025)

Arumugam and Ishak (2025) developed a web-based Final Year Project Management System to automate project tracking and improve communication between students, supervisors, and project coordinators. The system provides role-based access control, progress tracking, deadline reminders, and reporting features to enhance project administration. Nevertheless, it focuses on workflow automation and does not integrate Artificial Intelligence services such as semantic similarity analysis or proposal assistance.

Study 5: Prototype Development of Final Year Project Management System to Monitor Student's Performance (2024)

Isa et al. (2024) developed a web-based Final Year Project Management System (FYPMS) to improve project supervision and monitor students' academic performance. The system provides a centralized dashboard for students, supervisors, and coordinators, enabling progress tracking, communication, and deadline management. The study demonstrated improvements in project monitoring and administrative efficiency. However, it focuses on project monitoring and does not integrate Artificial Intelligence features such as semantic similarity analysis or intelligent proposal assistance.

Study 6: Development of Sentence Similarity Detection Application with Semantic Similarity and Machine Learning Approaches (2025)

Rahman et al. (2025) proposed an intelligent application for detecting semantic similarity between undergraduate thesis titles using Sentence-BERT and machine learning techniques. The system achieved high accuracy in identifying similar topics and demonstrated its effectiveness in supporting academic quality assurance. Nevertheless, the proposed solution is dedicated to similarity detection only and does not provide a complete graduation project management environment that integrates proposal submission, supervision, workflow management, and AI-assisted academic services within a unified platform.

3.2 Modern Technologies and Frameworks in the Field

Web-based information systems

Web-based information systems have become the standard solution for managing academic and administrative processes in higher education institutions. These systems provide centralized access through web browsers, enabling students, supervisors, and administrators to perform their tasks from different locations without requiring software installation. Compared with traditional desktop applications, web-based systems offer better accessibility, scalability, maintainability, and ease of deployment, making them suitable for managing graduation project workflows.

Laravel Framework and MVC Architecture

Laravel is one of the most widely adopted PHP frameworks for modern web application development. It follows the Model–View–Controller (MVC) architectural pattern, which separates business logic, user interfaces, and data management into independent components. This separation improves code maintainability, scalability, reusability, and simplifies future system enhancements.

Relational Database Management Systems

Relational Database Management Systems (RDBMS) organize data into related tables connected through primary and foreign keys, ensuring data consistency and integrity. They support efficient data storage, querying, and transaction management, making them suitable for academic management systems that handle students, supervisors, projects, requests, and administrative records.

Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is a security model that restricts system access according to predefined user roles. Instead of assigning permissions directly to individual users, permissions are assigned to roles such as Administrator, Supervisor, and Student. This approach simplifies permission management, enhances security, and ensures that each user can access only the functions relevant to their responsibilities.

Semantic similarity analysis

Semantic similarity analysis measures the conceptual similarity between textual documents based on their meaning rather than exact keyword matching. Modern Natural Language Processing (NLP) models transform text into numerical embeddings that capture semantic relationships, allowing systems to identify conceptually similar project proposals even when different wording is used.

Artificial Intelligence in Academic Management Systems

Artificial Intelligence has become an important component of modern educational management systems by providing intelligent decision support and automating complex tasks. AI technologies can assist students during proposal preparation, improve document analysis, support recommendation systems, and reduce administrative effort. Recent advances in Large Language Models (LLMs) have further expanded AI capabilities, enabling more interactive and context-aware academic assistance.

3.3 Comparative Analysis of Existing Systems and Products

After reviewing the previous studies and modern technologies, a comparative analysis was conducted to evaluate existing graduation project management systems and identify their capabilities and limitations. The comparison focuses on the main features relevant to the proposed Graduation Project Management Platform, including project management, workflow automation, role management, Artificial Intelligence support, and semantic similarity analysis. This analysis highlights the existing research gap and justifies the need for the proposed system.

Feature	OBE-Based GPMS (2024)	Microservice GPMS (2021)	Cloud FYPMS (2024)	Web FYPMS (2025)	Proposed Platform
Web-based platform	✓	✓	✓	✓	✓
Student management	✓	✓	✓	✓	✓
Supervisor management	✓	✓	✓	✓	✓
Graduation project management	✓	✓	✓	✓	✓
Team management	✓	✓	✓	✓	✓
Workflow automation	✓	✓	✓	✓	✓
Progress tracking	✓	✓	✓	✓	✓
Role-based access control	✓	✓	✓	✓	✓
AI-assisted proposal support	✗	✗	✗	✗	✓
Semantic similarity analysis	✗	✗	✗	✗	✓
Local AI processing	✗	✗	✗	✗	✓

Table 3 Comparative analysis of existing systems and the proposed platform

3.4 Research Gap and Proposed Innovation

The review of previous studies and existing graduation project management systems indicates that considerable progress has been made in automating academic workflows, project allocation, supervision, and progress tracking. Most existing solutions successfully improve administrative efficiency by providing centralized web-based platforms for managing graduation projects and facilitating communication between students, supervisors, and academic coordinators.

Despite these advances, several limitations remain. Existing systems primarily focus on workflow automation and administrative management while providing limited intelligent support during project proposal preparation. Furthermore, most systems rely on traditional duplicate detection techniques and do not employ semantic similarity analysis to identify conceptually similar project ideas. In addition, AI capabilities are generally absent or implemented as standalone components rather than being integrated into a complete graduation project management environment.

To address these limitations, the proposed **Graduation Project Management Platform** integrates conventional graduation project management functions with Artificial Intelligence services within a single web-based platform. In addition to supporting project management, supervision, team formation, and workflow automation, the platform incorporates AI-assisted proposal support and semantic similarity analysis to help students prepare higher-quality project proposals and reduce the submission of conceptually similar project ideas. This integration provides a more intelligent and comprehensive solution for managing graduation projects while maintaining a unified environment for students, supervisors, and administrators.

3.5 Theoretical Framework and Reference Studies

From an Artificial Intelligence perspective, the platform utilizes Natural Language Processing (NLP) concepts to analyze textual project proposals. Semantic similarity analysis is achieved by transforming textual content into vector embeddings and comparing their semantic relationships using similarity measurement techniques. This approach enables the identification of conceptually similar project ideas regardless of differences in wording.

Furthermore, the platform applies the principles of Role-Based Access Control (RBAC) to regulate user permissions according to predefined roles, ensuring secure access to system resources while supporting collaboration among students, supervisors, and administrators.

3.6 Modern and Future Trends in the Field

Recent advances in information technology and Artificial Intelligence have significantly influenced the development of academic management systems. Modern educational platforms are increasingly incorporating intelligent technologies to improve decision-making, automate administrative processes, and enhance the overall user experience. Among these technologies, Natural Language Processing (NLP), Large Language Models (LLMs), semantic search, and intelligent recommendation systems have become key research areas for supporting academic activities.

Cloud computing has also enabled educational institutions to deploy scalable and accessible management systems that support collaboration among students, supervisors, and administrators regardless of location. In addition, the growing adoption of AI-powered assistants is expected to simplify project proposal preparation, provide personalized recommendations, and improve academic guidance throughout the graduation project lifecycle.

Another important trend is the increasing use of semantic analysis techniques to evaluate textual similarity based on meaning rather than keyword matching. These techniques help reduce duplicate project submissions, improve project quality, and support more informed academic decision-making.

Future graduation project management systems are expected to integrate advanced AI capabilities, predictive analytics, intelligent workflow automation, and explainable AI models. These technologies will enable educational institutions to provide more personalized, efficient, and intelligent services while maintaining data security and supporting continuous improvement in academic project management.

Chapter 4: Requirements Modeling and Analysis

4.1 Feasibility Study

4.1.1 Technical Feasibility

The proposed Graduation Project Management Platform is technically feasible because it is implemented using mature and widely adopted technologies. The system is developed as a web-based application using the Laravel framework following the MVC architectural pattern, with PHP as the backend programming language, Blade for the presentation layer, and a relational database for data management. The implemented architecture supports multiple user roles, secure authentication, workflow automation, and AI-assisted services while remaining maintainable and scalable. In addition, the required software tools, development environment, and hardware resources are readily available, making the implementation technically achievable.

4.1.2 Operational Feasibility

The system is operationally feasible because it supports the actual graduation project workflow followed within the faculty. It enables students, supervisors, and administrators to perform their responsibilities through a centralized web interface, reducing manual paperwork and improving communication. The platform automates common academic processes such as project publication, project requests, project idea submission, supervisor review, progress monitoring, messaging, and project management. Since each user interacts only with functions related to their role, the platform is easy to learn and suitable for daily academic use.

4.1.3 Economic Feasibility

The proposed system can be deployed using open-source technologies, including Laravel, PHP, and a relational database management system, minimizing software licensing costs. The platform also reduces administrative effort by digitizing manual processes, decreasing paperwork, improving record management, and reducing the time required for project administration. Consequently, the operational benefits outweigh the development and deployment costs.

4.2 Requirements Gathering

The requirements of the proposed Graduation Project Management Platform were collected through multiple elicitation techniques to ensure that the developed system satisfies the actual needs of the Faculty of Information Technology. Information was gathered from different stakeholders, including students, supervisors, and academic staff, to identify the functional and non-functional requirements of the platform.

4.2.1 Interviews

Interviews were conducted with the project supervisor and several faculty members to understand the current graduation project management process, identify existing challenges, and determine the expected functionalities of the proposed platform. The interviews also helped define the responsibilities of different user roles and validate the overall system workflow.

4.2.2 Observation

Direct observation was used to study the existing graduation project management procedures within the faculty. The manual handling of project announcements, project selection, proposal submission, communication, and progress tracking was observed to identify inefficiencies and opportunities for automation.

4.2.3 Document Analysis

Existing graduation project forms, regulations, academic procedures, and faculty guidelines were analyzed to identify the required workflow, approval process, and project management rules. This analysis ensured that the proposed system complies with the faculty's graduation project procedures.

4.2.4 Requirement Validation

The collected requirements were reviewed and refined through continuous discussions with the project supervisor during the iterative development process. Feedback obtained throughout the project was incorporated into subsequent development iterations to ensure that the implemented system accurately reflects stakeholder expectations and project objectives.

4.3 Stakeholder

The success of the proposed Graduation Project Management Platform depends on the collaboration of several stakeholders, each having specific responsibilities and system interactions. Identifying these stakeholders helps define system requirements, user roles, and access privileges.

Table 4 Stakeholders and Their Responsibilities

Stakeholder	Main Responsibilities
Student	Browse projects, submit project requests, submit project ideas, communicate with supervisors, track progress, receive notifications, use AI proposal assistance and semantic similarity analysis.
Supervisor	Publish projects, review requests and ideas, supervise students, evaluate progress, communicate with students, manage projects.
Administrator	Manage users, projects, supervisors, students, system settings, and overall platform administration.

4.4 Functional Requirements

The functional requirements are organized into five categories: platform access and authentication, Administrator functions, Student functions, Supervisor functions, and shared workflow and system functions.

4.4.1 Platform Access and Authentication Requirements

FR1: The system shall display a public landing page containing information about the platform and access to the authentication functions.

FR2: The system shall allow registered users to log in using their university number and password.

FR3: The system shall allow authenticated Students, Supervisors, and Administrators to log out securely.

FR4: The system shall allow a user to request password recovery using the registered university number.

FR5: The system shall allow a user to reset the account password using a valid reset token and university number.

FR6: The system shall require a Student or Supervisor whose account does not contain an email address to complete the missing email information before accessing the protected portal.

FR7: The system shall prevent inactive user accounts from logging in or continuing to access protected areas.

FR8: The system shall allow authenticated Students, Supervisors, and Administrators to view and manage their internal notifications.

4.4.2 Administrator Functional Requirements

FR9: The system shall display an Administrator dashboard containing platform summary indicators.

FR10: The system shall allow the Administrator to list and inspect user accounts.

FR11: The system shall allow the Administrator to create Student accounts.

FR12: The system shall allow the Administrator to create Supervisor accounts together with their linked Supervisor profiles.

FR13: The system shall allow the Administrator to activate or deactivate existing user accounts.

FR14: The system shall allow the Administrator to reset the password of another user account.

FR15: The system shall allow the Administrator to view all graduation projects in read-only mode.

FR16: The system shall allow the Administrator to view all project join requests in read-only mode.

FR17: The system shall allow the Administrator to view all submitted project ideas in read-only mode.

FR18: The system shall allow the Administrator to view all milestone submissions in read-only mode.

FR19: The system shall allow the Administrator to view the recorded activity audit log.

4.4.3 Student Functional Requirements

FR20: The system shall display a Student dashboard adapted to the Student's discovery, pending-application, or enrolled-project state.

FR21: The system shall allow eligible Students to browse available graduation projects and related Supervisors.

FR22: The system shall allow an eligible Student to submit a request to join an existing graduation project together with the proposed team-member information.

FR23: The system shall allow an eligible Student to submit a new graduation project idea containing a title, optional proposal description, selected Supervisor, and proposed team members.

FR24: The system shall allow a Student to track the current status of a submitted project join request.

FR25: The system shall allow a Student to track the current status of a submitted project idea.

FR26: The system shall allow an enrolled Student to view the assigned graduation project details.

FR27: The system shall allow an enrolled Student to view the members of the assigned project team.

FR28: The system shall allow an enrolled Student to view the project timeline, milestone dates, and calculated project progress.

FR29: The system shall allow an enrolled Student to upload a file for an authorized project milestone.

FR30: The system shall allow an enrolled Student to download authorized submission files belonging to the assigned project.

FR31: The system shall allow a Student to send a message to the responsible Supervisor.

FR32: The system shall allow a Student to view Supervisor replies and announcements related to the assigned project.

FR33: The system shall allow a Student to change the current account password.

FR34: The system shall allow a Student to generate an advisory AI-assisted project-proposal suggestion from an initial idea description.

FR35: The system shall allow a Student to review, edit, and store an optional proposal description with the submitted project idea.

FR36: The system shall allow a Student to perform an advisory semantic-similarity analysis before submitting the final project idea.

4.4.4 Supervisor Functional Requirements

FR37: The system shall display a Supervisor dashboard containing the Supervisor's projects, applications, ideas, submissions, and communication records.

FR38: The system shall allow a Supervisor to create new graduation projects and update projects owned by that Supervisor.

FR39: The system shall allow a Supervisor to view owned graduation projects and their enrolled team members.

FR40: The system shall allow the responsible Supervisor to accept or reject pending project join requests associated with owned projects.

FR41: The system shall allow the responsible Supervisor to accept or reject pending project ideas assigned to that Supervisor.

FR42: The system shall allow a Supervisor to reply to Student messages assigned to that Supervisor.

FR43: The system shall allow a Supervisor to publish announcements for teams enrolled in owned projects.

FR44: The system shall allow the responsible Supervisor to review milestone submissions, provide feedback, and mark them as Approved or Needs Revision.

FR45: The system shall allow a Supervisor to download submission files belonging to supervised projects.

FR46: The system shall allow a Supervisor to change the current account password.

FR47: The system shall allow the responsible Supervisor to view the complete proposal description submitted with a project idea.

FR48: The system shall allow the responsible Supervisor to view the stored advisory semantic-similarity snapshot associated with a submitted project idea.

4.4.5 Shared Workflow and System Functional Requirements

FR49: The system shall enforce workflow-protection rules before executing critical project, application, enrollment, and submission operations.

FR50: The system shall prevent a Student from belonging to more than one active graduation project.

FR51: The system shall prevent Students and proposed team members from maintaining conflicting pending project requests or project ideas.

FR52: The system shall maintain consistency among project lifecycle state, application decisions, official team membership, and enrollment status.

FR53: The system shall protect milestone-submission files from unauthorized upload, viewing, or download operations.

FR54: The system shall record selected administrative and workflow operations in the activity audit log.

FR55: The system shall generate internal notifications for supported project, application, submission, review, and communication events.

FR56: The system shall generate and store a privacy-safe, server-calculated semantic-similarity snapshot when the final project idea is submitted.

4.5 Non-Functional Requirements

The non-functional requirements define the quality attributes, operational constraints, security controls, reliability mechanisms, usability characteristics, and maintainability practices of the Graduation Project Management Platform.

Table 5 Non-Functional Requirements of the Graduation Project Management Platform

ID	Category	Non-Functional Requirement	Verification Criterion
NFR-PERF-01	Performance	The Administrator dashboard shall retrieve its principal indicators using aggregated database queries rather than loading complete record collections into application memory.	Source-code inspection confirms the use of database-level counting and aggregation for the displayed summary indicators.
NFR-PERF-02	Performance	The administrative activity log shall use pagination to restrict the number of records displayed in one request.	The activity-log interface retrieves and displays records in pages containing 25 records.
NFR-PERF-03	Performance	The AI Proposal Assistant and Semantic Similarity Assistant shall apply request-rate limits to reduce excessive repeated model execution.	Proposal-assistance requests are limited to 10 requests per minute per Student, while similarity requests are limited to 6 requests per minute per Student.
NFR-PERF-04	Performance	Artificial intelligence operations shall use bounded inputs, configurable timeouts, controlled embedding batches, and a limited result set.	Proposal input is bounded by configuration, AI timeouts are configurable, embedding requests use controlled processing, and similarity responses display a maximum of five matches by default.
NFR-SEC-01	Security	User passwords shall be stored using Laravel's configured one-way password-hashing mechanism, and authentication shall use the university number and password.	Database inspection confirms that passwords are not stored in plain text, while login requests use the university number as the account identifier.
NFR-SEC-02	Security	Protected portals shall require an authenticated, active, email-complete user assigned to the correct operational role.	Authentication, account-status, email-completion, and role middleware are applied before access to protected Student, Supervisor, and Administrator routes.
NFR-SEC-03	Security	State-changing web forms shall use Cross-Site Request Forgery protection, and logout shall invalidate the authenticated session.	Laravel CSRF tokens protect state-changing forms, while logout invalidates the session and regenerates the CSRF token.
NFR-SEC-04	Security	Milestone-submission files shall be privately stored, validated by supported type and size, and downloaded only after resource-level authorization.	Submission files are stored outside direct public access, and download routes verify project membership or Supervisor ownership before returning a file.
NFR-USE-01	Usability	Students, Supervisors, and Administrators shall access the platform through one authentication entry point and be redirected to the portal corresponding to their role.	Manual role-based verification confirms that valid accounts use the same login interface and reach the correct role dashboard.

ID	Category	Non-Functional Requirement	Verification Criterion
NFR-USE-02	Usability	The Student dashboard shall adapt to discovery, pending-application, and enrolled-project states and shall display understandable workflow-status information.	The interface conditionally presents project browsing, application tracking, or the enrolled-project workspace according to the resolved Student state.
NFR-USE-03	Usability	Forms and workflow pages shall provide visible labels, validation feedback, status indicators, empty states, and rejection or review reasons where applicable.	Manual interface verification confirms that invalid input and workflow outcomes are communicated through contextual messages and status elements.
NFR-USE-04	Usability	Artificial intelligence output shall be clearly identified as advisory and shall require explicit Student action before being applied or submitted.	Generated proposals provide an explicit Apply action, similarity results contain advisory wording, and neither function submits or decides an idea automatically.
NFR-REL-01	Reliability	Shared workflow guards shall prevent duplicate pending applications and multiple active project enrollments.	Automated tests and source-code inspection verify that conflicting requests, ideas, and active project memberships are rejected.
NFR-REL-02	Reliability	Critical application, enrollment, decision, and submission operations shall preserve database and lifecycle consistency.	Related multi-record changes use validation, relational constraints, and database transactions where partial execution could create inconsistent records.
NFR-REL-03	Reliability	Unavailability, timeout, or invalid output from the local artificial intelligence service shall not prevent the primary project-idea submission workflow.	Proposal failure returns an editable fallback, while similarity failure returns an unavailable status without blocking final idea submission.
NFR-REL-04	Reliability	The complete automated regression suite shall pass before the final software baseline is accepted.	The final verified execution result must contain no failing automated tests before delivery.
NFR-MAIN-01	Maintainability	The application shall follow Laravel's Model–View–Controller separation among request handling, persistent models, and interface rendering.	The project structure separates controllers, Eloquent models, and Blade views into their corresponding Laravel directories.
NFR-MAIN-02	Maintainability	Shared workflow, enrollment-state, activity-logging, proposal-assistance, and similarity-processing logic shall be organized into reusable service classes.	Source-code inspection confirms the use of services including WorkflowGuard, StudentEnrollmentService, ActivityLogger, ProjectProposalAssistantService, and ProjectSimilarityService.
NFR-MAIN-03	Maintainability	Database evolution shall be managed through version-controlled migrations, Eloquent relationships, factories, and seeders.	Database tables, constraints, relationships, and controlled data preparation are reproducible through the Laravel project files.
NFR-MAIN-04	Maintainability	Environment-dependent settings shall be configurable without modifying the principal application source code.	Database, mail, file-storage, AI provider, model, threshold, input-limit, result-limit, and timeout settings are controlled through Laravel configuration and environment variables.

4.6 Requirements Prioritization

The functional requirements of the Graduation Project Management Platform were prioritized using the MoSCoW technique.

The prioritization represents functional importance rather than implementation status. All requirements listed in this section were implemented in the final evaluated platform. No documented functional requirement is classified as Won't Have because out-of-scope functions are not included in the authoritative FR1–FR56 requirement set.

Table 6 MoSCoW Prioritization of Functional Requirements

ID	Functional Requirement	Primary Actor	Priority
FR1	Display the public landing page	Guest	C
FR2	Log in using university number and password	Guest	M
FR3	Log out securely	Student, Supervisor, Administrator	M
FR4	Request password recovery using the university number	Guest	S
FR5	Reset the password using a valid token and university number	Guest	M
FR6	Complete a missing email address before accessing the protected portal	Student, Supervisor	M
FR7	Prevent inactive accounts from accessing protected areas	System	M
FR8	View and manage internal notifications	Student, Supervisor, Administrator	S
FR9	Display the Administrator dashboard and platform indicators	Administrator	S
FR10	List and inspect user accounts	Administrator	M
FR11	Create Student accounts	Administrator	M

ID	Functional Requirement	Primary Actor	Priority
FR12	Create Supervisor accounts and linked Supervisor profiles	Administrator	M
FR13	Activate or deactivate user accounts	Administrator	M
FR14	Reset another user's password	Administrator	S
FR15	View graduation projects in read-only mode	Administrator	S
FR16	View project join requests in read-only mode	Administrator	S
FR17	View submitted project ideas in read-only mode	Administrator	S
FR18	View milestone submissions in read-only mode	Administrator	S
FR19	View the activity audit log	Administrator	S
FR20	Display a Student dashboard adapted to the current enrollment state	Student	M
FR21	Browse available graduation projects and Supervisors	Student	M
FR22	Submit a project join request with proposed team information	Student	M
FR23	Submit a new graduation project idea	Student	M
FR24	Track the status of a project join request	Student	M

ID	Functional Requirement	Primary Actor	Priority
FR25	Track the status of a submitted project idea	Student	M
FR26	View the assigned graduation project details	Student	M
FR27	View the assigned project team members	Student	M
FR28	View the project timeline, milestones, and progress	Student	S
FR29	Upload an authorized milestone submission	Student	M
FR30	Download authorized submission files belonging to the assigned project	Student	M
FR31	Send messages to the responsible Supervisor	Student	S
FR32	View Supervisor replies and project announcements	Student	S
FR33	Change the Student account password	Student	S
FR34	Generate an advisory AI-assisted project-proposal suggestion	Student	S
FR35	Review, edit, and store an optional proposal description	Student	S
FR36	Perform an advisory pre-submission semantic-similarity analysis	Student	S
FR37	Display the Supervisor dashboard	Supervisor	M

ID	Functional Requirement	Primary Actor	Priority
FR38	Create new graduation projects and update owned projects	Supervisor	M
FR39	View owned projects and enrolled team members	Supervisor	M
FR40	Accept or reject project join requests associated with owned projects	Supervisor	M
FR41	Accept or reject project ideas assigned to the Supervisor	Supervisor	M
FR42	Reply to Student messages	Supervisor	S
FR43	Publish announcements for teams enrolled in owned projects	Supervisor	S
FR44	Review milestone submissions and provide feedback	Supervisor	M
FR45	Download submission files belonging to supervised projects	Supervisor	M
FR46	Change the Supervisor account password	Supervisor	S
FR47	View the complete submitted proposal description	Supervisor	S
FR48	View the stored advisory semantic-similarity snapshot	Supervisor	S
FR49	Enforce workflow-protection rules before critical operations	System	M
FR50	Prevent a Student from belonging to more than one active project	System	M

ID	Functional Requirement	Primary Actor	Priority
FR51	Prevent conflicting pending project requests and project ideas	System	M
FR52	Maintain consistency among project state, decisions, membership, and enrollment	System	M
FR53	Protect milestone-submission files from unauthorized access	System	M
FR54	Record selected administrative and workflow operations in the activity log	System	S
FR55	Generate internal notifications for supported workflow events	System	S
FR56	Generate and store a privacy-safe server-calculated similarity snapshot	System	S

4.7 UML Requirements Diagrams

The section includes a high-level use case diagram, detailed use case descriptions, activity diagrams for the most significant multi-step workflows, and sequence diagrams for each principal use case.

4.7.1 High-Level Use Case Diagram

The high-level use case diagram summarizes the principal services provided by the platform without displaying every individual form action, validation rule, or internal database operation.

The diagram contains the following seven principal use cases:

1. Access Platform;
2. Administer Users and Monitor Platform;
3. Manage Project Applications;
4. Manage Project Ideas;
5. Manage Graduation Projects;

6. Manage Submissions and Progress;
7. Manage Communication and Notifications.

Internal system requirements such as workflow validation, duplicate prevention, lifecycle consistency, file authorization, activity logging, notification generation, and similarity-snapshot persistence are implemented as supporting behavior within these principal use cases. They are not represented as an external System actor.

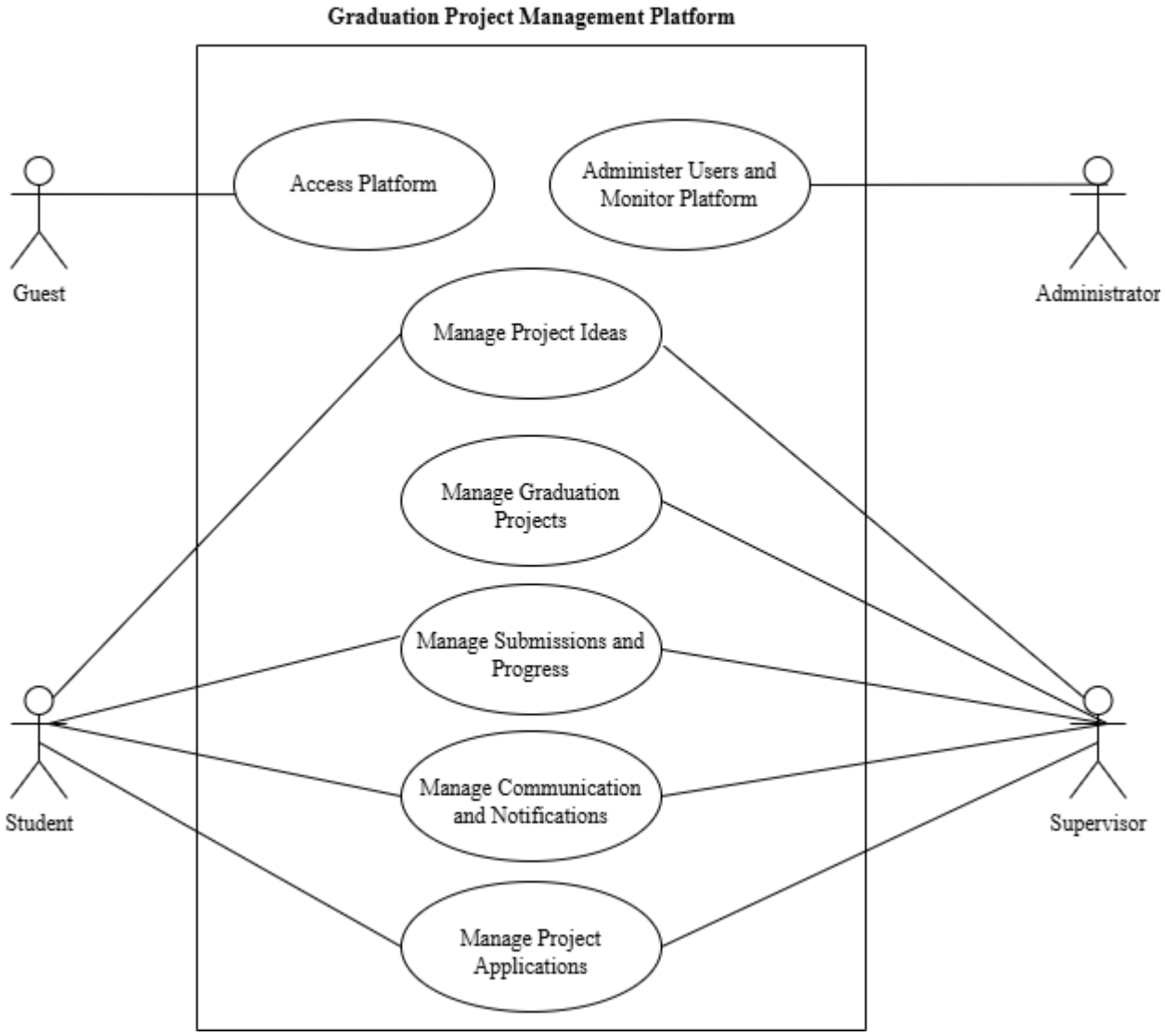


Figure 4 High-Level Use Case Diagram of the Graduation Project Management Platform

4.7.2 Use Case Descriptions

Table 7 Access Platform Use Case Description

Element	Description
Case Number	UC-01
Use Case Name	Access Platform
Primary Actors	Guest, Student, Supervisor, and Administrator
Secondary Actors	Mail Service
Description	Enables users to access the public platform, authenticate using a university number and password, complete missing email information, recover or reset a password, and terminate an authenticated session securely.
Triggers	The User opens the platform or selects the login or password-recovery option.
Preconditions	The platform is available. Login requires an existing user account. Access to an operational portal requires the account to be active, email-complete, and assigned to an authorized role.
Postconditions	Depending on the selected flow, an authenticated session is created or terminated, missing email information is completed, or a password-recovery or reset operation is processed.
Input	University number, password, email address when completion is required, password-reset token, and new-password information.
Output	Public page, authentication result, validation messages, role-specific dashboard, password-recovery status, email-completion result, or logout confirmation.
Main Flow	1. The User opens the platform. 2. The system displays the public landing page. 3. The User selects the login option. 4. The User enters the university number and password. 5. The system validates the credentials and account status. 6. The system checks whether the user's email address is complete.

Element	Description
	7. The system identifies the user's role. 8. The system redirects the user to the Student, Supervisor, or Administrator portal.
Alternative Flow	A. Invalid credentials: the system rejects the login attempt and displays an authentication error. B. Inactive account: the system denies access. C. Missing email address: the authenticated user is redirected to the email-completion page. D. Forgotten password: the user requests a reset link using the university number. E. Invalid or expired reset token: the password is not changed. F. Logout: the system invalidates the current session and redirects the user to the public platform. G. Expired session: the system redirects the user to the login page.
Frequency	High
Priority	High

Table 8 Administer Users and Monitor Platform Use Case Description

Element	Description
Case Number	UC-02
Use Case Name	Administer Users and Monitor Platform
Primary Actors	Administrator
Secondary Actors	None
Description	Enables the Administrator to create Student and Supervisor accounts, manage account activation, reset other users' passwords, inspect platform indicators, and monitor projects, requests, ideas, submissions, and activity records in read-only mode.
Triggers	The Administrator opens the administration dashboard or selects an account-management or monitoring function.

Element	Description
Preconditions	The Administrator is authenticated, the account is active, and the required email information is complete.
Postconditions	The requested account operation is stored, or the requested monitoring information is displayed.
Input	Student or Supervisor account type, name, university number, optional email address, initial password, account-status operation, password-reset information, and selected monitoring page.
Output	Created account and assigned role, linked Supervisor profile when applicable, updated activation state, password-reset confirmation, platform indicators, read-only monitoring records, and activity-log entries.
Main Flow	1. The Administrator opens the administration dashboard. 2. The system displays operational indicators. 3. The Administrator opens the user-management section. 4. The Administrator creates a Student or Supervisor account or selects an existing user. 5. The system validates the entered information. 6. The system stores the account and assigns the appropriate role. 7. For a Supervisor, the system creates and links a Supervisor profile. 8. The Administrator may activate, deactivate, or reset the password of an existing user. 9. The Administrator may view projects, requests, ideas, submissions, and activity records. 10. The system records supported administrative operations.
Alternative Flow	A. Duplicate university number or email: the system rejects the data. B. Missing or invalid information: validation errors are displayed. C. Unauthorized operation: access is denied. D. The selected record does not exist: the system displays an appropriate message. E. A technical failure occurs: no incomplete account data is stored.
Frequency	Medium
Priority	High

Table 9 Manage Project Applications Use Case Description

Element	Description
Case Number	UC-03
Use Case Name	Manage Project Applications
Primary Actors	Student
Secondary Actors	Supervisor
Description	Enables a Student to browse available projects, submit a team application, and receive the Supervisor's acceptance or rejection decision.
Triggers	The Student selects an available graduation project and chooses to submit a join request.
Preconditions	The Student is authenticated, is not enrolled in an active project, and has no conflicting pending application.
Postconditions	The request is stored as pending, rejected, or accepted. An accepted request creates project membership records.
Input	Selected project, proposed team-member university numbers, Supervisor decision, and rejection reason when applicable.
Output	Application record, request status, project membership records, updated project availability, and workflow notifications.
Main Flow	1. The Student opens the available-projects section. 2. The system displays available projects and their Supervisors. 3. The Student selects a project. 4. The Student enters the team members' university numbers. 5. The system validates project availability, team data, and workflow eligibility. 6. The Student submits the application.

Element	Description
	7. The system stores the request and notifies the Supervisor. 8. The Supervisor opens and reviews the request. 9. The Supervisor accepts or rejects the request. 10. If accepted , The system creates official project-membership records, marks the project as taken, updates the Student count, and stores the accepted request state. 11. The system notifies the Student team of the decision.
Alternative Flow	A. The project is no longer available: submission is blocked. B. A university number is invalid or repeated: the system requests corrected data. C. A team member already belongs to an active project: submission is blocked. D. A conflicting pending request or idea exists: submission is blocked. E. The Supervisor rejects the request: the rejection status and reason are stored.
Frequency	High
Priority	High

Table 10 Manage Project Ideas Use Case Description

Element	Description
Case Number	UC-04
Use Case Name	Manage Project Ideas
Primary Actors	Student
Secondary Actors	Supervisor
Description	Enables a Student to prepare and submit a project idea, optionally use AI-assisted proposal generation and semantic similarity analysis, and receive the Supervisor's decision.
Triggers	The Student opens the project-idea form and chooses to prepare a new graduation project proposal.

Element	Description
Preconditions	The Student is authenticated, is eligible to submit an idea, and has no conflicting enrollment or pending application.
Postconditions	The project idea is stored as pending, rejected, or accepted. The submitted idea retains the Student-approved proposal content and a server-generated similarity snapshot when available. An accepted idea creates a graduation project and official project-membership records.
Input	Initial idea text, project title, proposal description, selected Supervisor, team data, and rejection reason when applicable.
Output	AI-generated advisory suggestion, semantic similarity advisory result, stored idea, stored similarity snapshot, Supervisor decision, and workflow notifications.
Main Flow	1. The Student opens the project-idea form. 2. The Student enters an initial idea or requests an AI-generated proposal suggestion. 3. The system returns an advisory suggestion. 4. The Student reviews, applies, and edits the title and proposal description. 5. The Student may request a semantic similarity check. 6. The system displays the advisory similarity result. 7. The Student selects a Supervisor and enters the team information. 8. The system validates the proposal and workflow eligibility. 9. The Student submits the final idea. 10. The system stores the idea and team members. 11. The system generates and stores a server-side similarity snapshot. 12. The system notifies the Supervisor. 13. The Supervisor reviews the proposal and similarity information. 14. The Supervisor accepts or rejects the idea. 15. If accepted, the system creates the project and enrolls the team.
Alternative Flow	A. The AI service is unavailable: the system provides the configured fallback and permits manual editing. B. The similarity service is unavailable: the Student may continue without being blocked. C. A high similarity score is detected: the system displays an advisory warning only. D. Team data or workflow eligibility is invalid: submission is blocked. E. The Supervisor rejects the idea: the decision and rejection reason are stored. F. The Student modifies similarity values in the browser request: the supplied values are ignored, and the server generates the Supervisor-facing snapshot independently.
Frequency	Medium
Priority	High

Table 11 Manage Graduation Projects Use Case Description

Element	Description
Case Number	UC-05
Use Case Name	Manage Graduation Projects
Primary Actors	Supervisor
Secondary Actors	None
Description	Enables a Supervisor to create a graduation project, optionally assign valid Student members, and update projects owned by that Supervisor.
Triggers	The Supervisor opens the project-management section and chooses to create or update a project.
Preconditions	The Supervisor is authenticated and has a linked Supervisor profile.
Postconditions	A graduation project is created or updated and remains associated with the responsible Supervisor.
Input	Project name, description, department, team capacity, milestone dates, availability data, and optional team-member information.
Output	Created or updated project record, validation messages, project details, and team information.
Main Flow	1. The Supervisor opens the project-management section. 2. The system displays the Supervisor's projects. 3. The Supervisor selects the create or update option. 4. The Supervisor enters or modifies the project information. 5. The system validates the submitted data. 6. The system creates or updates the project.

Element	Description
	7. The system associates the project with the authenticated Supervisor. 8. The system displays the updated project information.
Alternative Flow	A. Required data is missing: the system displays validation errors. B. The project name already exists: the system rejects the operation. C. Milestone dates are invalid: the changes are not stored. D. The selected project does not belong to the authenticated Supervisor: the operation is denied. E. A technical failure occurs: the existing project data remains unchanged.
Frequency	Medium
Priority	High

Table 12 Manage Submissions and Progress Use Case Description

Element	Description
Case Number	UC-06
Use Case Name	Manage Submissions and Progress
Primary Actors	Student
Secondary Actors	Supervisor
Description	Enables an enrolled Student to upload milestone submissions and enables the responsible Supervisor to review them and update project progress through approval decisions.
Triggers	The Student selects a project milestone and chooses to upload a submission, or the Supervisor opens a pending submission for review.
Preconditions	The Student is enrolled in the project, the milestone exists, and the Supervisor is responsible for the project.

Element	Description
Postconditions	A submission is stored with the Submitted status, or an existing submission is marked as Approved or Needs Revision. Supervisor feedback and review information are stored, and approved milestones contribute to the displayed project progress.
Input	Milestone, submission file, title, notes, review decision, and Supervisor feedback.
Output	Protected submission file, submission status, review feedback, updated progress information, and workflow notifications.
Main Flow	1. The Student opens the project workspace. 2. The system displays the milestones and current progress. 3. The Student selects a milestone and uploads a file. 4. The system validates the file and the Student's project membership. 5. The system stores the file in protected storage. 6. The system records the submission and notifies the Supervisor. 7. The Supervisor opens the submission section. 8. The Supervisor downloads and reviews the authorized file. 9. The Supervisor marks the submission as approved or as requiring revision. 10. The Supervisor may enter review feedback. 11. The system stores the review result and notifies the Student. 12. If approved, the system reflects the milestone in the project progress.
Alternative Flow	A. The file type or size is invalid: the upload is rejected. B. The Student is not a member of the project: access is denied. C. A pending or approved submission already exists for the milestone: duplicate upload is blocked. D. Revision is required: progress is not advanced and resubmission becomes available. E. An unauthorized user requests the file: the download is denied. F. File storage fails: no completed submission is recorded.
Frequency	High
Priority	High

Table 13 Manage Communication and Notifications Use Case Description

Element	Description
Case Number	UC-07
Use Case Name	Manage Communication and Notifications
Primary Actors	Student, Supervisor, and Administrator
Secondary Actors	None
Description	Enables Students to send messages to selected Supervisors, enables Supervisors to reply and publish announcements for owned project teams, and enables authenticated users to view and manage their own workflow notifications.
Triggers	A Student sends a message, a Supervisor sends a reply or announcement, or a supported workflow event generates a notification.
Preconditions	The user is authenticated, active, email-complete, and assigned to the appropriate role. The selected Supervisor must exist. Project announcements require a project owned by the authenticated Supervisor.
Postconditions	The message, reply, or announcement is stored for the authorized participants. Supported workflow events generate database notifications for the intended recipients, and notification read status may be updated.
Input	Selected Supervisor, message subject, optional message content, reply content, owned project, announcement content, selected notification, and mark-as-read operation.
Output	Stored message, reply or announcement, database notification, unread count, and updated notification status.
Main Flow	1. The Student opens the communication section. 2. In the discovery or pending state, the Student selects a Supervisor; when enrolled, the responsible Supervisor is selected automatically.

Element	Description
	3. The Student enters a subject and optional message content. 4. The system validates and stores the message. 5. The system generates a notification for the selected Supervisor. 6. The Supervisor opens an assigned message and submits a reply. 7. The system stores the reply and notifies the Student. 8. The Supervisor may publish an announcement for an owned project. 9. Students enrolled in the corresponding project can view the announcement. 10. A supported workflow event may generate a database notification for a Student, Supervisor, or Administrator. 11. The authenticated recipient opens the notification page. 12. The recipient views a notification, marks it as read, or marks all owned notifications as read.
Alternative Flow	A. The selected Supervisor does not exist: the message is rejected. B. The subject is missing or the input exceeds its maximum length: validation errors are displayed. C. The Supervisor attempts to reply to another Supervisor's message: access is denied. D. The Supervisor does not own the selected project: the announcement is rejected. E. No messages, announcements, or notifications exist: the system displays an empty state. F. A user attempts to access or modify another user's notification: access is denied.
Frequency	High
Priority	Medium

4.7.3 Activity Diagrams

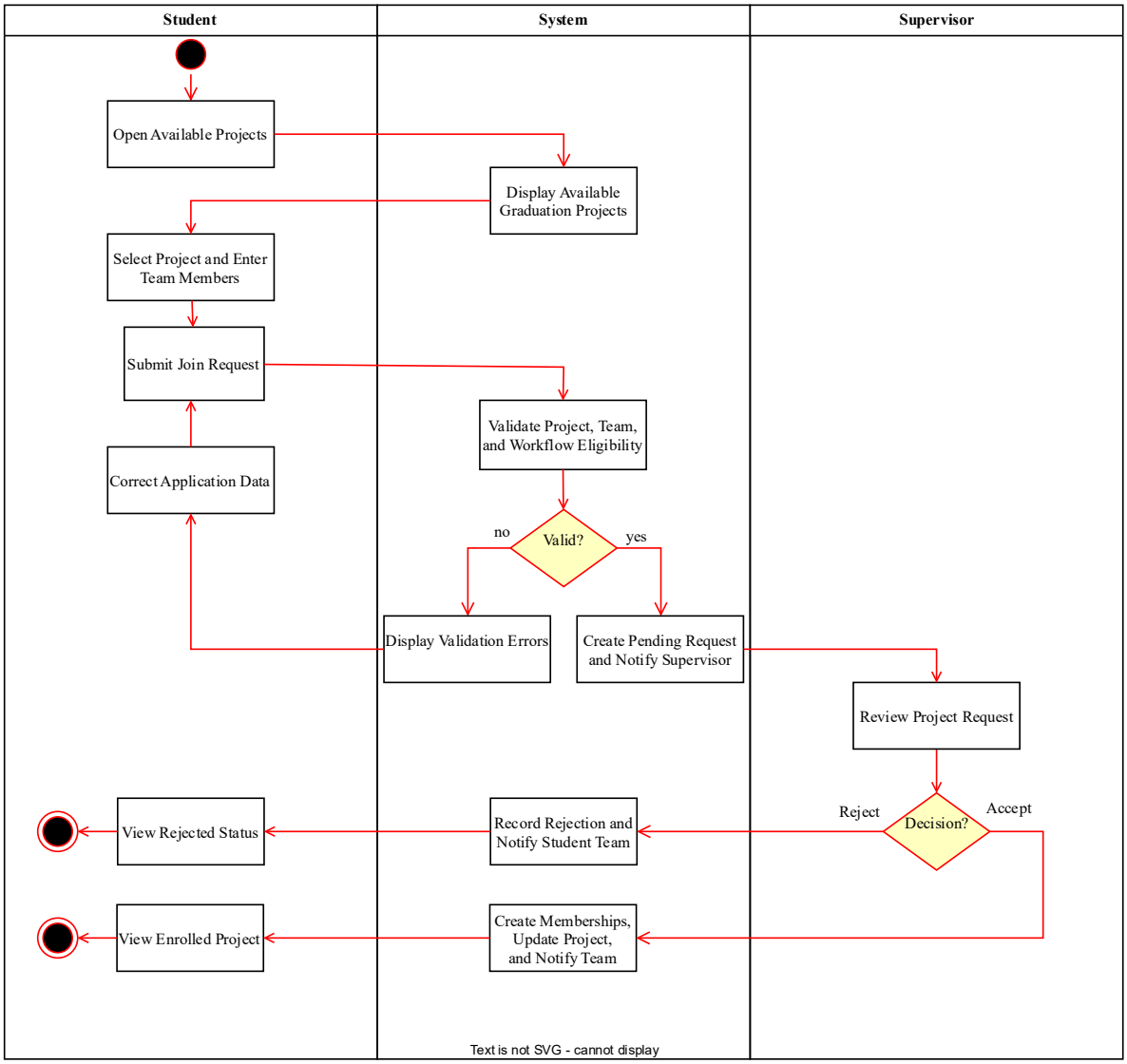


Figure 5 Activity Diagram for the Project Application Workflow

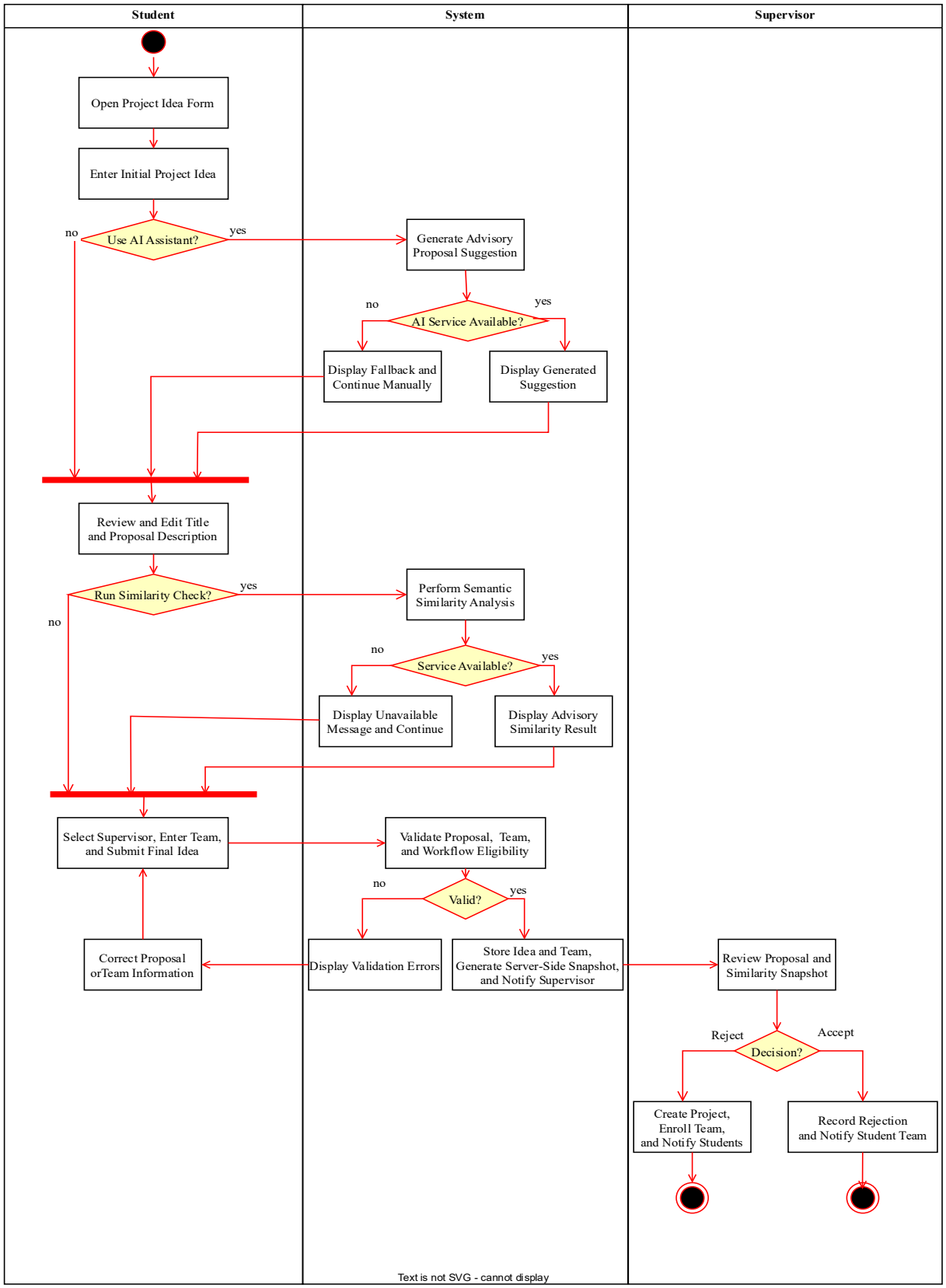


Figure 6 Activity Diagram for Project Idea Submission and Evaluation

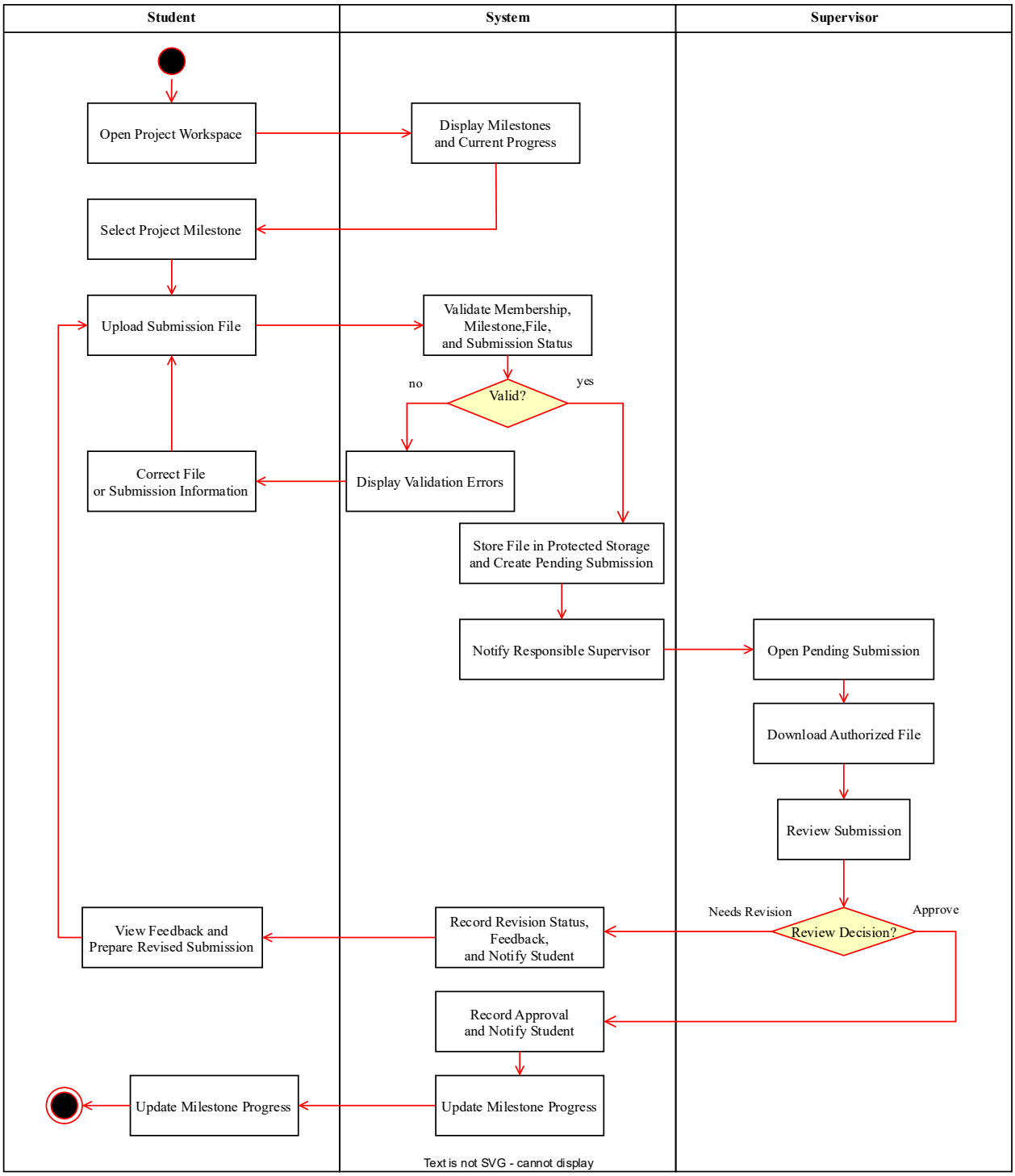


Figure 7 Activity Diagram for Milestone Submission and Review

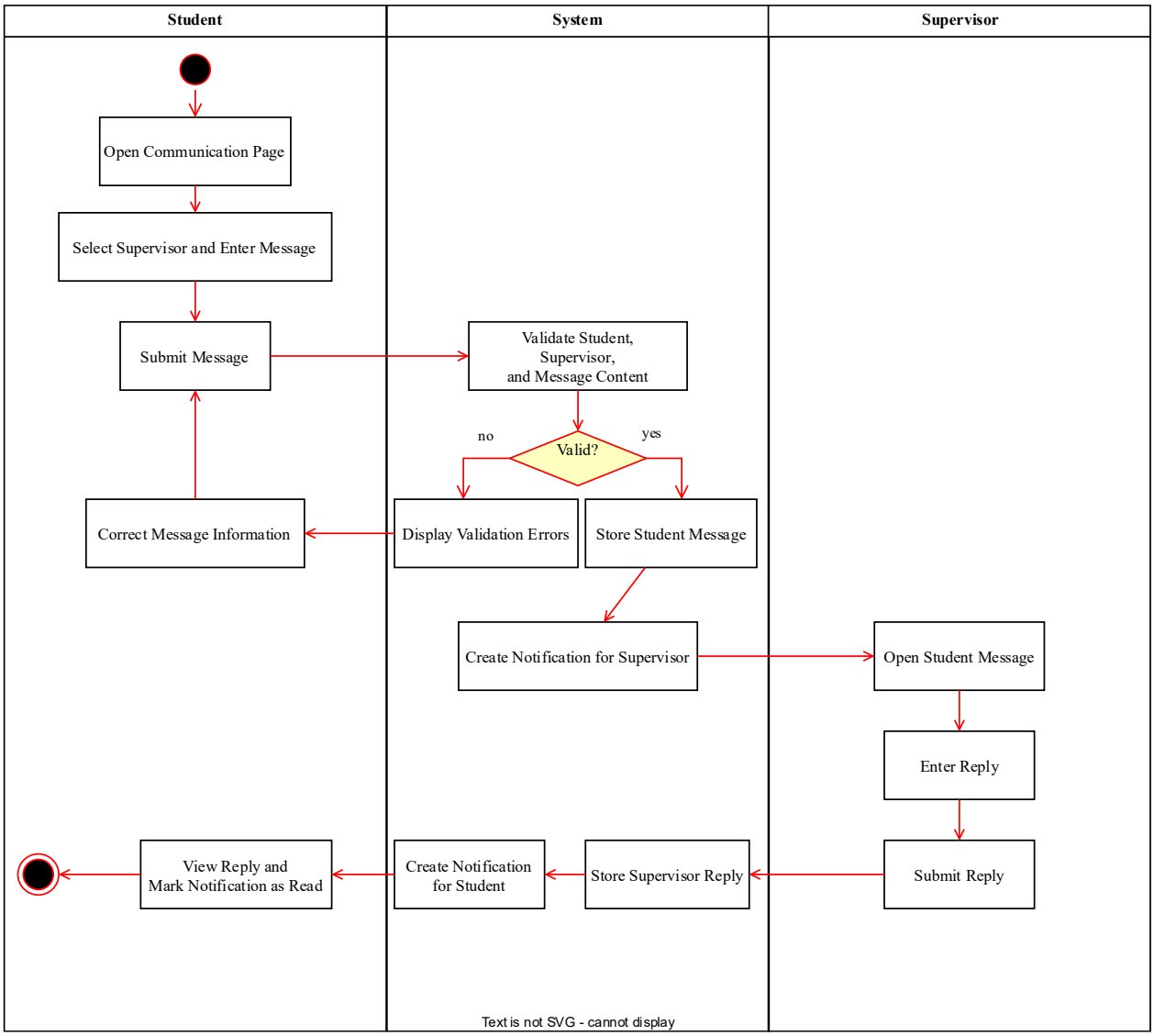


Figure 8 Activity Diagram for Student–Supervisor Communicat

4.7.4 Sequence Diagrams

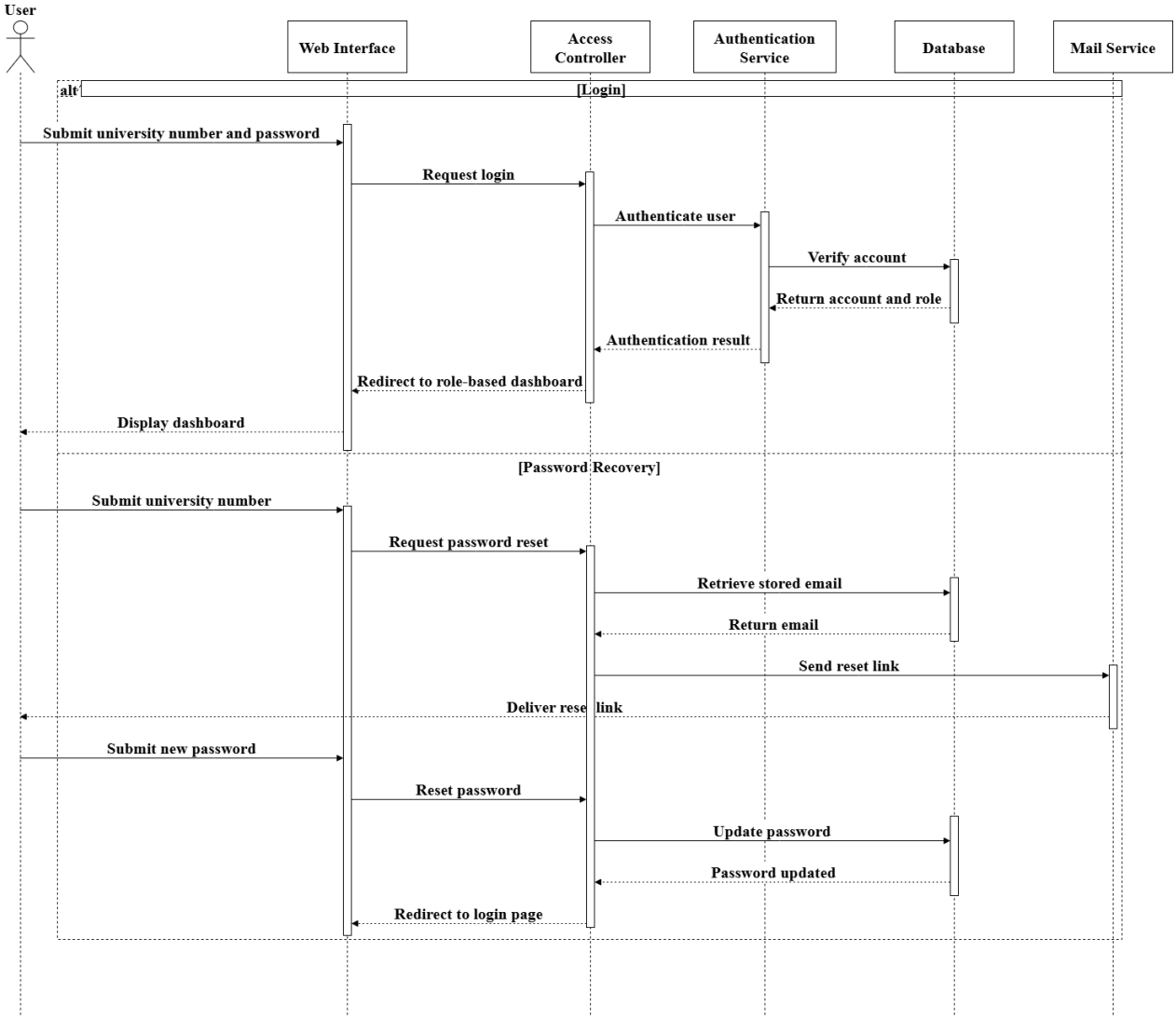


Figure 9 Sequence Diagram for Platform Access

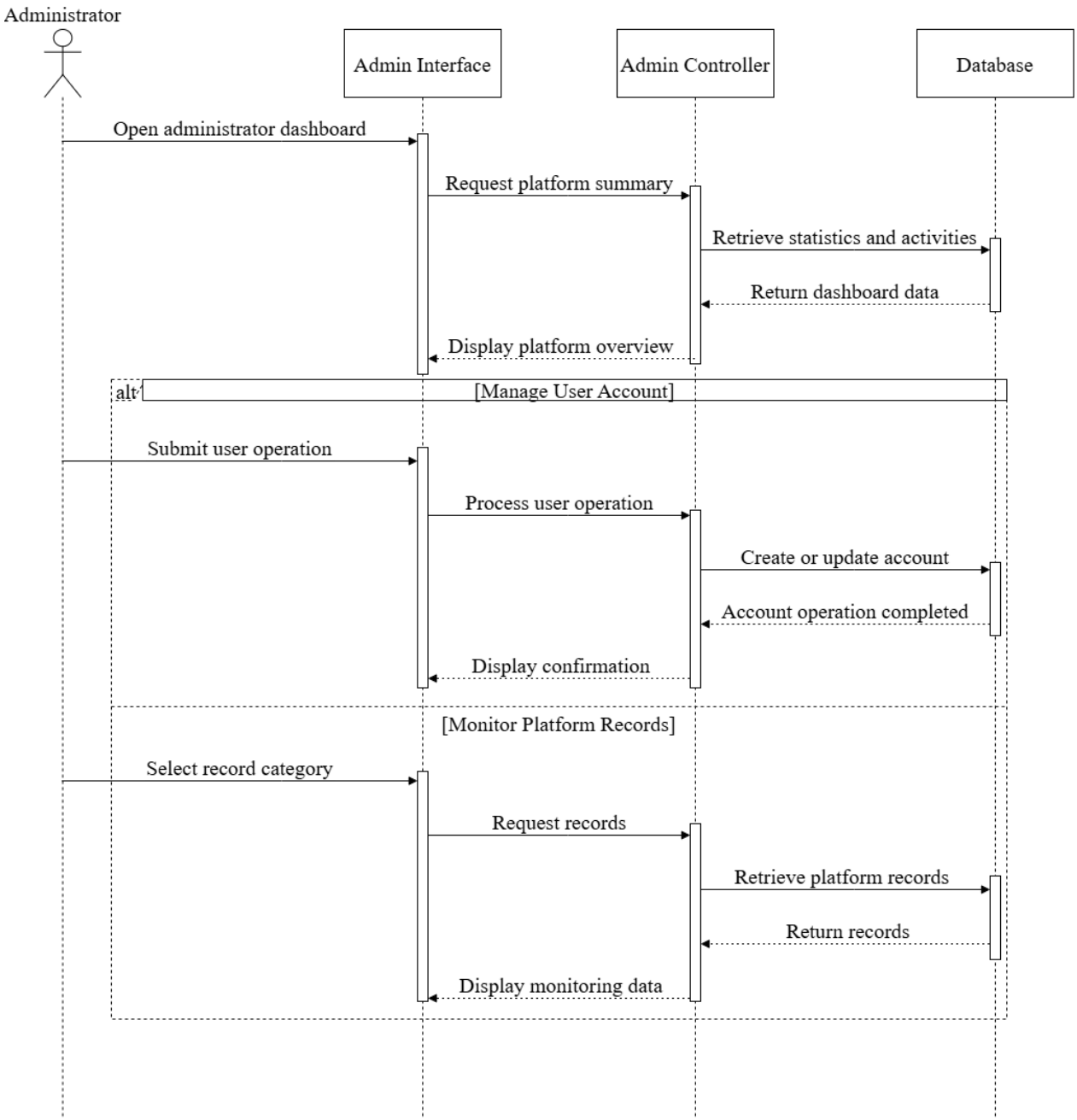


Figure 10 10 Sequence Diagram for User Administration and Platform Monitoring

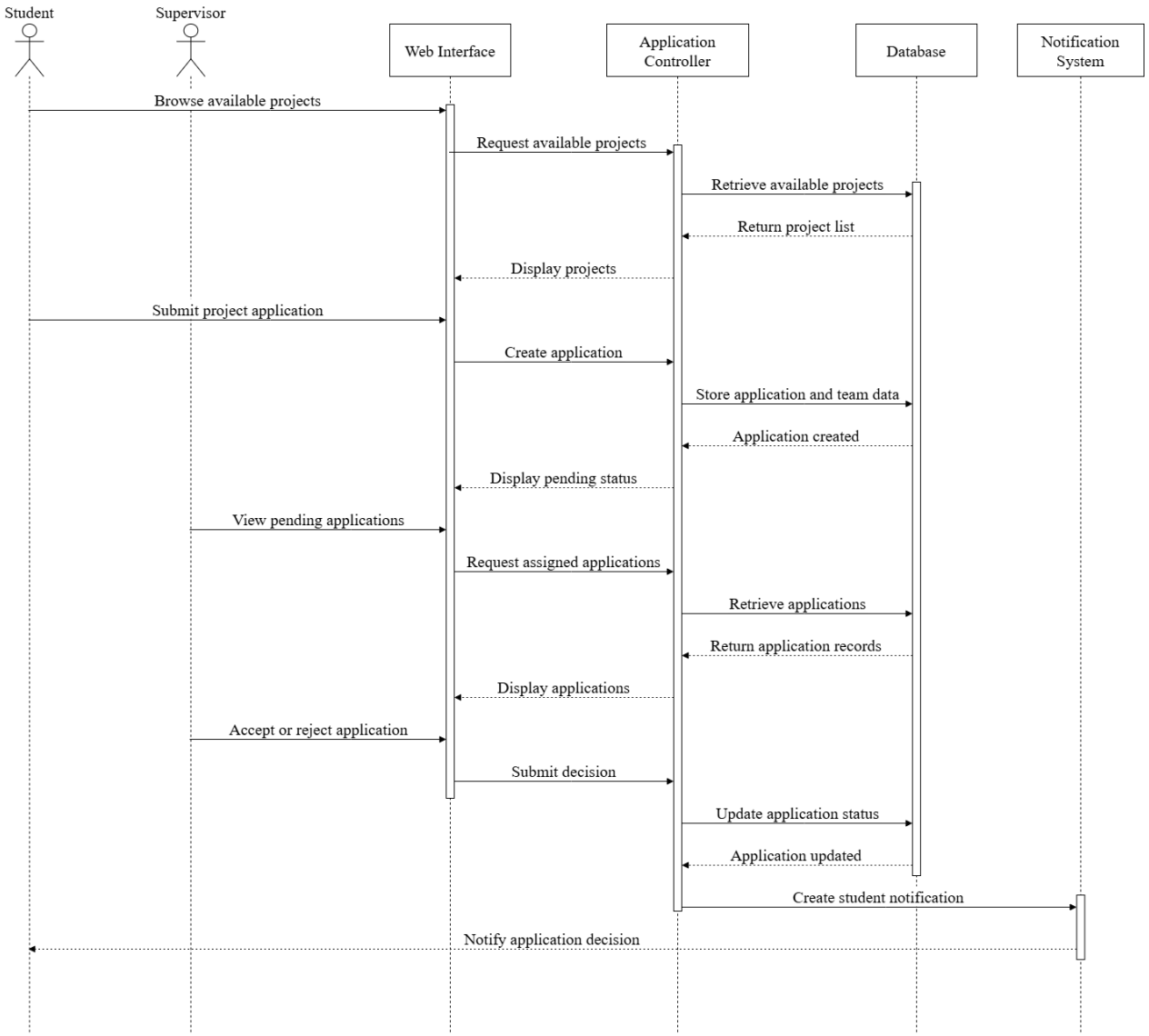


Figure 11 Sequence Diagram for Project Application Management

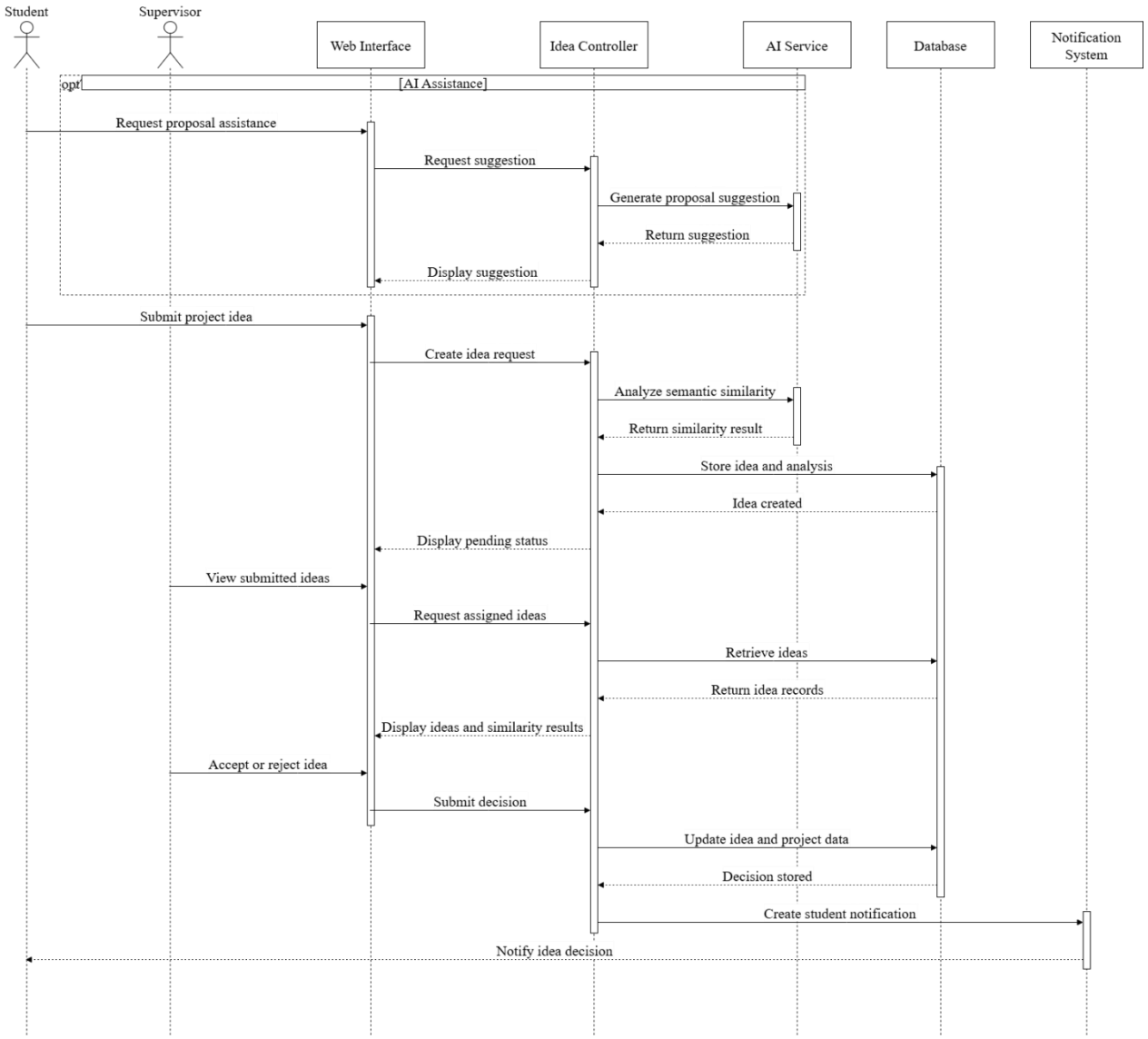


Figure 12 Sequence Diagram for Project Idea Management

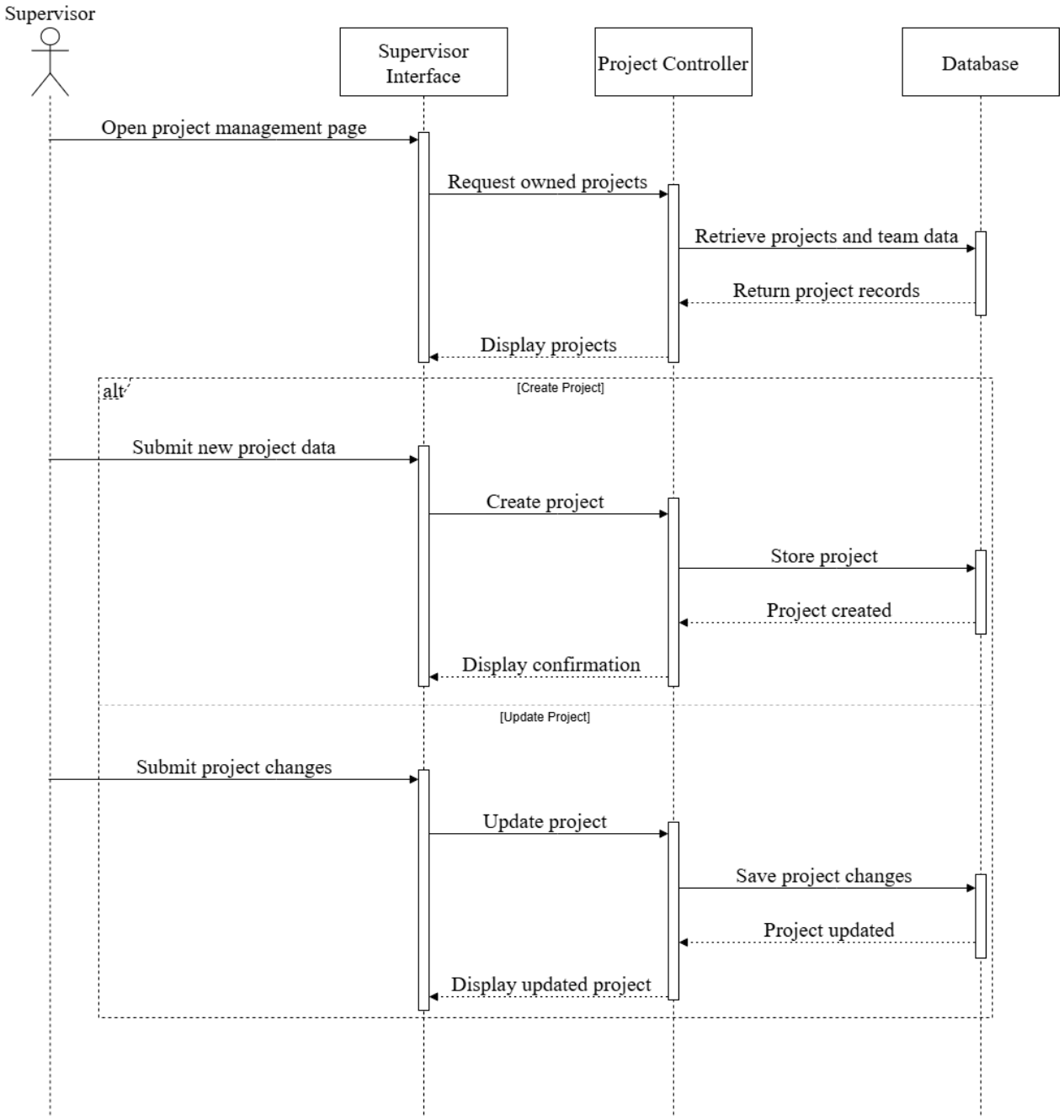


Figure 13 Diagram for Graduation Project Management

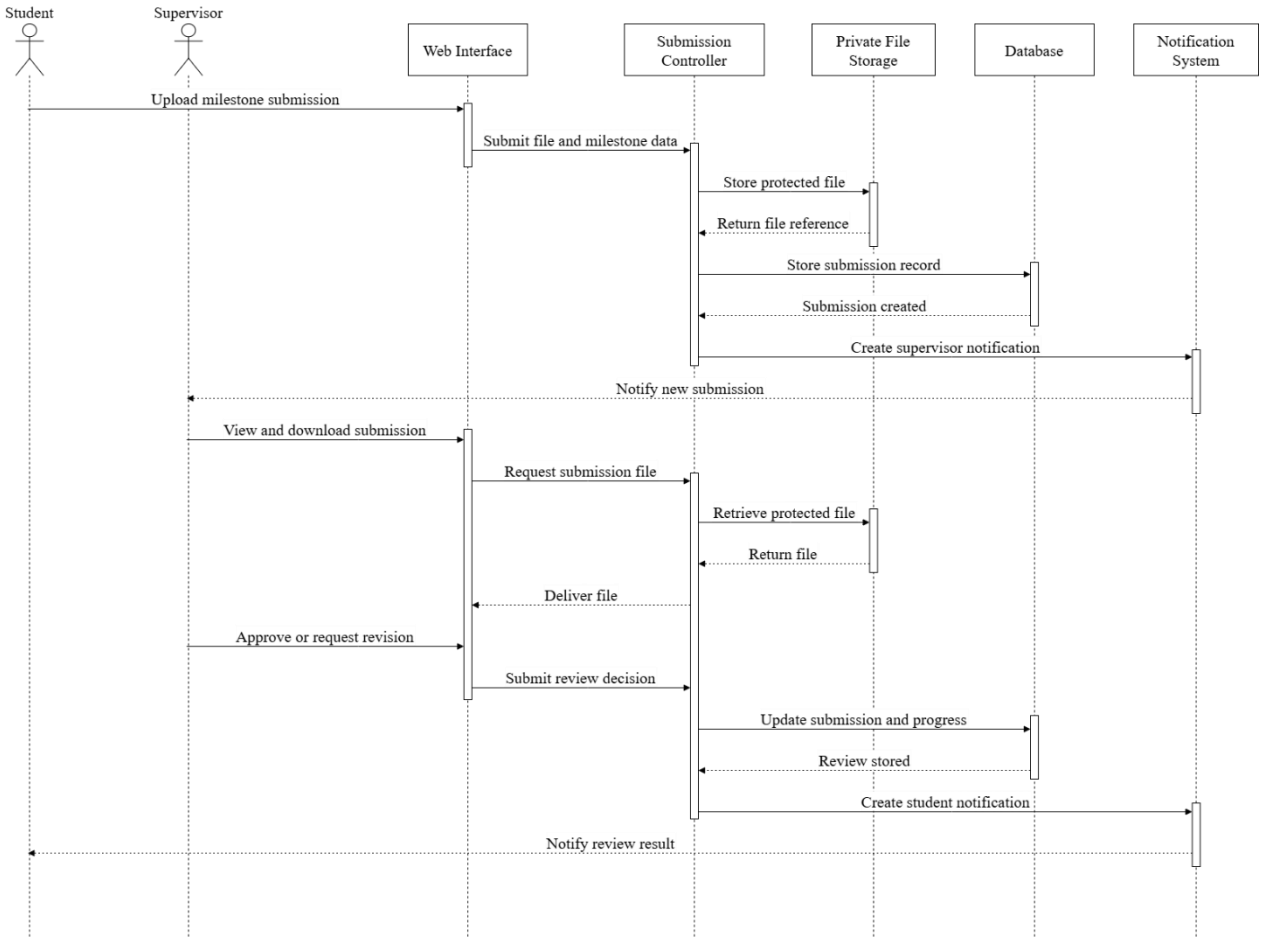


Figure 14 Sequence Diagram for Submission and Progress Management

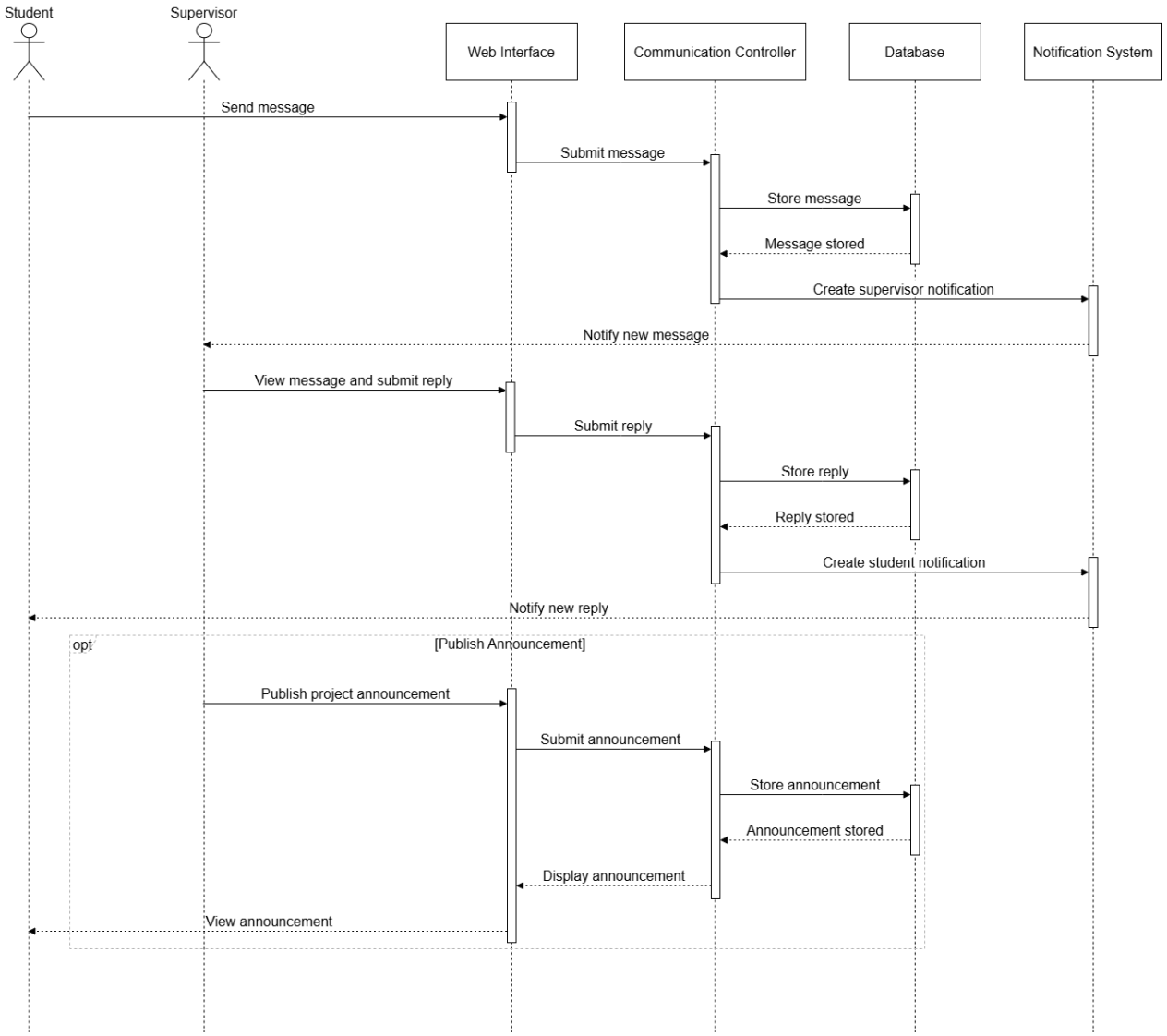


Figure 15 Sequence Diagram for Student–Supervisor Communication

4.8 Feature List

Table 14 Feature List of the Graduation Project Management Platform

Feature ID	Feature Group	Main Capabilities	Related Functional Requirements	Status
FL-01	Platform Access and Account Completion	Public landing page, university-number login, secure logout, password recovery, password reset, missing-email completion, inactive-account enforcement, and internal-notification access.	FR1–FR8	Implemented
FL-02	Administrator Account Management and Platform Monitoring	Administrator dashboard, user-account listing, Student account creation, Supervisor account creation with linked profiles, account activation and deactivation, password reset, read-only monitoring of projects, requests, ideas, and submissions, and activity-log inspection.	FR9–FR19	Implemented
FL-03	Student Project Exploration and Join Requests	State-aware Student dashboard, browsing available projects and Supervisors, submitting project join requests with proposed team information, and tracking request status.	FR20–FR22, FR24	Implemented
FL-04	Project Idea Submission and AI Assistance	Project-idea submission, proposal-description storage, idea-status tracking, AI-assisted proposal suggestions, pre-submission semantic-similarity analysis, and server-generated similarity-snapshot persistence.	FR23, FR25, FR34–FR36, FR56	Implemented
FL-05	Student Project Workspace	Viewing the assigned project, team members, milestone timeline, calculated progress, submission upload and authorized download, Supervisor communication, replies, announcements, and Student password change.	FR26–FR33	Implemented
FL-06	Supervisor Project and Application Management	Supervisor dashboard, project creation and update, viewing owned projects and enrolled teams, reviewing project join requests and submitted ideas, viewing complete proposal descriptions and stored similarity snapshots, and Supervisor password change.	FR37–FR41, FR46–FR48	Implemented
FL-07	Submission Review and Academic Communication	Replying to Student messages, publishing project announcements, reviewing milestone submissions, providing feedback, assigning Approved or Needs Revision status, and downloading authorized project-submission files.	FR42–FR45	Implemented
FL-08	Shared Workflow Protection and System Consistency	Workflow validation, prevention of multiple active enrollments, prevention of conflicting pending applications, project and membership consistency, private-file protection, activity logging, and workflow-notification generation.	FR49–FR55	Implemented

4.9 Requirements Analysis and Documentation

4.9.1 Requirements Traceability Matrix

Table 15 Requirements Traceability Matrix

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR1	Display a public landing page containing platform information and access to authentication functions.	Workflow Analysis	UC-01 — Access Platform	**TC-01**	**Implemented**
FR2	Allow registered users to log in using their university number and password.	Workflow Analysis	UC-01— Access Platform; Workflow and Authorization Design	TC-02	**Implemented**
FR3	Allow authenticated Students, Supervisors, and Administrators to log out securely.	Stakeholder Requirements	UC-01— Access Platform	TC-02	**Implemented**
FR4	Allow a user to request password recovery using the registered university number.	Stakeholder Requirements	UC-01— Access Platform	TC-03	**Implemented**
FR5	Allow a user to reset the account password using a valid reset token and university number.	Stakeholder Requirements	UC-01— Access Platform	TC-03	**Implemented**
FR6	Require an authenticated user whose account does not contain an email address to complete the missing email information before accessing any protected operational portal.	Stakeholder Requirements	UC-01— Access Platform; Workflow and Authorization Design	TC-04	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR7	Prevent inactive user accounts from logging in or continuing to access protected areas.	Stakeholder Requirements	UC-01— Access Platform; Workflow and Authorization Design	TC-02, TC-04	**Implemented**
FR8	Allow authenticated Students, Supervisors, and Administrators to view and manage their internal notifications.	Stakeholder Requirements	UC-07 — Communication and Notifications	TC-05	**Implemented**
FR9	Display an Administrator dashboard containing platform summary indicators.	Stakeholder Requirements	UC-02 — Administration and Monitoring	TC-06	**Implemented**
FR10	Allow the Administrator to list and inspect user accounts.	Stakeholder Requirements	UC-02— Administration and Monitoring	TC-06	**Implemented**
FR11	Allow the Administrator to create Student accounts.	Stakeholder Requirements	UC-02— Administration and Monitoring; Workflow and Authorization Design	TC-07	**Implemented**
FR12	Allow the Administrator to create Supervisor accounts together with their linked Supervisor profiles.	Stakeholder Requirements	UC-02— Administration and Monitoring; Workflow and Authorization Design	TC-07	**Implemented**
FR13	Allow the Administrator to activate or deactivate existing user accounts.	Stakeholder Requirements	UC-02— Administration and Monitoring; Workflow and Authorization Design	TC-07	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR14	Allow the Administrator to reset the password of another user account.	System Design Decisions	UC-02— Administration and Monitoring; UC-01 — Access Platform; Workflow and Authorization Design	TC-07	**Implemented**
FR15	Allow the Administrator to view all graduation projects in read-only mode.	System Design Decisions	UC-02 — Administration and Monitoring	TC-08	**Implemented**
FR16	Allow the Administrator to view all project join requests in read-only mode.	System Design Decisions	UC-02 — Administration and Monitoring	TC-08	**Implemented**
FR17	Allow the Administrator to view all submitted project ideas in read-only mode.	System Design Decisions	UC-02 — Administration and Monitoring	TC-08	**Implemented**
FR18	Allow the Administrator to view all milestone submissions in read-only mode.	System Design Decisions	UC-02— Administration and Monitoring	TC-08	**Implemented**
FR19	Allow the Administrator to view the recorded activity audit log.	System Design Decisions	UC-02— Administration and Monitoring; Workflow and Authorization Design	TC-08	**Implemented**
FR20	Display a Student dashboard adapted to the Student's discovery, pending-application, or enrolled-project state.	System Design Decisions	UC-03 — Project Applications; Student Project Workspace	TC-09, TC-12	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR21	Allow eligible Students to browse available graduation projects and related Supervisors.	Stakeholder Requirements	UC-03 — Project Applications	TC-09	**Implemented**
FR22	Allow an eligible Student to submit a request to join an existing graduation project together with proposed team-member information.	Workflow Analysis	UC-03 — Project Applications; Workflow and Authorization Design	TC-10	**Implemented**
FR23	Allow an eligible Student to submit a new graduation project idea containing a title, optional proposal description, selected Supervisor, and proposed team members.	Workflow Analysis	UC-04 — Project Ideas; Workflow and Authorization Design	TC-11	**Implemented**
FR24	Allow a Student to track the current status of a submitted project join request.	Workflow Analysis	UC-03 — Project Applications	TC-10	**Implemented**
FR25	Allow a Student to track the current status of a submitted project idea.	Workflow Analysis	UC-04 — Project Ideas	TC-11	**Implemented**
FR26	Allow an enrolled Student to view the assigned graduation project details.	Workflow Analysis	Student Project Workspace	TC-12	**Implemented**
FR27	Allow an enrolled Student to view the members of the assigned project team.	Workflow Analysis	Student Project Workspace	TC-12	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR28	Allow an enrolled Student to view the project timeline, milestone dates, and calculated project progress.	Workflow Analysis	Student Project Workspace; UC-06 — Submissions and Progress	TC-12	**Implemented**
FR29	Allow an enrolled Student to upload a file for an authorized project milestone.	Workflow Analysis	UC-06 — Submissions and Progress; Workflow and Authorization Design	TC-13	**Implemented**
FR30	Allow an enrolled Student to download authorized submission files belonging to the assigned project.	Workflow Analysis	UC-06 — Submissions and Progress; Workflow and Authorization Design	TC-13, TC-14	**Implemented**
FR31	Allow a Student to send a message to a selected Supervisor; when the Student is enrolled in a project, the responsible Supervisor is selected automatically.	Workflow Analysis	UC-07 — Communication and Notifications	TC-14	**Implemented**
FR32	Allow a Student to view Supervisor replies and announcements related to the assigned project.	Stakeholder Requirements	UC-07 — Communication and Notifications	TC-14	**Implemented**
FR33	Allow a Student to change the current account password.	Stakeholder Requirements	UC-01 — Access Platform	TC-14	**Implemented**
FR34	Allow a Student to generate an advisory AI-assisted project-proposal suggestion from an initial idea description.	Stakeholder Requirements	UC-04 — Project Ideas; MVC and Service-Layer Architecture; Artificial Intelligence Integration Design	TC-11	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR35	Allow a Student to review, edit, and store an optional proposal description with the submitted project idea.	Stakeholder Requirements	UC-04 — Project Ideas	TC-11	**Implemented**
FR36	Allow a Student to perform an advisory semantic-similarity analysis before submitting the final project idea.	Stakeholder Requirements	UC-04 — Project Ideas; MVC and Service-Layer Architecture; Artificial Intelligence Integration Design	TC-11	**Implemented**
FR37	Display a Supervisor dashboard containing the Supervisor's projects, applications, ideas, submissions, and communication records.	Stakeholder Requirements	UC-05 — Graduation Project Management	TC-15	**Implemented**
FR38	Allow a Supervisor to create new graduation projects and update projects owned by that Supervisor.	Stakeholder Requirements	UC-05 — Graduation Project Management; Workflow and Authorization Design	TC-15	**Implemented**
FR39	Allow a Supervisor to view owned graduation projects and their enrolled team members.	Stakeholder Requirements	UC-05 — Graduation Project Management	TC-15	**Implemented**
FR40	Allow the responsible Supervisor to accept or reject pending project join requests associated with owned projects.	Stakeholder Requirements	UC-03 — Project Applications; UC-05 — Graduation Project Management; Workflow and Authorization Design	TC-16	**Implemented**
FR41	Allow the responsible Supervisor to accept or reject pending project ideas assigned to that Supervisor.	Stakeholder Requirements	UC-04 — Project Ideas; UC-05 — Graduation Project Management; Workflow and Authorization Design	TC-17	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR42	Allow a Supervisor to reply to Student messages assigned to that Supervisor.	Stakeholder Requirements	UC-07 — Communication and Notifications	TC-18	**Implemented**
FR43	Allow a Supervisor to publish announcements for teams enrolled in owned projects.	Stakeholder Requirements	UC-07 — Communication and Notifications	TC-18	**Implemented**
FR44	Allow the responsible Supervisor to review milestone submissions, provide feedback, and mark them as Approved or Needs Revision.	Stakeholder Requirements	UC-06 — Submissions and Progress; Workflow and Authorization Design	TC-19	**Implemented**
FR45	Allow a Supervisor to download submission files belonging to supervised projects.	Stakeholder Requirements	UC-06 — Submissions and Progress; Workflow and Authorization Design	TC-19, TC-24	**Implemented**
FR46	Allow a Supervisor to change the current account password.	Stakeholder Requirements	UC-01 — Access Platform	TC-18	**Implemented**
FR47	Allow the responsible Supervisor to view the complete proposal description submitted with a project idea.	Stakeholder Requirements	UC-04 — Project Ideas; UC-05 — Graduation Project Management	TC-17	**Implemented**
FR48	Allow the responsible Supervisor to view the stored advisory semantic-similarity snapshot associated with a submitted project idea.	Stakeholder Requirements	UC-04 — Project Ideas; UC-05 — Graduation Project Management; Artificial Intelligence Integration Design	TC-17	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR49	Enforce workflow-protection rules before executing critical project, application, enrollment, and submission operations.	Stakeholder Requirements	Workflow and Authorization Design	TC-20	**Implemented**
FR50	Prevent a Student from belonging to more than one active graduation project.	Stakeholder Requirements	Workflow and Authorization Design	TC-20	**Implemented**
FR51	Prevent Students and proposed team members from maintaining conflicting pending project requests or project ideas.	Stakeholder Requirements	Workflow and Authorization Design	TC-20	**Implemented**
FR52	Maintain consistency among project lifecycle state, application decisions, official team membership, and enrollment status.	Stakeholder Requirements	UC-05 — Graduation Project Management; Workflow and Authorization Design; Relational Database Design	TC-20	**Implemented**
FR53	Protect milestone-submission files from unauthorized upload, viewing, or download operations.	Stakeholder Requirements	UC-06 — Submissions and Progress; Workflow and Authorization Design	TC-19	**Implemented**
FR54	Record selected administrative and workflow operations in the activity audit log.	Stakeholder Requirements	UC-02 — Administration and Monitoring; Workflow and Authorization Design	TC-08, TC-20	**Implemented**
FR55	Generate internal notifications for supported project, application, submission, review, and communication events.	Stakeholder Requirements	UC-07 — Communication and Notifications; Workflow and Authorization Design	TC-05, TC-20	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
FR56	Generate and store a privacy-safe, server-calculated semantic-similarity snapshot when the final project idea is submitted.	Stakeholder Requirements	UC-04 — Project Ideas; Workflow and Authorization Design; MVC and Service-Layer Architecture; Artificial Intelligence Integration Design	TC-11, TC-20	**Implemented**
NFR-PERF-01	The Administrator dashboard shall retrieve its principal indicators using aggregated database queries rather than loading complete record collections into application memory.	System Design Decisions	UC-02 — Administration and Monitoring; MVC and Service-Layer Architecture	TC-21	**Implemented**
NFR-PERF-02	The administrative activity log shall use pagination to restrict the number of records displayed in one request.	System Design Decisions	UC-02 — Administration and Monitoring; Workflow and Authorization Design	TC-21	**Implemented**
NFR-PERF-03	The AI Proposal Assistant and Semantic Similarity Assistant shall apply request-rate limits to reduce excessive repeated model execution.	System Design Decisions	UC-04 — Project Ideas; Workflow and Authorization Design; MVC and Service-Layer Architecture; Artificial Intelligence Integration Design	TC-21	**Implemented**
NFR-PERF-04	Artificial intelligence operations shall use bounded inputs, configurable timeouts, controlled embedding batches, and a limited result set.	System Design Decisions	UC-04 — Project Ideas; MVC and Service-Layer Architecture; Artificial Intelligence Integration Design	TC-21	**Implemented**
NFR-SEC-01	User passwords shall be stored using Laravel's configured one-way password-hashing mechanism, and authentication shall use the university number and password.	System Design Decisions	UC-01 — Access Platform; Workflow and Authorization Design	TC-02, TC-22	**Implemented**
NFR-SEC-02	Protected portals shall require an authenticated, active, email-complete user assigned to the correct operational role.	System Design Decisions	UC-01 — Access Platform; Workflow and Authorization Design	TC-02, TC-04, TC-22, TC-23	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
NFR-SEC-03	State-changing web forms shall use Cross-Site Request Forgery protection, and logout shall invalidate the authenticated session.	System Design Decisions	UC-01 — Access Platform; Workflow and Authorization Design	TC-02, TC-22	**Implemented**
NFR-SEC-04	Milestone-submission files shall be privately stored, validated by supported type and size, and downloaded only after resource-level authorization.	System Design Decisions	UC-06 — Submissions and Progress; Workflow and Authorization Design; Relational Database Design	TC-19, TC-24	**Implemented**
NFR-USE-01	Students, Supervisors, and Administrators shall access the platform through one authentication entry point and be redirected to the portal corresponding to their role.	System Design Decisions	UC-01 — Access Platform; Responsive Interface Design	TC-02, TC-25	**Implemented**
NFR-USE-02	The Student dashboard shall adapt to discovery, pending-application, and enrolled-project states and shall display understandable workflow-status information.	System Design Decisions	UC-03 — Project Applications; Student Project Workspace; Responsive Interface Design	TC-09, TC-12, TC-25	**Implemented**
NFR-USE-03	Forms and workflow pages shall provide visible labels, validation feedback, status indicators, empty states, and rejection or review reasons where applicable.	System Design Decisions	UC-03 — Project Applications; UC-04 — Project Ideas; UC-06 — Submissions and Progress; UC-07 — Communication and Notifications; Responsive Interface Design	TC-10, TC-11, TC-14, TC-19, TC-25	**Implemented**
NFR-USE-04	Artificial intelligence output shall be clearly identified as advisory and shall require explicit Student action before being applied or submitted.	System Design Decisions	UC-04 — Project Ideas; Responsive Interface Design; Artificial Intelligence Integration Design	TC-11, TC-25	**Implemented**
NFR-REL-01	Shared workflow guards shall prevent duplicate pending applications and multiple active project enrollments.	System Design Decisions	Workflow and Authorization Design	TC-20	**Implemented**

Requirement ID	Requirement Description	Source	Design Reference	Test Case	Status
NFR-REL-02	Critical application, enrollment, decision, and submission operations shall preserve database and lifecycle consistency.	System Design Decisions	UC-05 — Graduation Project Management; UC-06 — Submissions and Progress; Workflow and Authorization Design; Relational Database Design	TC-16, TC-17, TC-19, TC-20	**Implemented**
NFR-REL-03	Unavailability, timeout, or invalid output from the local artificial intelligence service shall not prevent the primary project-idea submission workflow.	System Design Decisions	UC-04 — Project Ideas; Workflow and Authorization Design; MVC and Service-Layer Architecture; Artificial Intelligence Integration Design	TC-11, TC-24	**Implemented**
NFR-REL-04	The complete automated regression suite shall pass before the final software baseline is accepted.	System Design Decisions	Workflow and Authorization Design; MVC and Service-Layer Architecture	TC-24	**Implemented**
NFR-MAIN-01	The application shall follow Laravel's Model–View–Controller separation among request handling, persistent models, and interface rendering.	System Design Decisions	MVC and Service-Layer Architecture	TC-25	**Implemented**
NFR-MAIN-02	Shared workflow, enrollment-state, activity-logging, proposal-assistance, and similarity-processing logic shall be organized into reusable service classes.	System Design Decisions	Workflow and Authorization Design; MVC and Service-Layer Architecture; Artificial Intelligence Integration Design	TC-25	**Implemented**
NFR-MAIN-03	Database evolution shall be managed through version-controlled migrations, Eloquent relationships, factories, and seeders.	System Design Decisions	Relational Database Design; MVC and Service-Layer Architecture	TC-25	**Implemented**
NFR-MAIN-04	Environment-dependent settings shall be configurable without modifying the principal application source code.	System Design Decisions	UC-04 — Project Ideas; MVC and Service-Layer Architecture; Artificial Intelligence Integration Design	TC-25	**Implemented**

4.9.2 Preliminary Test Cases

Preliminary test cases were defined during requirements analysis to ensure that the principal system workflows and quality requirements are verifiable. These cases specify the related requirements, test scenario, principal test procedure, and expected result. Detailed execution evidence, actual results, and test metrics are presented later in Chapter 7.

Table 16 Preliminary Functional Test Cases

Test Case ID	Related Requirements	Test Scenario	Preliminary Test Procedure	Expected Result
TC-01	FR1	Public platform access	Open the public landing page without authentication and inspect the available access options.	The landing page is displayed successfully, while protected portals remain inaccessible to unauthenticated users.
TC-02	FR2, FR3, FR7	Login, role routing, and logout	Submit valid and invalid university-number credentials, test an inactive account, then log out after successful authentication.	Invalid credentials are rejected, inactive accounts are blocked, valid users are redirected according to role, and logout terminates the session.
TC-03	FR4, FR5	Password recovery and reset	Submit a registered university number, use the delivered reset link, then enter the university number and a new password.	A generic recovery response is displayed, the reset link is delivered to the stored email when available, and a valid token allows the password to be updated.
TC-04	FR6, FR7	Missing-email completion gate	Log in with an active account that has no email address, submit a valid email, and attempt to enter the portal.	Portal access is blocked until the email is completed, after which the user is redirected to the authorized dashboard.
TC-05	FR8, FR55	Internal notification management	Generate a supported workflow event, open the notification list, mark one notification as read, and mark all notifications as read.	The correct recipient receives the notification, and its read status is updated without affecting other users' notifications.
TC-06	FR9, FR10	Administrator dashboard and user listing	Log in as Administrator, open the dashboard, and access the user-management page.	Platform indicators and user-account records are displayed to the Administrator.
TC-07	FR11–FR14	Administrator account management	Create Student and Supervisor accounts, attempt duplicate data, activate or deactivate an account, and reset another user's password.	Valid accounts are created with the correct roles and profiles, duplicate data is rejected, and account lifecycle operations are applied correctly.
TC-08	FR15–FR19, FR54	Administrator monitoring and audit log	Open the projects, join requests, ideas, submissions, and activity-log pages as Administrator.	The Administrator can inspect the authorized records in read-only mode and view recorded administrative and workflow activities.
TC-09	FR20, FR21	Student dashboard and project browsing	Log in as a Student in discovery state, inspect the dashboard, and browse projects and supervisors.	The dashboard reflects the Student's current state and displays the available project and supervisor information.
TC-10	FR22, FR24, FR49–FR51	Project join-request workflow	Submit a valid join request with team data, attempt a duplicate or invalid request, and inspect the request status.	A valid request is stored as pending, invalid or duplicate submissions are rejected, and the Student can track the current status.

Test Case ID	Related Requirements	Test Scenario	Preliminary Test Procedure	Expected Result
TC-11	FR23, FR25, FR34–FR36, FR56	Project-idea submission and AI assistance	Enter an idea, request an AI proposal suggestion, perform a similarity check, submit the idea, and inspect its stored data.	Advisory AI output is displayed when available, the idea remains submittable if AI is unavailable, and the pending idea stores its proposal and server-calculated similarity snapshot when available.
TC-12	FR26–FR28	Enrolled Student workspace	Log in as a Student enrolled in a project and open the project, team, timeline, and progress sections.	The Student can view only the assigned project, its team members, timeline, milestones, and calculated progress.
TC-13	FR29, FR30, FR53	Student submission upload and download	Upload a valid milestone file, attempt an invalid upload, then download an authorized and an unauthorized submission.	Valid files are stored with a submission record, invalid files are rejected, and downloads are permitted only to authorized project members.
TC-14	FR31–FR33	Student communication and password change	Send a message to the responsible Supervisor, view replies and announcements, and change the current password.	The message is stored for the correct Supervisor, authorized replies and announcements are displayed, and the password is changed after valid confirmation.
TC-15	FR37–FR39	Supervisor project management	Log in as Supervisor, view the dashboard and teams, create a project, update an owned project, and attempt to update another Supervisor's project.	The Supervisor can manage only owned projects and can view the associated team information.
TC-16	FR40, FR49–FR52	Join-request review	Open a pending request for an owned project, accept or reject it, then attempt to review it again.	Only the responsible Supervisor can review the pending request; acceptance creates the correct enrollment, rejection stores the decision, and repeated processing is prevented.
TC-17	FR41, FR47, FR48, FR56	Project-idea review	Open an assigned pending idea, inspect the proposal and stored similarity result, then accept or reject the idea.	The Supervisor can view the complete proposal and advisory analysis, and the selected decision updates the idea and related workflow consistently.
TC-18	FR42, FR43, FR46	Supervisor communication and password change	Reply to an authorized Student message, publish an announcement to an owned project, and change the Supervisor password.	The reply is associated with the correct message, the announcement is visible to the related project team, and the password is updated successfully.
TC-19	FR44, FR45, FR53	Submission review and protected download	Open a submission for an owned project, download the file, provide feedback, and select Approved or Needs Revision.	Only the responsible Supervisor can access and review the submission, the decision and feedback are stored, and the Student sees the updated status.
TC-20	FR49–FR55	Shared workflow protection	Attempt unauthorized access, duplicate pending applications, multiple active enrollments, invalid lifecycle transitions, and cross-project file access.	The system rejects unauthorized or inconsistent operations, preserves lifecycle integrity, records supported important actions, and generates the applicable workflow notifications.

Table 17 Preliminary Non-Functional Verification Cases

Test Case ID	NFR Category	Related Requirements	Preliminary Verification Procedure	Expected Result
TC-21	Performance	Performance NFRs	Inspect dashboard aggregation and pagination behavior, exercise AI request controls, and verify configured input and timeout boundaries.	Lists and dashboards use bounded retrieval mechanisms, while AI operations apply the configured request, input, and timeout controls.
TC-22	Security	Security NFRs	Inspect authentication middleware, role restrictions, password hashing, CSRF protection, session invalidation, file validation, and download authorization.	Protected operations require the correct authenticated role, passwords are not stored in plain text, state-changing requests are protected, and private files cannot be accessed without authorization.
TC-23	Usability	Usability NFRs	Manually perform the principal Student, Supervisor, and Administrator workflows on desktop and narrow-screen layouts.	Navigation, labels, validation feedback, status indicators, and workflow actions remain understandable and usable across the reviewed layouts.
TC-24	Reliability	Reliability NFRs	Simulate invalid workflow states, repeated decisions, duplicate applications, unauthorized operations, and unavailable AI services.	The system preserves consistent records, prevents repeated or conflicting operations, and primary workflows continue when advisory AI services are unavailable.
TC-25	Maintainability	Maintainability NFRs	Review the Laravel project structure, service classes, migrations, relationships, shared components, environment configuration, and regression-test organization.	Responsibilities are separated through MVC and reusable services, schema changes are versioned, environment-dependent settings are configurable, and regression tests remain executable.

Chapter 5: Architectural and Detailed Design

This chapter presents the architectural and detailed design of the Graduation Project Management Platform. It explains the system's high-level structure, architectural pattern, principal components, database design, external interfaces, and security mechanisms. The design translates the requirements defined in Chapter 4 into a structured technical solution that supports authentication, role-based access, project workflows, milestone submissions, communication, notifications, audit logging, and local artificial intelligence services.

The chapter focuses on the logical organization of the platform and the interaction between its components. Implementation-specific details, development tools, source-code structure, and representative code excerpts are presented separately in Chapter 6.

5.1 High-Level Architectural Design

The Graduation Project Management Platform is implemented as a web-based client–server system. Users access the platform through a web browser, while the Laravel application processes requests, enforces workflow and authorization rules, communicates with the database and supporting services, and returns dynamically generated web interfaces.

At the application level, the platform follows a layered monolithic architecture based on the Model–View–Controller architectural pattern. All functional modules are deployed as one integrated Laravel application, while responsibilities are logically separated into presentation, request handling, business logic, data-access, storage, and supporting-service layers.

5.1.1 Architectural Pattern

The architectural design combines three complementary concepts:

Client–Server Architecture: The user's web browser acts as the client, while the Laravel application acts as the server responsible for authentication, request processing, workflow execution, data access, and response generation.

Model–View–Controller Pattern: Laravel separates the user interface into Blade views, request handling into controllers, and persistent domain data into Eloquent models. This separation reduces direct dependencies between the interface, application logic, and database structure.

Layered Architecture: Business rules and shared workflow operations are organized into middleware and reusable service classes rather than being implemented entirely inside controllers. This provides clearer separation between presentation, application control, business logic, and data access.

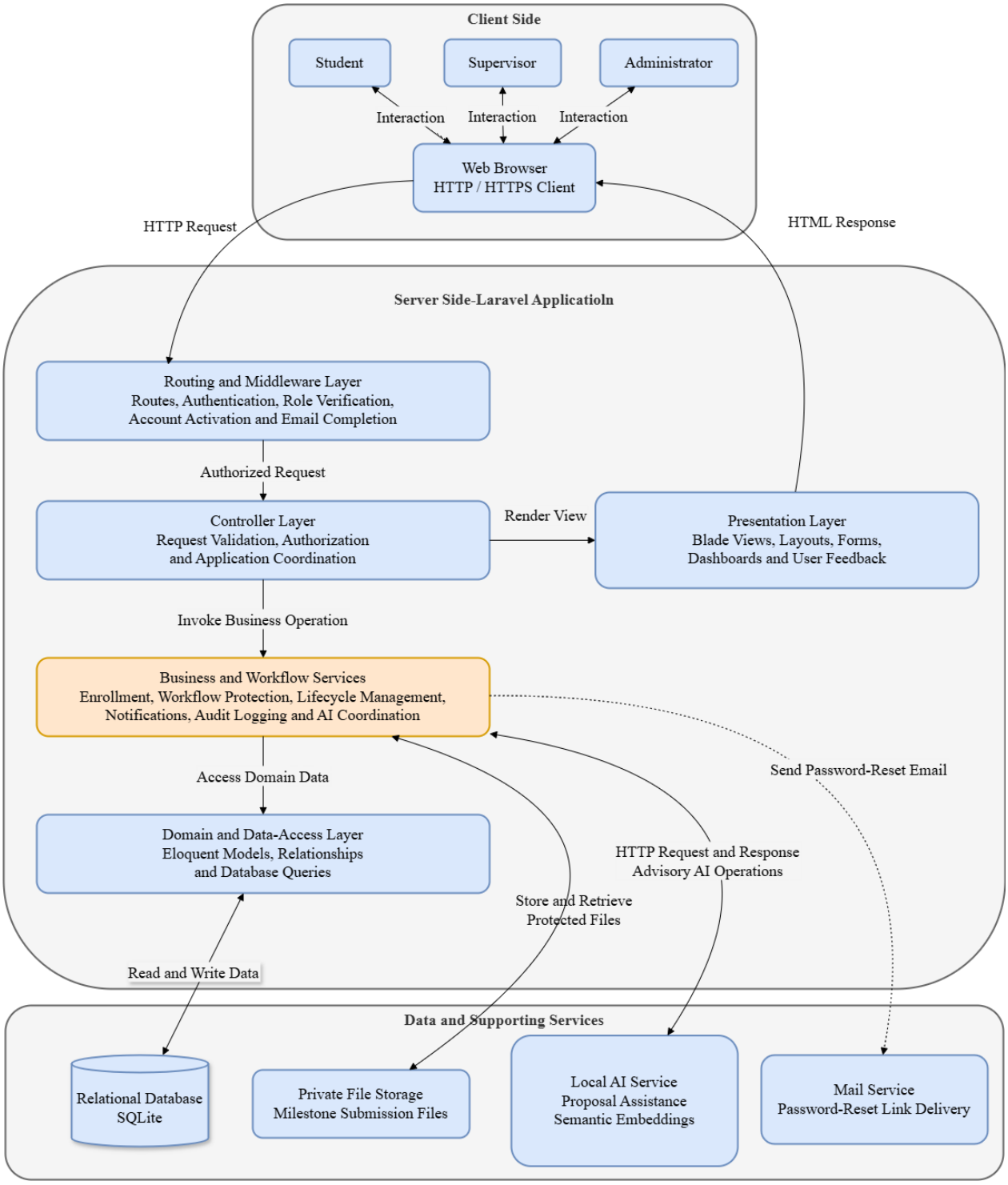


Figure 16 Architecture of the Graduation Project Management Platform

5.1.2 Main Architectural Components

The Graduation Project Management Platform consists of a set of logically separated architectural components. Each component has a defined responsibility and communicates with other components through controlled interfaces. This separation reduces coupling, improves maintainability, and supports the enforcement of authentication, authorization, and workflow rules.

Client Layer

The Client Layer represents the web browsers used by Students, Supervisors, and Administrators. Users interact with the platform through role-specific dashboards, forms, project pages, notifications, file-upload interfaces, and communication pages. The browser sends HTTP requests to the Laravel application and displays the returned HTML responses.

Routing and Middleware Layer

The Routing and Middleware Layer receives incoming requests and determines the corresponding application operation. Laravel routes connect request URLs and HTTP methods to controller actions. Before a request reaches a controller, middleware verifies the user's authentication status, account activation, email completion, and assigned role.

This layer prevents unauthorized requests from reaching protected application operations and ensures that access rules are applied consistently across the platform.

Controller Layer

The Controller Layer receives authorized requests, validates user input, coordinates the requested operation, and selects the appropriate response. Controllers communicate with business services and Eloquent models, then return Blade views, redirects, validation messages, file responses, or error responses.

Controllers are responsible for application coordination rather than implementing all workflow rules directly. Complex and reusable operations are delegated to the Business and Workflow Services Layer.

Business and Workflow Services Layer

The Business and Workflow Services Layer contains the platform's principal business rules and reusable workflow operations. It manages processes such as project enrollment, application evaluation, project lifecycle consistency, milestone submission handling, internal notifications, activity logging, proposal assistance, and semantic similarity analysis.

This layer also applies shared workflow restrictions, including preventing duplicate pending applications, preventing a Student from joining more than one active project, validating team membership, and ensuring that only the responsible Supervisor can process project requests and ideas.

Artificial intelligence operations are coordinated through this layer. The local AI service provides advisory proposal suggestions and semantic similarity analysis. These operations do not independently accept, reject, or block a submitted project idea.

Model and Data-Access Layer

The Model and Data-Access Layer uses Laravel Eloquent models to represent the main system entities and their relationships. Models provide a structured interface for retrieving, creating, updating, and relating persistent records.

This layer includes entities representing users, supervisor profiles, projects, project members, join requests, project ideas, milestone submissions, messages, announcements, notifications, roles, and activity-log records.

Eloquent relationships allow the application to navigate related data while reducing the need for direct SQL statements inside controllers and services.

Presentation Layer

The Presentation Layer consists of Laravel Blade templates, shared layouts, reusable interface components, forms, dashboards, validation messages, and role-specific pages. It presents system data to users and provides the controls needed to execute authorized operations.

The displayed interface changes according to the authenticated role and the current workflow state. For example, the Student dashboard differs depending on whether the Student is exploring projects, waiting for an application decision, or already enrolled in a project.

Relational Database

The Relational Database stores the persistent operational data of the platform. It maintains account information, roles, projects, team membership, application records, project ideas, submissions, communication records, notifications, activity logs, and similarity-analysis snapshots.

Database relationships and constraints support referential integrity and reduce the possibility of inconsistent records. The current implementation uses SQLite, while the application accesses the database through Laravel Eloquent ORM.

Private File Storage

Private File Storage contains milestone-submission files uploaded by Students. These files are not exposed through direct public URLs. Download requests are handled by the Laravel application, which verifies that the requesting user is an authorized project member, the responsible Supervisor, or another explicitly permitted role.

This design separates file storage from public interface assets and prevents unauthorized users from accessing project documents by guessing file locations.

Local Artificial Intelligence Service

The Local Artificial Intelligence Service is provided through a locally hosted Ollama environment. It supports two advisory operations: generating a proposal suggestion from the Student's idea description and producing semantic embeddings for project-idea similarity analysis.

Communication between Laravel and Ollama is performed through HTTP requests initiated by the application services. The AI service is designed as an optional supporting component. Therefore, its unavailability does not prevent the Student from completing the principal idea-submission workflow.

Mail Service

The Mail Service is used to deliver password-reset links to the email address stored in the user account. Email is not used as the authentication identifier. Users log in using their university number and password, while email is used only for account recovery and related communication.

5.1.3 Component Interaction

The processing of a typical platform request follows a controlled sequence. First, a user performs an action through the web browser, such as submitting a project application, uploading a milestone file, or reviewing a project idea. The browser sends an HTTP request to the Laravel application.

The routing layer identifies the appropriate controller action, while middleware verifies authentication, account status, role, and other applicable access conditions. When the request is authorized, the controller validates the submitted data and invokes the required business service.

The business service applies the relevant workflow rules and communicates with Eloquent models to retrieve or modify persistent data. The models execute the required database operations and return the result to the service and controller.

The controller then returns an appropriate response. This response may be a rendered Blade view, a redirect with a success or validation message, a notification update, or an authorized file download.

For project-idea operations, the business service may also communicate with the local artificial intelligence service. For password recovery, the application communicates with the mail service. File-submission operations communicate with private storage only after authorization and workflow checks have succeeded.

5.1.4 Architectural Style Justification

The layered monolithic MVC architecture was selected because it is appropriate for the platform's functional scope, development environment, and deployment requirements. The platform's modules are strongly related and share the same users, roles, projects, workflows, and relational data. Deploying them as one integrated Laravel application reduces operational complexity while maintaining logical separation between system responsibilities.

The MVC pattern separates interface rendering, request coordination, and persistent domain data. Middleware centralizes authentication and access checks, while reusable service classes centralize workflow rules that are shared across several controller operations.

This architecture is more suitable for the project than a microservices architecture. Microservices would introduce additional deployment, communication, monitoring, and data-consistency complexity without providing a proportionate benefit for the current system scale.

The selected architecture also supports future improvement. Individual services, such as artificial intelligence processing, mail delivery, or file storage, can be replaced or extended through configuration without redesigning the principal application workflows.

5.2 Database Design

The database design of the Graduation Project Management Platform translates the system requirements and workflows into a relational data structure. The design supports identity and role management, Supervisor profiles, graduation projects, team membership, project applications, Student-proposed ideas, milestone submissions, communication, notifications, and activity auditing.

Laravel Eloquent ORM is used as the data-access mechanism between the application and the relational database. The current development environment uses SQLite, while the database connection remains configurable through the Laravel environment settings. Database migrations are used to create and evolve the schema in a controlled and reproducible manner.

5.2.1 Entity–Relationship Diagram

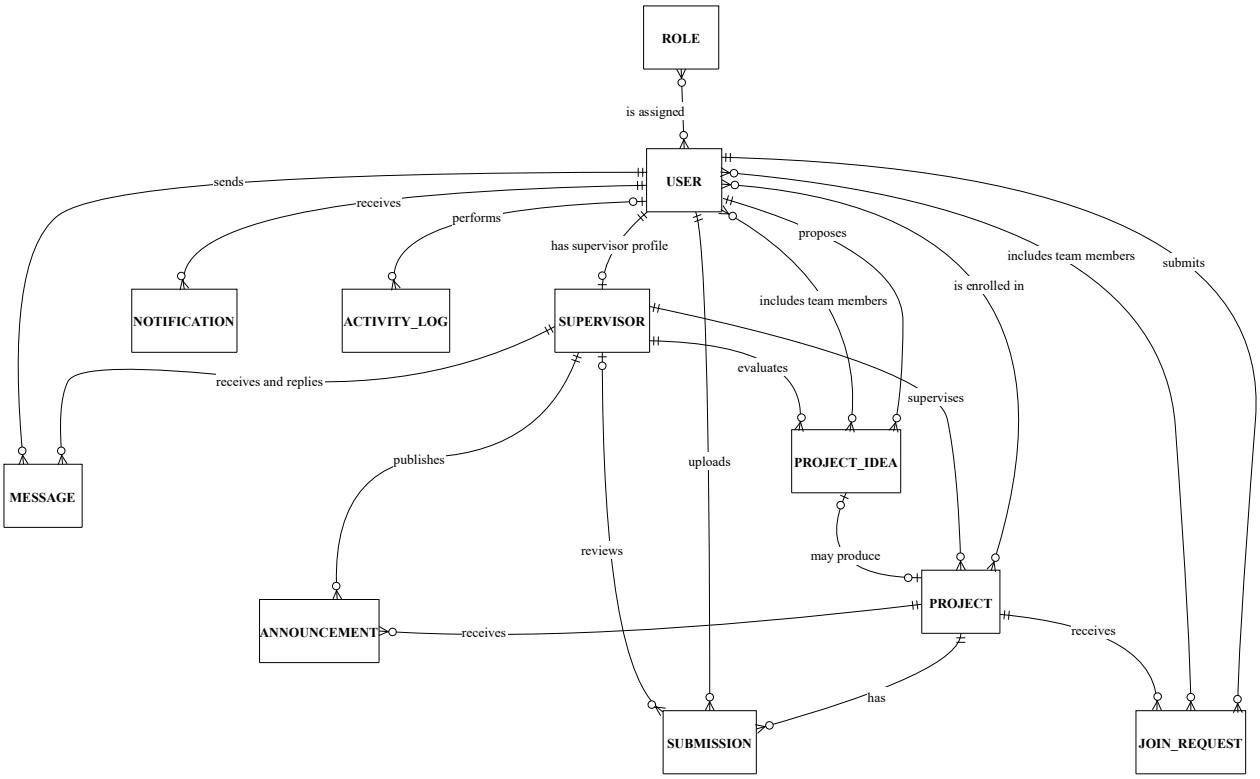


Figure 17 conceptual Entity–Relationship Diagram of the Graduation Project Management Platform

5.2.2 Relational Schema

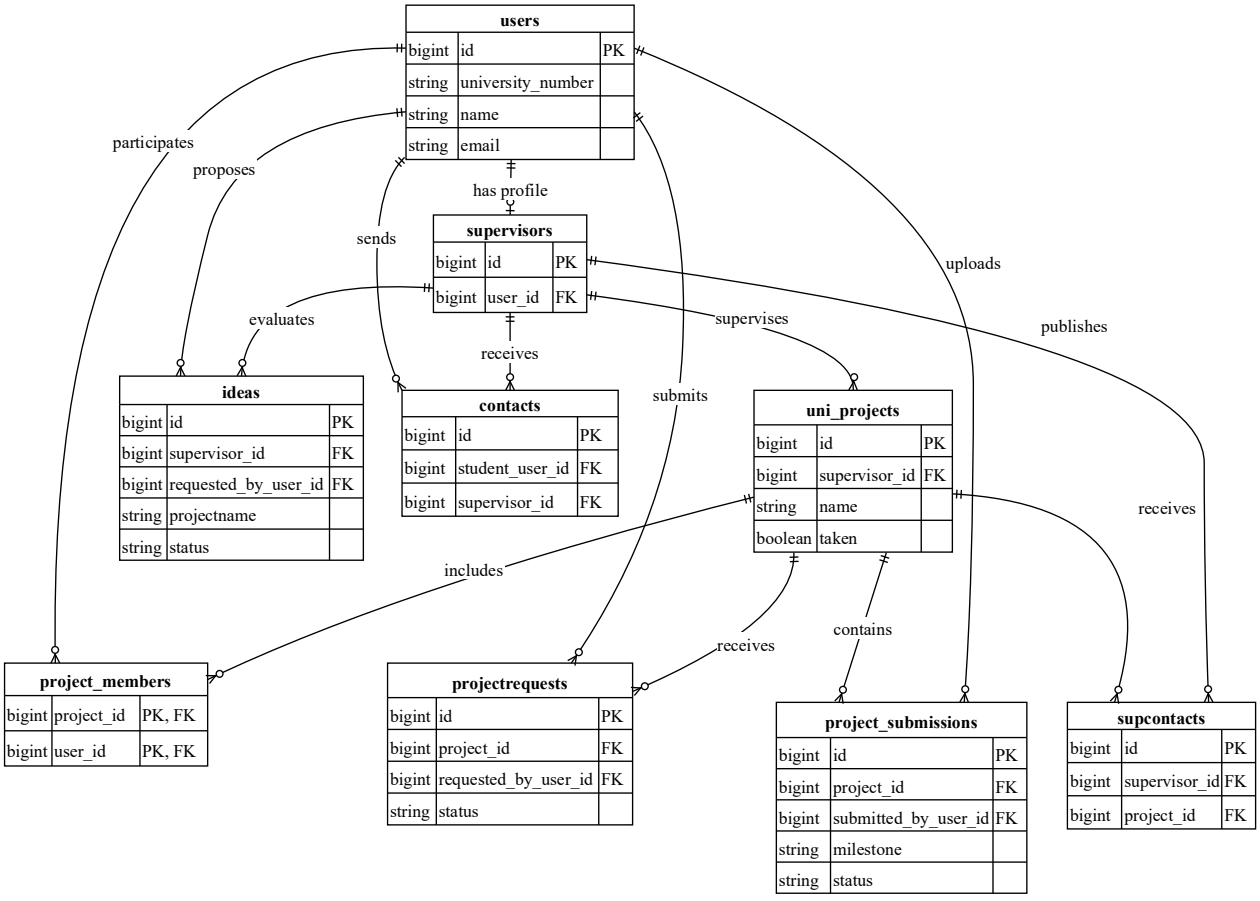


Figure 18 High-Level Relational Schema of the Graduation Project Management Platform

5.3 Interface Design

The system uses a limited set of HTTP-based interfaces for its local artificial intelligence functions. These interfaces include two internal JSON endpoints consumed by the Student dashboard and two outbound interfaces used by the Laravel application to communicate with the local Ollama service. Each interface is documented according to its endpoint, HTTP method, input parameters, response, and error codes.

5.3.1 Internal Application Interfaces

Table 18 Internal JSON Interfaces of the Graduation Project Management Platform

Interface	Endpoint	Method	Parameters	Response	Error Codes
AI Proposal Assistance	/student/ai/proposal	POST	raw_idea: textual description of the initial project idea. The input must satisfy the configured minimum and maximum length limits.	200 OK JSON containing ok, mode, disclaimer, and a structured suggestion with a title, problem statement, objectives, scope, and functional requirements.	401/403 for invalid access, 422 for validation failure, and 429 when the request limit is exceeded. Ollama failure is handled through a successful fallback response rather than a workflow error.
Semantic Similarity Check	/student/ai/similarity	POST	title: current project title. description: current proposal description.	200 OK privacy-safe JSON containing the similarity status, the highest relevant matches, similarity percentages and levels, source types, titles, and an advisory disclaimer.	401/403 for invalid access, 422 for validation failure, and 429 when the request limit is exceeded. If the embedding service is unavailable, the interface returns a soft-unavailable result without blocking idea submission.

Both internal interfaces are restricted to authenticated Students through Laravel middleware. The proposal-assistance interface is limited to ten requests per minute, while the similarity-check

interface is limited to six requests per minute. Neither interface submits a project idea, modifies its acceptance status, or notifies a Supervisor. The Student remains responsible for reviewing the generated information and explicitly submitting the final idea.

5.3.2 Local Ollama Service Interfaces

Interface	Endpoint	Method	Parameters	Response	Error Codes
Proposal Generation	<AI_BASE_URL>/api/chat	POST	model: configured chat model, normally llama3.2:3b; messages: system and user prompts; stream: false; format: json.	JSON response containing the generated structured proposal in message.content. The application parses and normalizes this content before returning it to the Student interface.	Connection failure, timeout, 404 when the model is unavailable, or 5xx service errors. These conditions are converted into a deterministic fallback proposal.
Text Embedding Generation	<AI_BASE_URL>/api/embed	POST	model: nomic-embed-text; input: the project title and description or a batch of corpus documents.	JSON response containing numerical embedding vectors. The vectors are used temporarily to calculate cosine similarity within the Laravel application.	Connection failure, timeout, 404 when the model is unavailable, or 5xx service errors. These conditions produce an unavailable similarity result without interrupting the main workflow.

The Ollama service is configured through the application environment and normally runs locally at http://localhost:11434. The Laravel application communicates with Ollama only from the server side; the browser does not call Ollama directly. Proposal suggestions and embedding vectors are

request-scoped and are not stored in the database. Only the Student-authored proposal description and the privacy-safe final similarity snapshot are persisted with the submitted idea.

5.4 Security and Authorization Design

Security in the Graduation Project Management Platform is implemented through multiple complementary controls at the authentication, authorization, session, data-access, file-storage, workflow, and external-service levels. The security design is based on Laravel’s web-security mechanisms together with application-specific middleware, role checks, resource-ownership validation, workflow guards, and activity auditing.

The platform uses session-based authentication rather than token-based authentication.

5.4.1 Authentication Design

The platform provides a single authentication entry point for Students, Supervisors, and Administrators. Users authenticate using their university number and password. The university number represents the primary login identifier, while the email address is used for password-recovery delivery and account-profile completion.

User accounts are created by the Administrator, and public self-registration is disabled. This design prevents unauthorized users from creating accounts or assigning roles to themselves. After successful authentication, the system determines the authenticated user's role and redirects the user to the corresponding portal.

Passwords are not stored in plain text. Laravel's one-way password-hashing mechanism is used before passwords are persisted in the database. Authentication compares the submitted password with the stored hash without exposing the original password.

The authentication process also verifies that the account is active. Deactivated accounts are rejected during login and are prevented from continuing to access protected pages during an existing session. When an account does not contain an email address, the user is redirected to the email-completion page before accessing the main portal.

After successful login, the session identifier is regenerated to reduce session-fixation risks. During logout, the authenticated session is terminated, invalidated, and assigned a new CSRF token. Password-recovery responses are presented generically to avoid exposing whether a university number or email address exists in the system. These controls are implemented through Laravel session authentication and application middleware.

5.4.1 Authorization and Role-Based Access Control

Authorization is implemented using role-based access control. Laratrust is used to assign roles to users, while Laravel middleware protects each role-specific portal. The platform defines three operational roles: Administrator, Student, and Supervisor.

Protected routes pass through authentication, account-status, email-completion, and role-specific middleware. Consequently, a user must be authenticated, active, email-complete, and assigned to the correct role before accessing a protected operation.

The Administrator role is authorized to provision Student and Supervisor accounts, activate or deactivate users, reset user passwords, monitor system records, view summary indicators, and inspect the activity log. Administrative access to projects, applications, ideas, and submissions is primarily read-only; the Administrator does not perform Supervisor decisions or download project-submission files.

The Student role is authorized to browse available projects, submit project applications and ideas, access the assigned project workspace, upload and download authorized submissions, communicate with the responsible Supervisor, and use the advisory artificial intelligence functions.

The Supervisor role is authorized to create and update owned projects, review applications and proposed ideas assigned to the Supervisor, review submissions belonging to supervised projects, reply to Student messages, and publish project announcements.

Role checks are complemented by resource-level authorization. A valid role alone is not sufficient to access every record. The system verifies project membership, project ownership, application ownership, and Supervisor responsibility before permitting sensitive actions. Notifications are also scoped to the authenticated user, preventing one user from reading or modifying another user's notifications.

5.4.2 HTTPS and Deployment Considerations

The current implementation is developed and evaluated in a local development environment. HTTPS enforcement is therefore considered a deployment-level responsibility rather than a feature of the local application environment.

A production deployment should place the Laravel application behind an HTTPS-enabled web server or reverse proxy, configure a valid TLS certificate, redirect HTTP requests to HTTPS, enable secure session-cookie settings, protect environment configuration files, restrict database and Ollama network access, and establish backup and monitoring procedures.

The system documentation does not claim that transport encryption, a web application firewall, antivirus scanning, or an independent security audit is already provided by the local development environment.

The security design combines session-based authentication, role-based authorization, resource-ownership checks, private file storage, workflow validation, audit logging, and privacy-aware local artificial intelligence integration. These controls protect the principal academic workflows while maintaining a clear separation between implemented application-level protections and additional controls required for a production deployment.

Chapter 6: Development Environment and Software Implementation

This chapter documents the development environment, software tools, programming languages, frameworks, libraries, dependencies, source-control practices, testing environment, and implementation structure used to build the Graduation Project Management Platform.

6.1 Software Tools and Libraries

Table 19 Software Tools Used in the Development of the Platform

Tool	Version or Configuration	Purpose
Visual Studio Code	Desktop edition	Auxiliary source-code inspection and editing
PHP	8.5.7 runtime	Server-side execution environment
Laravel	13.x	Main web-application framework
Composer	2.10.1	PHP package and dependency management
SQLite	Version 3, accessed through PDO SQLite	Local relational database
TablePlus	Desktop edition	Database inspection and data verification
Git	Version 2.38.1	Local source-code version control
GitHub	Hosted repository service	Remote repository, branch storage, and change history
Postman	Desktop edition	Inspection and testing of HTTP-based interfaces
Ollama	Local runtime	Execution of the local language and embedding models
draw.io	Desktop edition	Preparation of architectural, UML, and database diagrams
Visual Paradigm	Desktop edition	Supporting UML modeling and diagram review
Microsoft Word	Microsoft 365	Academic report preparation and formatting
Google Chrome and Microsoft Edge	Current desktop releases	Manual interface and compatibility testing

6.2 Development Environment and Project Structure

6.2.1 Development Environment

The principal development environment consisted of the following components:

- **Operating System:** Windows 11, 64-bit.
- **Primary IDE:** Visual Studio Code.
- **Local Runtime Environment:** Laravel Herd.
- **PHP Runtime:** PHP 8.5.7.
- **Application Framework:** Laravel 13.
- **Database Environment:** SQLite.
- **Local AI Runtime:** Ollama.
- **Source Management:** Git and GitHub.
- **Testing Framework:** Pest.

Laravel Herd was used instead of a manually configured Apache or XAMPP environment. It provided the local PHP runtime and simplified the execution of the Laravel application. SQLite was used as the development database because it requires no separate database server and can be configured directly through the Laravel environment file.

6.2.2 Project Directory Structure

Figure 19 presents the high-level directory structure of the Graduation Project Management Platform. The structure follows Laravel conventions and highlights the directories responsible for request handling, business logic, data access, interface rendering, configuration, database management, storage, and automated testing.

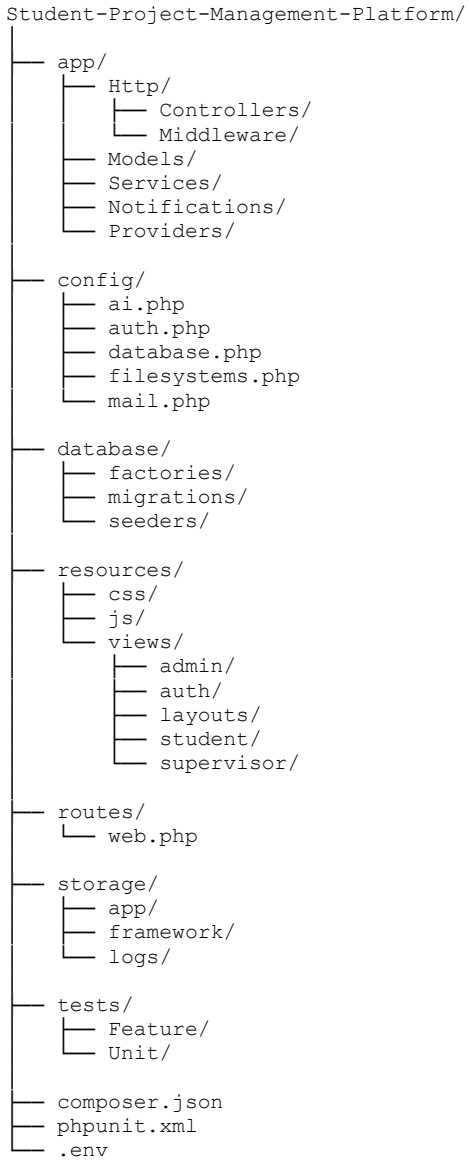


Figure 19 Directory Structure of the Graduation Project Management Platform

6.3 Database Server and Operating System

6.4.1 Database Access and Management

Database access is performed through Laravel Eloquent ORM rather than through manually constructed SQL statements in the main application workflows. Eloquent provides object-oriented access to database records and supports relationships, query construction, pagination, and controlled record updates.

Multi-record workflow operations use database transactions where required. These operations include account provisioning, project-application submission, project-idea submission, and acceptance workflows that create project-membership records. Transactions ensure that related changes are completed together or rolled back when an operation fails.

TablePlus was used during development to inspect the SQLite database, verify migration results, review stored records, and confirm the effects of workflow operations. Direct database editing was not used as part of the normal user workflows; application data is created and modified through validated Laravel operations.

6.4.2 Operating System and Local Runtime

The platform was developed and tested on a 64-bit Windows 11 operating system. Visual Studio Code was used as the primary integrated development environment, while Laravel Herd provided the local PHP runtime and web-serving environment.

The final development environment uses PHP 8.5.7 with Laravel 13. Laravel Herd manages the PHP runtime and allows the application to run locally without requiring a manually configured Apache, Nginx, or XAMPP installation.

The SQLite database, Laravel application, private file storage, and local Ollama artificial intelligence runtime were executed within the local development environment. The platform was accessed and manually evaluated through modern desktop web browsers.

6.4 Programming Languages and Frameworks

The Graduation Project Management Platform was implemented using PHP as the principal server-side programming language and Laravel as the main web-application framework. HTML, CSS, and JavaScript were used to construct the browser-based user interfaces, while Laravel Blade was used as the server-side template engine.

1.4.1PHP

PHP is the primary server-side programming language used to implement the application logic. It is responsible for request processing, input validation, authentication, authorization, workflow execution, database access, file handling, notification generation, and communication with the local artificial intelligence service.

1.4.2Laravel Framework

Laravel 13 is used as the principal application framework. It provides the structural foundation for routing, middleware, session-based authentication, request validation, database migrations, file storage, notifications, password recovery, and automated testing.

1.4.3Blade Template Engine

Blade is Laravel's server-side template engine and is used to construct the interfaces displayed to Students, Supervisors, Administrators, and unauthenticated users. Blade templates combine HTML markup with controlled server-side expressions, reusable layouts, partial views, conditional rendering, and form components.

1.4.4HTML and CSS

HTML is used to define the semantic structure of the platform's pages, including forms, navigation elements, dashboards, tables, project cards, submission records, messages, notifications, and modal dialogs.

CSS is used to provide the visual design, responsive layouts, role-specific interface themes, form styling, status badges, navigation components, and dashboard presentation. The application mainly uses custom CSS files organized according to interface area and component responsibility.

1.4.5JavaScript

JavaScript is used on the client side to provide interactive behavior without replacing Laravel's server-rendered architecture. Its responsibilities include dashboard-tab switching, interface-state updates, form assistance, modal interaction, project filtering, and communication with the internal JSON interfaces used by the artificial intelligence features.

The combination of PHP, Laravel, Blade, HTML, CSS, and JavaScript provides a unified technology stack suitable for the platform's academic workflows. Laravel and PHP implement the server-side logic and security controls, Blade generates the application pages, and the front-end technologies provide responsive and interactive user interfaces.

6.5 Libraries and Dependencies

dependencies. The required packages and their version constraints are declared in the composer.json file, while the composer.lock file records the exact installed versions used in the final development and testing environment.

Composer supports reproducible dependency installation and separates the application's operational dependencies from libraries used only during development and automated testing. Table 21 presents the principal direct dependencies installed in the project and explains their actual roles within the platform.

Table 20 Principal Libraries and Dependencies Used by the Platform

Dependency	Version	Category	Purpose in the Platform
laravel/framework	13.18.1	Application Framework	Provides routing, middleware, session authentication, validation, Blade rendering, Eloquent ORM, migrations, notifications, file storage, mail integration, and testing support.
santigarcor/laratrust	8.5.5	Role Management	Assigns the Administrator, Student, and Supervisor roles and supports role-based route protection.
guzzlehttp/guzzle	7.13.1	HTTP Client	Provides server-side HTTP communication between the Laravel application and the local Ollama service.
laravel/sanctum	4.3.2	Authentication Package	Installed as an application dependency; however, the platform's principal workflows use Laravel session-based web authentication rather than Sanctum API tokens.
laravel/tinker	3.0.2	Development Utility	Provides an interactive environment for inspecting models, executing Laravel operations, and verifying application data.
pestphp/pest	4.7.4	Testing Framework	Provides the primary syntax and execution environment for automated feature and unit tests.
pestphp/pest-plugin-laravel	4.1.0	Testing Integration	Integrates Pest with Laravel application bootstrapping, authentication helpers, database utilities, and HTTP assertions.
fakerphp/faker	1.24.1	Test-Data Generation	Generates controlled sample data for model factories, seeders, and automated tests.

Dependency	Version	Category	Purpose in the Platform
mockery/mockery	1.6.12	Mocking Library	Supports dependency isolation and mocked objects during automated tests.
nunomaduro/collision	8.9.4	Development Utility	Improves command-line error, exception, and test-failure presentation.
laravel/pint	1.29.3	Code Formatting	Applies consistent PHP code-formatting rules during development.
spatie/laravel-ignition	2.12.0	Development Diagnostics	Provides detailed local exception pages and debugging information during development.

6.6 Version Control and Source Management

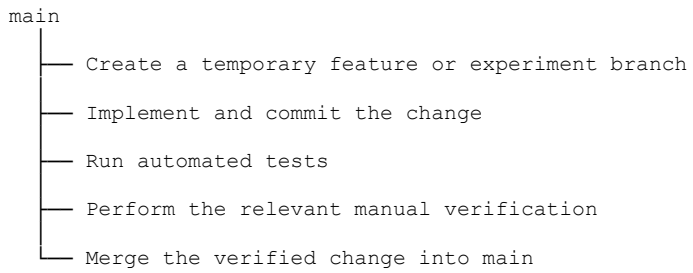
Git was used as the primary source-control and software-configuration-management tool during the development of the Graduation Project Management Platform. It maintained the local history of source-code changes, database migrations, configuration files, automated tests, interface resources, and project documentation.

GitHub was used as the remote repository for preserving the project history and synchronizing the tested source code. The repository contains the Laravel application directories, automated tests, configuration templates, dependency files, and report resources. At the time of documentation, the repository maintains main as its default branch and contains a structured history of development commits.

6.6.1 Branching Strategy

The project used a lightweight branch-based development strategy rather than a complete GitFlow model. The main branch was maintained as the integrated and stable version of the platform. Changes with a limited and well-understood scope could be committed after verification, while risky or extensive technical changes were developed on temporary isolated branches.

After the automated test suite and the required manual workflows were verified, the accepted changes were integrated into main. Temporary branches were then removed when they were no longer required. Consequently, the current remote repository exposes main as the default active branch rather than maintaining permanent develop, release, or hotfix branches.



This strategy was suitable for the project because development was performed by a small team and the majority of changes were closely related within one Laravel application. Temporary branches provided sufficient isolation for higher-risk changes without introducing the overhead of a multi-branch enterprise workflow.

6.7 Testing Environment

testing and manual workflow verification. Automated tests were executed locally using Pest and its Laravel integration, while manual testing was performed through modern web browsers using controlled Student, Supervisor, and Administrator accounts.

The testing environment was separated from normal application data through Laravel's testing configuration. SQLite was used as the relational database during automated tests, allowing the database structure and test records to be recreated in a controlled environment without modifying the primary development database.

6.7.1 Automated Testing Environment

Plugin version 4.1.0. The tests were organized under the tests/Feature and tests/Unit directories.

Feature tests verify complete application behaviors that involve routes, middleware, controllers, services, database records, authorization rules, redirects, validation responses, and rendered pages. Unit tests focus on isolated logic where appropriate, including reusable workflow and service behavior.

The automated tests cover the principal system areas, including:

- authentication and session handling;
- account activation and email-completion enforcement;
- role-based and resource-level authorization;
- Administrator account-management operations;

- project and idea application workflows;
- project enrollment and team membership;
- milestone submission and review;
- private file-access restrictions;
- Student–Supervisor communication;
- notifications and activity logging;
- proposal-assistance behavior;
- semantic-similarity processing and fallback behavior.

6.7.2 Manual Testing Environment

browsers. Controlled accounts were used for the Student, Supervisor, and Administrator roles to verify role-specific dashboards, navigation, forms, validation messages, workflow transitions, and interface behavior.

Manual verification followed representative end-to-end workflows, including:

- logging in with each operational role;
- creating and managing Student and Supervisor accounts;
- browsing graduation projects;
- submitting and evaluating project applications;
- submitting and evaluating project ideas;
- creating accepted project membership;
- uploading and reviewing milestone submissions;
- exchanging messages and announcements;
- viewing notifications and activity records;
- using proposal assistance and semantic-similarity analysis.

6.8 Software Implementation

6.8.1 Main Module Implementation

6.8.1.1 Access and Identity Module

The Access and Identity Module implements the common entry point used by Students, Supervisors, and Administrators. Authentication is performed using the university number and

password through Laravel's session-based authentication mechanism. After successful authentication, the user is redirected to the portal associated with the assigned role.

Public account registration is disabled. Student and Supervisor accounts are created through the Administrator portal, while Administrator accounts are prepared through controlled system initialization. This prevents users from creating accounts or assigning roles to themselves.

The module also implements secure logout, password recovery, token-based password reset, missing-email completion, account-activation enforcement, and role-specific route protection. Laravel middleware verifies that protected users are authenticated, active, email-complete, and assigned to the required role before allowing access to a portal.

The principal implementation components of this module include:

- UserController for login, logout, dashboard routing, and account operations;
- Laravel password-reset services for recovery and reset operations;
- EnsureAccountActive for blocking deactivated accounts;
- EnsureEmailComplete for enforcing profile completion;
- role middleware for Administrator, Student, and Supervisor access;
- User and Supervisor Eloquent models for identity and profile data.

6.8.1.2 Administrator Module

The Administrator Module provides institutional oversight and user-account management. Its dashboard calculates summary indicators related to users, projects, requests, ideas, submissions, and Supervisor workloads.

Administrators can list user accounts, create Student accounts, create Supervisor accounts with linked Supervisor profiles, activate or deactivate accounts, and reset another user's password. The implementation includes safeguards that prevent inappropriate account-management operations, such as deactivating the final active Administrator or using the administrative password-reset operation on the current Administrator account.

The module also provides read-only monitoring pages for graduation projects, join requests, project ideas, and milestone submissions. Administrators can inspect these records but do not accept or reject academic applications and do not perform Supervisor submission reviews.

Important account-management and workflow operations are recorded through the ActivityLogger service and displayed through the administrative activity-log page. The actual system analysis confirms that the Administrator performs account lifecycle management and read-only platform oversight rather than Supervisor academic decisions.

6.8.1.3 Student Module

The Student Module is implemented through a state-aware dashboard that adapts according to the Student's current project status. The interface distinguishes among the discovery state, pending-application state, and enrolled-project state.

In the discovery state, the Student can browse available graduation projects and Supervisors, submit a request to join an existing project, or propose a new project idea. Team members are entered using university numbers and are validated before the request or idea is stored.

In the pending state, the Student can follow the acceptance or rejection status of submitted join requests and project ideas. Rejection reasons are displayed when supplied by the responsible Supervisor.

In the enrolled state, the Student can view the assigned project, team members, milestone timeline, and progress. The Student can upload milestone files, download authorized files belonging to the assigned project, send messages to the responsible Supervisor, view Supervisor replies and announcements, manage notifications, and change the account password.

The Student dashboard obtains its operating state through StudentEnrollmentService. This prevents interface logic from depending on scattered database checks and provides a consistent interpretation of whether the Student is exploring, waiting for a decision, or officially enrolled.

The Student Module also contains the two advisory artificial intelligence tools: the Project Proposal Assistant and Semantic Similarity Assistant. These functions assist the Student while preparing an idea but do not automatically submit the idea or determine its academic decision.

6.8.1.4 Supervisor Module

The Supervisor Module provides an operational dashboard for managing owned graduation projects and evaluating Student workflows. Supervisors can create new projects, update projects they own, and view the enrolled teams associated with those projects.

Incoming join requests and proposed project ideas are displayed only to the responsible Supervisor. The Supervisor may accept or reject each pending application, subject to ownership and workflow validation. Accepting a request or idea creates the official project-membership records and updates the corresponding project state.

The module also allows Supervisors to read the proposal description attached to a Student idea and view the stored advisory similarity snapshot. Similarity information is presented only as supporting information and does not automatically accept or reject the idea.

Supervisors can review milestone submissions associated with their projects, download privately stored files, assign the status Approved or Needs Revision, and add textual feedback. They can also reply to Student messages, publish announcements for project teams, manage notifications, and change their passwords.

Resource-level checks ensure that a Supervisor cannot modify another Supervisor's projects, decide on unrelated applications, or review submissions belonging to unrelated teams.

6.8.1.5 Artificial Intelligence Support Module

The Artificial Intelligence Support Module contains two independent advisory functions: proposal assistance and semantic-similarity analysis.

The Proposal Assistant receives a short Student-written description and attempts to generate a structured proposal containing a suggested title, problem statement, objectives, scope, and functional requirements. The operation is coordinated by AiProposalController and ProjectProposalAssistantService. When the local model is unavailable, the service provides a deterministic editable fallback.

The Semantic Similarity Assistant compares the current proposal with existing graduation projects and previously accepted ideas. ProjectSimilarityService prepares the comparison corpus, OllamaEmbeddingService requests numerical embeddings, and CosineSimilarity calculates similarity scores in PHP.

A pre-submission similarity check returns a privacy-safe set of advisory matches without storing embedding vectors. When the final idea is submitted, the server recalculates the result and stores only a limited top-match snapshot with the idea record. Similarity values sent by the browser are ignored.

Both functions preserve human control. Generated proposals require explicit Student review and application to the form, and similarity results neither block submission nor determine the Supervisor's decision. The implemented AI integration remains part of the Laravel application and communicates with a locally configured Ollama runtime; it is not implemented as a separate microservice.

The principal modules collectively implement the complete graduation-project lifecycle, beginning with controlled account access and project discovery and continuing through applications, enrollment, milestone submissions, review, communication, notifications, audit records, and advisory artificial intelligence support.

6.8.2 Data-Access Layer Implementation Using Eloquent ORM

The data-access layer of the Graduation Project Management Platform is implemented using Laravel Eloquent ORM. Eloquent maps the principal relational database tables to PHP model classes and allows the application to retrieve, create, update, and relate records using object-oriented operations rather than manually written SQL statements.

Controllers and service classes use Eloquent models to access persistent data, while database migrations define the physical schema, foreign keys, unique constraints, indexes, and column types. This separation allows the application logic to operate through domain-oriented model relationships while the database preserves structural integrity.

6.8.2.1 Model Relationships

Eloquent relationships express the associations defined in the relational schema. They allow controllers and services to navigate from one domain object to related data without manually joining tables in each workflow.

The principal relationships include:

- a user may have one Supervisor profile;
- a Supervisor may manage multiple graduation projects;
- a graduation project may contain multiple Student members;
- a Student may participate in a project through the project_members junction table;
- a project request belongs to a project and to the Student who submitted it;
- a project idea belongs to a Supervisor and to the Student who submitted it;
- project requests and ideas contain proposed team members through separate junction tables;
- a milestone submission belongs to a project and to the user who uploaded it;
- communication records connect Students with Supervisors;
- announcements connect Supervisors with graduation projects;
- activity records may refer to an acting user and a target user.

6.8.2.2 Representative Eloquent Relationship Implementation

The following implementation excerpt shows how the ProjectSubmission model associates a submission with the authenticated user who uploaded it.

```
public function submittedBy(): BelongsTo
{
return $this->belongsTo(
User::class,
'submitted_by_user_id'
);
}
```

Listing 1 Eloquent Relationship Between a Submission and Its Submitter.

The submittedBy relationship uses the submitted_by_user_id foreign key in the project_submissions table. This identifier is obtained from the authenticated user when the file is uploaded and is not accepted from browser-supplied identity data.

Through this relationship, the platform can display the Student's current account information using:

```
$submission->submittedBy->name
```

The relationship also provides a more reliable source of identity than storing duplicated Student names or email addresses inside each submission record. The submission normalization process established submitted_by_user_id as the relational source of truth and connected it to users.id.

6.8.2.3 Relational Membership and Junction Tables

Many-to-many and temporary team relationships are implemented through dedicated junction tables. The project_members table represents official enrollment and provides the authoritative association between users and accepted graduation projects.

The project_request_members table stores the team proposed in a request to join an existing project, while idea_members stores the team proposed with a new project idea. These records remain separate from official project membership until the responsible Supervisor accepts the corresponding application.

Position values preserve the submitted ordering of team members. Database constraints prevent the same user from being repeated within the same request, idea, or accepted project membership. The relational model therefore supports team validation and lifecycle consistency at both the application and database levels.

6.8.2.4 Database Transactions

Eloquent operations are executed inside database transactions when a workflow modifies several related records. Transactions are particularly important when submitting or accepting project applications and project ideas.

For example, accepting an application may require the platform to:

1. verify that the application is still pending;
2. confirm that the project is still available;
3. update the application decision;
4. create official project-membership records;
5. update the project lifecycle state;

6. generate activity and notification records.

A database transaction ensures that these changes are committed as one logical operation. When an exception occurs, the associated database changes are rolled back rather than leaving partial membership or inconsistent application states.

6.8.2.5 Data-Access Authorization

Eloquent relationships are also used to support resource-level authorization. Authentication and role middleware determine whether a user may access a general module, while relational queries verify whether the user may access a specific project, request, idea, message, or submission.

Before allowing a Student to download a submission, the application verifies that a matching record exists in project_members for the authenticated user and the submission's project:

```
ProjectMember::where('project_id', $submission->project_id)
    ->where('user_id', Auth::id())
    ->exists();
```

A user who is not a member of the project is denied access. For Supervisor operations, the platform verifies that the project's supervisor_id matches the authenticated user's linked Supervisor profile.

These checks ensure that knowing a database identifier or modifying a request URL does not provide access to an unrelated project resource. Submission upload and download authorization are therefore grounded in authenticated relational identities and project membership rather than browser session names or email strings.

6.8.3 Business-Logic and Service-Layer Implementation

The business-logic layer contains the rules that control the platform's academic workflows independently of interface rendering and database presentation. Although Laravel controllers receive HTTP requests and coordinate responses, reusable and cross-cutting rules are extracted into service classes to reduce duplication and improve consistency.

The principal service classes include WorkflowGuard, StudentEnrollmentService, ActivityLogger, ProjectProposalAssistantService, and ProjectSimilarityService. Each service has a focused responsibility and may be reused by more than one controller or workflow.

Controllers remain responsible for request validation, authorization coordination, service invocation, and response generation. Services implement the reusable decisions and processing rules required by the system. This organization prevents complex workflow rules from being repeated in multiple controllers.

6.8.3.1 Workflow Validation

WorkflowGuard centralizes the principal constraints governing project applications, proposed ideas, team membership, project availability, and enrollment. These rules are applied before records are created or application decisions are finalized.

The service verifies conditions such as:

1. the Student does not already belong to an active graduation project;
2. the Student does not have another conflicting pending application;
3. the selected project remains available;
4. submitted university numbers correspond to valid Student accounts;
5. the proposed team does not contain duplicated members;
6. no team member already belongs to another active project;
7. the request or idea remains in the pending state before a decision;
8. the authenticated Supervisor is responsible for the affected project or idea.

These checks are applied to both project-join requests and Student-proposed ideas. Centralizing the rules ensures that equivalent workflows follow the same restrictions and reduces the risk of inconsistent validation behavior.

Table 21 Principal Workflow Rules Enforced by the Platform

Workflow Rule	Purpose
One active project per Student	Prevents a Student from belonging to multiple graduation projects simultaneously.
No conflicting pending application	Prevents parallel pending requests and ideas that could create inconsistent enrollment decisions.
Valid Student university numbers	Ensures that proposed team members correspond to existing Student accounts.

Workflow Rule	Purpose
No duplicated team members	Prevents the same user from appearing more than once in an application.
Project availability verification	Prevents applications or acceptance operations for an unavailable project.
Pending-state verification	Prevents an application from being accepted or rejected more than once.
Supervisor ownership verification	Restricts academic decisions to the responsible Supervisor.
Transactional acceptance	Ensures that decision, membership, project state, logs, and notifications remain consistent.

The workflow rules are enforced before database mutations are completed. Database constraints provide an additional structural layer, while the service layer provides domain-specific error handling and user-facing validation messages.

6.8.3.2 Separation Between Controllers and Services

Table 22 Responsibilities of Controllers and Service Classes

Component	Principal Responsibility
Controller	Receives the HTTP request, invokes validation, coordinates authorization, calls services, and returns a response or redirect.

Middleware	Applies authentication, account-state, email-completion, and role-level restrictions before controller execution.
Service	Implements reusable business, workflow, enrollment, logging, or artificial intelligence logic.
Eloquent Model	Represents persistent entities, relationships, casts, and query operations.
Blade View	Presents server-generated data and user-interface controls.

This separation is not a complete independent domain-layer architecture. The platform remains a Laravel MVC monolith, but complex or reusable operations are extracted into focused service classes. This provides an appropriate balance between maintainability and architectural simplicity for the project's scope.

The service layer implements the principal rules that preserve workflow consistency across project requests, proposed ideas, enrollment, submissions, auditing, notifications, and artificial intelligence operations. Centralized validation, authenticated identity resolution, database transactions, and reusable services reduce duplication and prevent inconsistent academic states.

6.8.4 User-Interface Implementation

The user interface of the Graduation Project Management Platform is implemented using Laravel Blade templates, HTML, CSS, and native JavaScript. The application follows a server-rendered approach in which controllers prepare the required data and Blade templates generate the pages returned to the browser.

Separate interface areas are provided for Students, Supervisors, and Administrators. Shared layouts, navigation elements, notification controls, status indicators, form components, and validation messages are reused where appropriate to maintain consistency across the platform.

The interface is organized around the responsibilities of each role rather than presenting all system functions within one common dashboard. After authentication, each user is redirected to the portal associated with the assigned role.

6.8.4.1 Authentication Interface

The authentication interface provides one common login entry point for Students, Supervisors, and Administrators. Users enter their university number and password, after which Laravel validates the credentials and redirects the authenticated account to the appropriate portal.

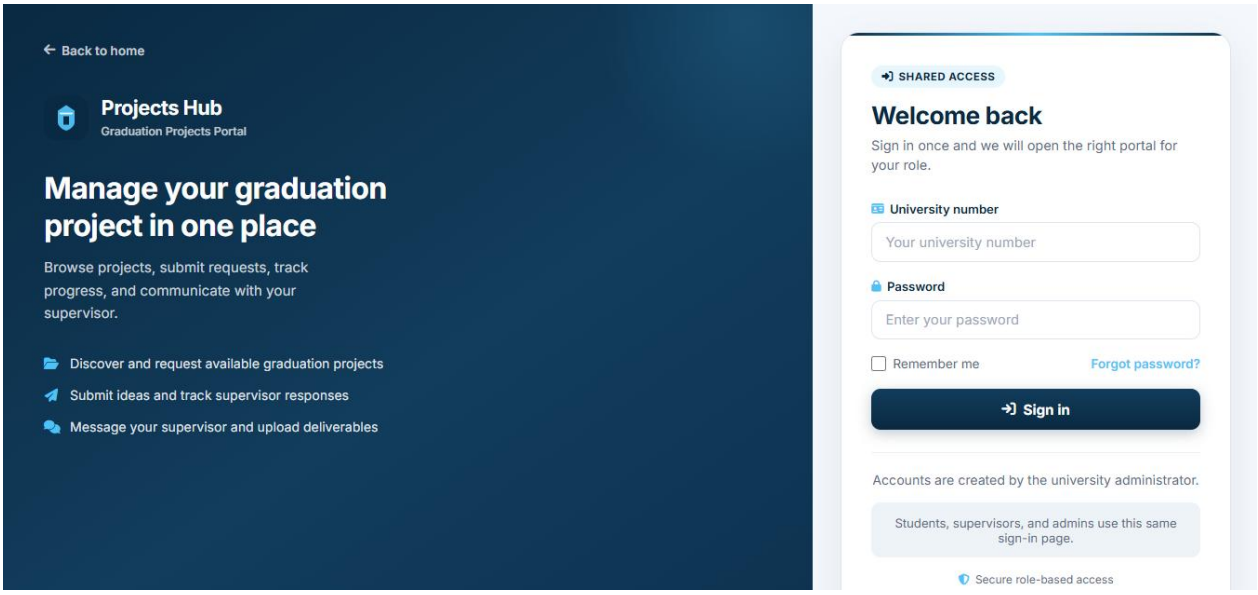


Figure 20 Shared Login Interface.

Figure 21 shows the shared login interface used by all operational roles. The university number acts as the principal authentication identifier, while the authenticated role determines the destination portal.

6.8.4.2 Student Interface

The Student interface uses a state-aware dashboard that changes according to the Student's current graduation-project status. The three principal interface states are discovery, pending application, and active enrollment.

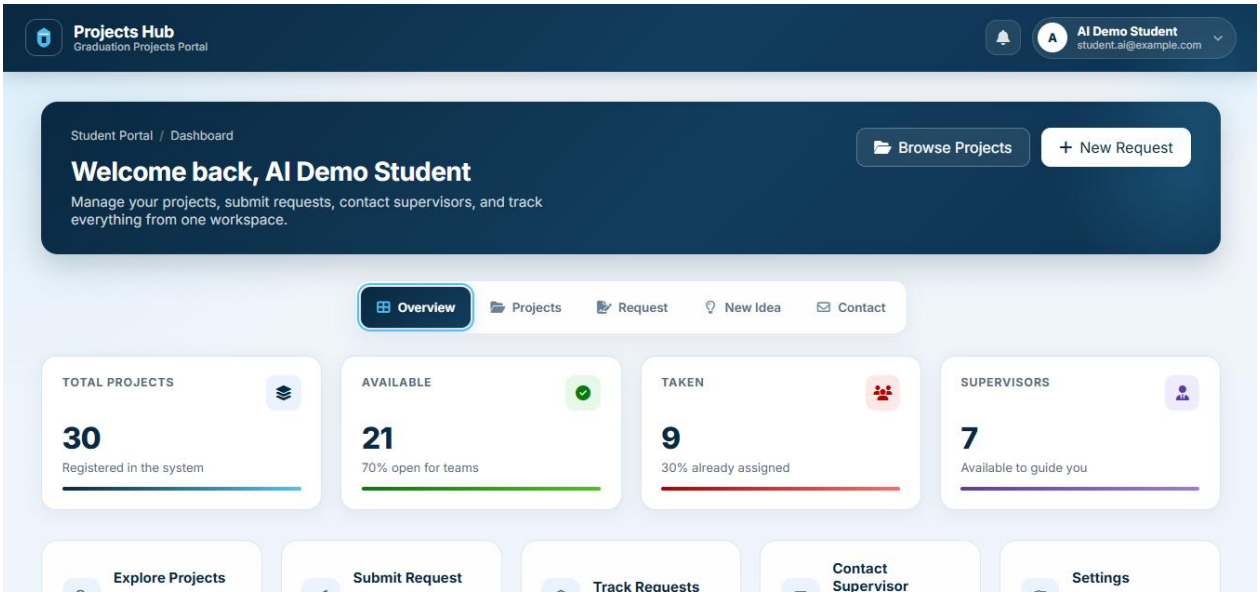


Figure 21 Student Dashboard in the Project-Discovery State.

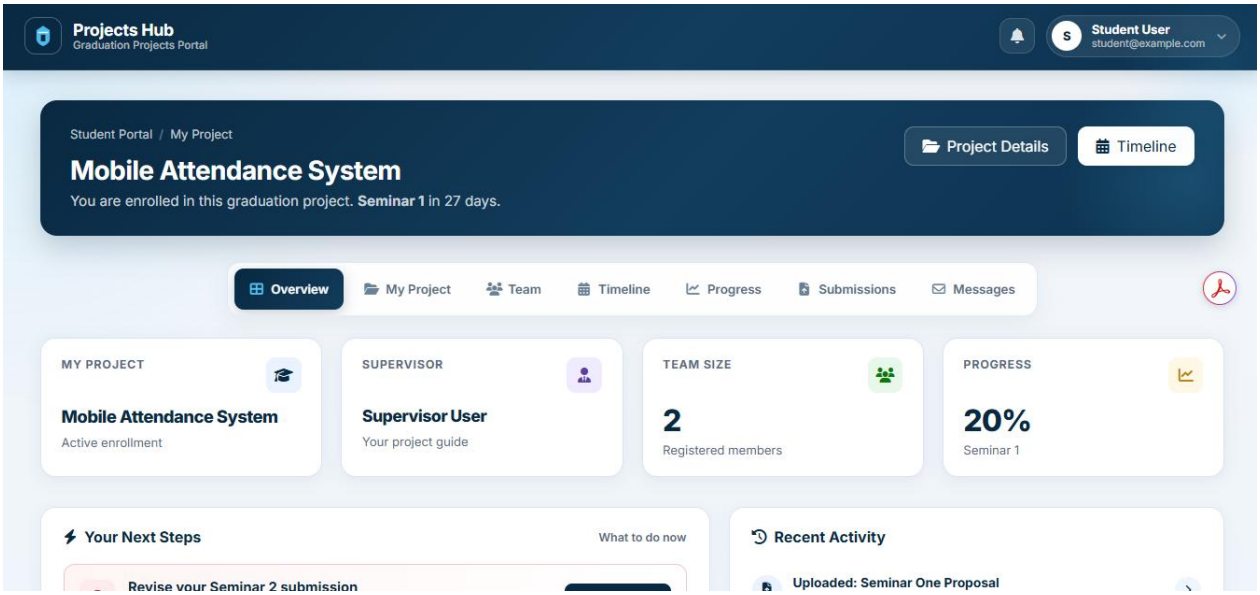


Figure 22 Student Enrolled-Project Workspace.

6.8.4.3 Artificial Intelligence Interface

The Student idea-submission interface includes two advisory artificial intelligence components: the Project Proposal Assistant and the Semantic Similarity Assistant.

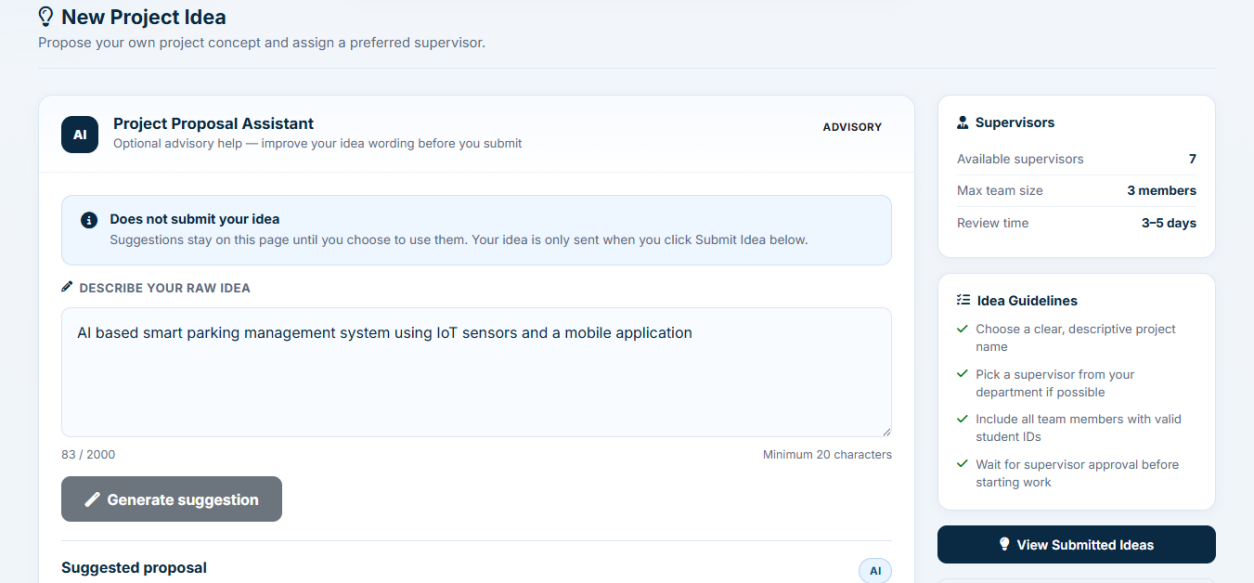


Figure 23 AI-Generated Proposal Suggestion.

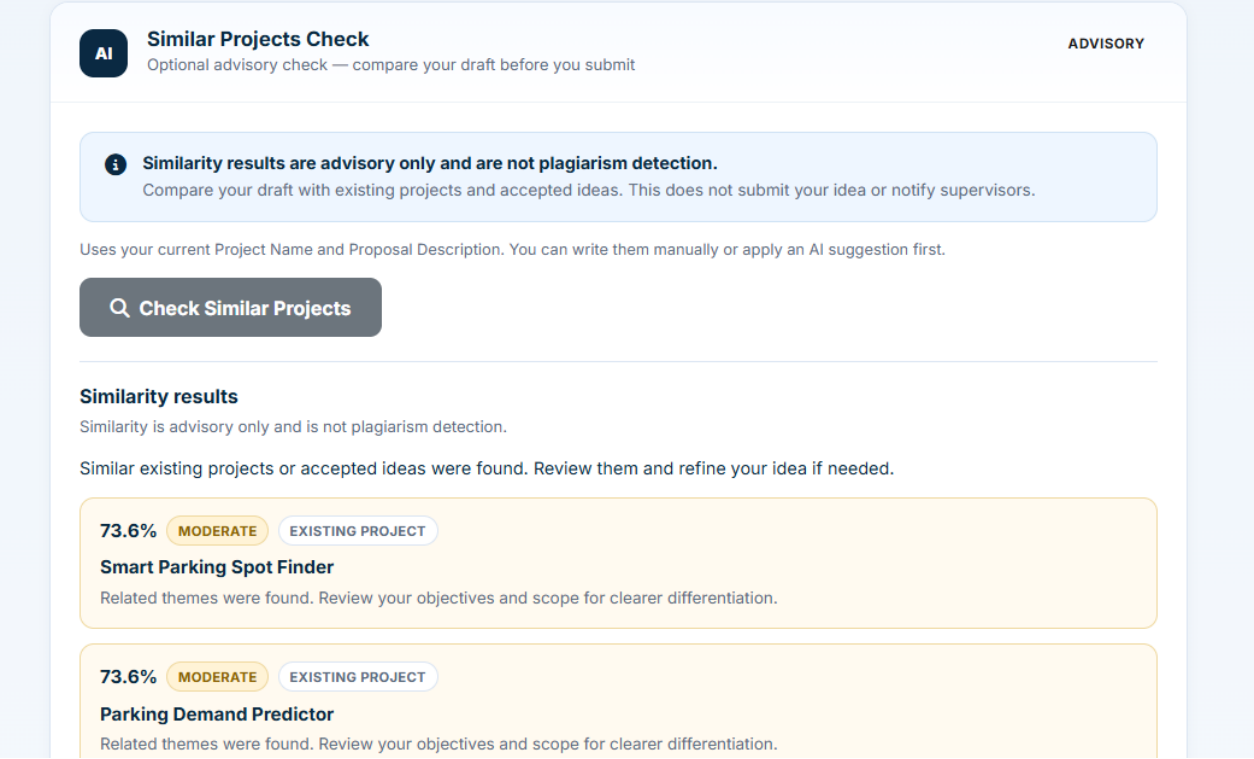


Figure 24 Advisory Semantic-Similarity Result.

6.8.4.4 Supervisor Interface

The Supervisor interface provides the operational functions required to manage owned graduation projects and evaluate Student workflows. The dashboard presents Supervisor projects, incoming join requests, proposed ideas, milestone submissions, Student messages, and project announcements.

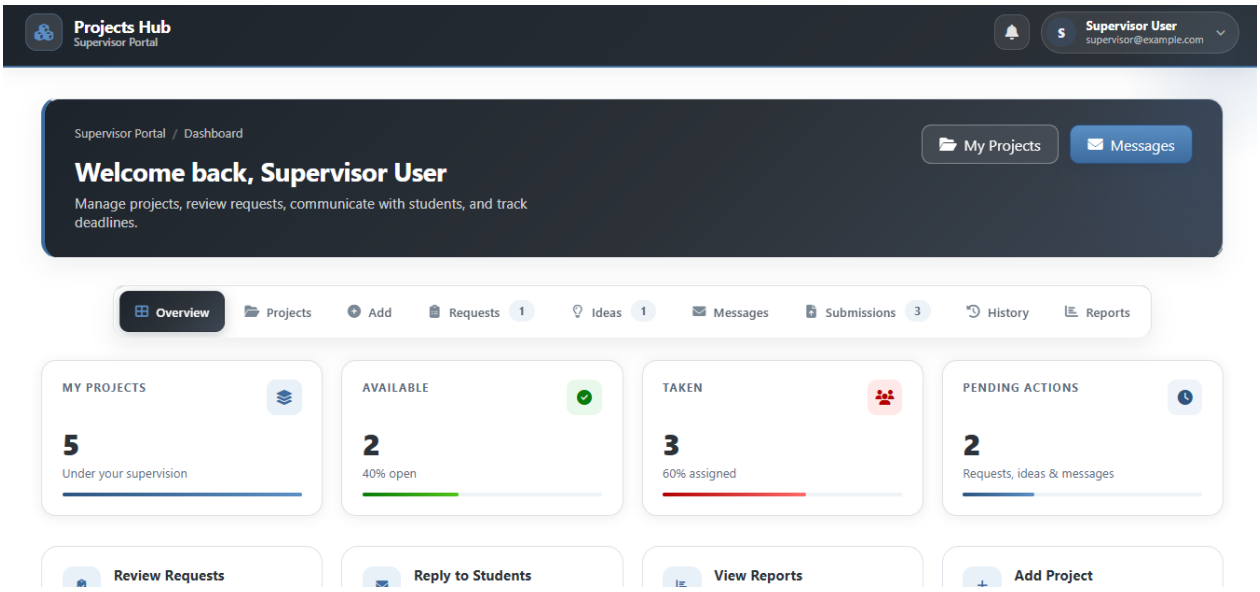


Figure 25 Supervisor Dashboard and Pending Academic Workflows.

6.8.4.5 Administrator Interface

The Administrator interface focuses on account lifecycle management and platform oversight. The dashboard displays summary indicators related to users, graduation projects, applications, ideas, submissions, and Supervisor workloads.

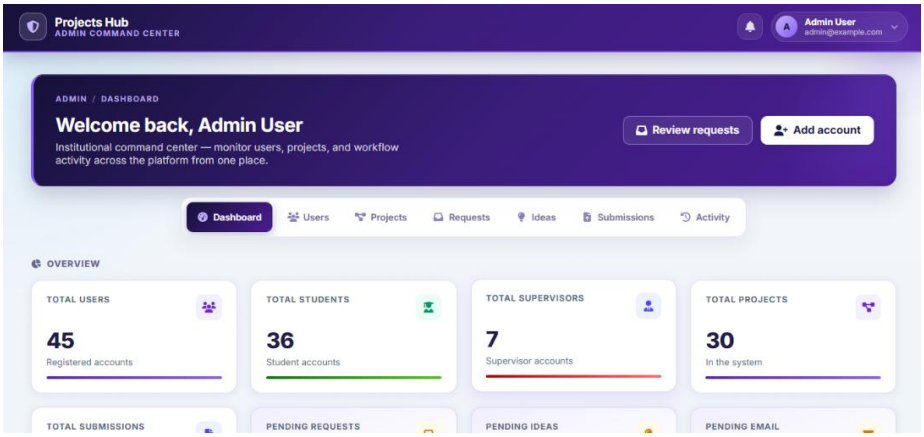


Figure 26 Administrator Dashboard and Platform Overview.

6.8.5 Artificial Intelligence Implementation

The Graduation Project Management Platform integrates two local artificial intelligence functions: the Project Proposal Assistant and the Semantic Similarity Assistant. Both functions are advisory and are implemented as supporting services within the Laravel application rather than as a separate artificial intelligence microservice.

The Laravel application communicates with a locally running Ollama service through server-side HTTP requests. The proposal assistant uses the llama3.2:3b language model, while semantic-similarity analysis uses the nomic-embed-text embedding model. Configuration values such as the provider, service address, model names, timeouts, input limits, similarity thresholds, and number of displayed matches are maintained through config/ai.php and environment variables.

The browser does not communicate directly with Ollama. Student requests are first received by authenticated Laravel routes and controllers, after which the appropriate service validates, processes, and normalizes the artificial intelligence response.

6.8.5.1 Artificial Intelligence Component Structure

Table 23 Artificial Intelligence Implementation Components

Component	Responsibility
AiProposalController	Receives and validates Student proposal-assistance requests and returns normalized JSON responses.
ProjectProposalAssistantService	Builds the language-model request, processes the model response, normalizes the proposal structure, and generates the fallback result when necessary.
ProjectSimilarityController	Receives and validates pre-submission semantic-similarity requests.

Component	Responsibility
ProjectSimilarityService	Builds the comparison corpus, coordinates embedding generation, calculates and ranks similarity results, and generates the final snapshot.
OllamaEmbeddingService	Sends text to the Ollama embedding interface and validates the returned vectors.
CosineSimilarity	Calculates the numerical similarity between the query vector and each comparison vector.
config/ai.php	Stores configurable AI provider, model, timeout, threshold, input-length, and result-limit settings.
Student Blade and JavaScript components	Display loading states, proposal suggestions, similarity results, disclaimers, and Apply controls.

6.8.5.2 Project Proposal Assistant Implementation

The Project Proposal Assistant helps a Student transform an initial informal idea into a structured project proposal. The Student enters a short description through the idea interface and explicitly activates the generation operation.

The request is sent to:

POST /student/ai/proposal

AiProposalController validates the raw_idea value before passing it to ProjectProposalAssistantService. The accepted input must contain at least 20 characters and must not exceed the configured maximum length, which is 2,000 characters by default.

The service creates a controlled instruction for the language model and requests a structured JSON result containing:

1. a suggested project title;
2. a problem statement;
3. project objectives;
4. the proposed scope;
5. functional requirements.

The server sends the request to Ollama's /api/chat interface using the configured llama3.2:3b model. The service requests JSON-formatted output and then validates and normalizes the returned fields before delivering them to the browser.

The generated content is not stored automatically. The Student must review the result and select the Apply operation before the suggested title and proposal content are copied into the normal idea form. Even after applying the result, the Student may edit the content and must explicitly submit the final project idea. The proposal assistant therefore does not create an idea record or notify a Supervisor independently.

6.8.5.3 Proposal Fallback Mechanism

The proposal assistant includes a deterministic fallback mechanism to preserve system availability. The fallback is used when artificial intelligence is disabled, the configured provider is unsupported, Ollama cannot be reached, the request times out, or the model returns malformed or incomplete output.

Instead of returning an unhandled exception, the service generates an editable proposal template based on the Student's original description. The JSON response identifies whether the returned result was produced by the artificial intelligence model or by the fallback mechanism.

This design preserves the primary academic workflow and allows the Student to continue preparing the project idea without depending on the availability of the local language model.

6.8.5.4 Semantic Similarity Implementation

The Semantic Similarity Assistant provides a pre-submission comparison between the Student's current proposal and previously stored project material. The objective is to provide an early advisory indication of possible conceptual overlap rather than to produce a plagiarism judgment.

The Student interface sends the current project title and proposal description to:

POST /student/ai/similarity

After validation, ProjectSimilarityService creates a concise textual representation using the project title and proposal description. The comparison corpus is assembled dynamically from all existing graduation projects and previously accepted, non-rejected project ideas. Pending and rejected ideas are excluded from the corpus.

The service uses separate prefixes for the query and comparison documents:

- search_query: for the Student's current proposal;
- search_document: for each stored project or accepted idea.

The prepared texts are sent to Ollama's /api/embed interface using the nomic-embed-text model. Ollama returns numerical embedding vectors representing the semantic content of each text. The vectors are processed only during the current request and are not stored in the relational database.

6.8.5.5 Cosine-Similarity Calculation

The semantic similarity between two embedding vectors is calculated using the cosine-similarity measure presented in Equation (1).

$$S_{\cos(A,B)} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}} \quad (1)$$

where:

- **Scos(A,B):** is the cosine-similarity score between the two embedding vectors.
- **A:** is the embedding vector of the Student's current project proposal.
- **B:** is the embedding vector of an existing project or an accepted project idea.
- **Ai and Bi:** are the i-th components of vectors A and B, respectively.
- **n:** is the number of dimensions in each embedding vector.
- **i:** is the vector-component index.

Listing 2 presents the implementation of the cosine-similarity calculation used to compare the embedding vector of the Student's proposal with the embedding vectors of existing projects and accepted ideas.

```
public static function score(array $a, array $b): ?float
{
    $dimA = count($a);
    $dimB = count($b);

    if ($dimA === 0 || $dimB === 0 || $dimA !== $dimB) {
        return null;
    }

    $dot = 0.0;
    $magA = 0.0;
    $magB = 0.0;

    for ($i = 0; $i < $dimA; $i++) {
        if (! is_numeric($a[$i]) || ! is_numeric($b[$i])) {
            return null;
        }

        $va = (float) $a[$i];
        $vb = (float) $b[$i];

        $dot += $va * $vb;
        $magA += $va * $va;
        $magB += $vb * $vb;
    }

    if ($magA <= 0.0 || $magB <= 0.0) {
        return null;
    }

    $cosine = $dot / (sqrt($magA) * sqrt($magB));

    if (! is_finite($cosine)) {
        return null;
    }

    return max(0.0, min(1.0, $cosine));
}
```

Listing 2 Implementation of the Cosine-Similarity Calculation

Chapter 7 Testing and Evaluation

This chapter presents the testing and evaluation activities performed to verify that the Graduation Project Management Platform operates according to its documented requirements. The evaluation combines automated feature and unit testing with manual role-based and end-to-end workflow verification.

7.1 Test Plan

7.1.1 Test Scope

Table 24 Testing Scope by System Area

System Area	Principal Items Tested
Authentication and Accounts	Login, logout, password recovery, password reset, email completion, account activation, session lifecycle, and role routing
Administrator Operations	User listing, Student creation, Supervisor creation, profile linking, activation/deactivation, password reset, oversight pages, and activity logs
Student Operations	Dashboard states, project browsing, join requests, project ideas, application tracking, project workspace, submissions, messages, and notifications
Supervisor Operations	Project creation and updating, ownership restrictions, application decisions, project teams, submission review, replies, announcements, and notifications
Workflow Integrity	One active project, duplicate-pending prevention, valid team membership, pending-state enforcement, project availability, and transactional acceptance
Data and File Security	Validation, CSRF protection, authenticated identities, private storage, authorized downloads, and resource-level restrictions
Artificial Intelligence	Proposal generation, fallback behavior, embedding requests, cosine calculation, result ranking, privacy-safe responses, and final server-side snapshots
User Interface	Validation feedback, dashboard navigation, role-specific controls, dialogs, status indicators, loading states, and representative responsive behavior

7.2 Types of Tests

Table 25 Testing Types Applied to the Graduation Project Management Platform

Test Type	Purpose	Application in the Platform	Status
Unit Testing	Verify isolated functions and service behavior	Cosine calculation, embedding-response processing, proposal normalization, fallback behavior, and reusable service logic	Implemented
Integration Testing	Verify cooperation among application components	Routes, middleware, controllers, services, Eloquent models, database records, notifications, storage, and AI HTTP communication	Implemented
System Testing	Verify complete workflows across the integrated platform	Authentication, applications, enrollment, submissions, communication, notifications, administration, and AI-assisted idea preparation	Implemented
Acceptance Testing	Confirm that implemented workflows satisfy documented requirements	Representative requirement-based and golden-path scenarios were manually verified	Partially implemented
Performance and Load Testing	Measure response time and behavior under increasing workload	Timeouts, throttling, batching, and practical local execution were verified; no formal concurrent-load experiment was conducted	Limited
Security Testing	Verify authentication, authorization, data protection, and invalid-operation rejection	Role separation, inactive accounts, CSRF-protected forms, project ownership, private files, notifications, and AI anti-tampering	Implemented
Usability Testing	Evaluate interface clarity and ease of completing workflows	Manual inspection of navigation, forms, feedback, status badges, dialogs, and role-based dashboards	Partially implemented
Compatibility Testing	Verify operation across supported environments	Manual checks using modern desktop browsers and reduced viewport sizes	Partially implemented
Regression Testing	Detect defects introduced by later changes	Complete automated suite executed after workflow, schema, framework, runtime, and AI changes	Implemented

7.3 Test Diagrams

7.3.1 Test-Case Execution Diagram

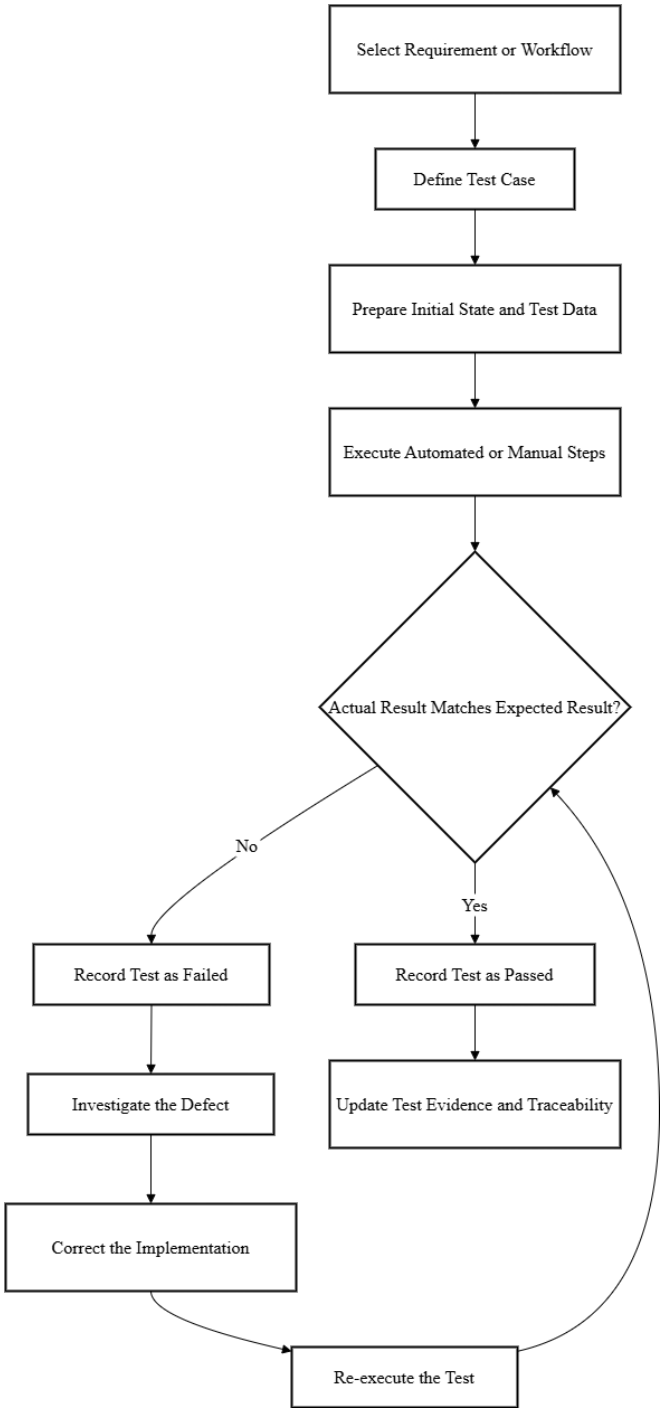


Figure 27 Test-Case Preparation and Execution Process.

7.3.2 Automated Test-Suite Distribution Diagram

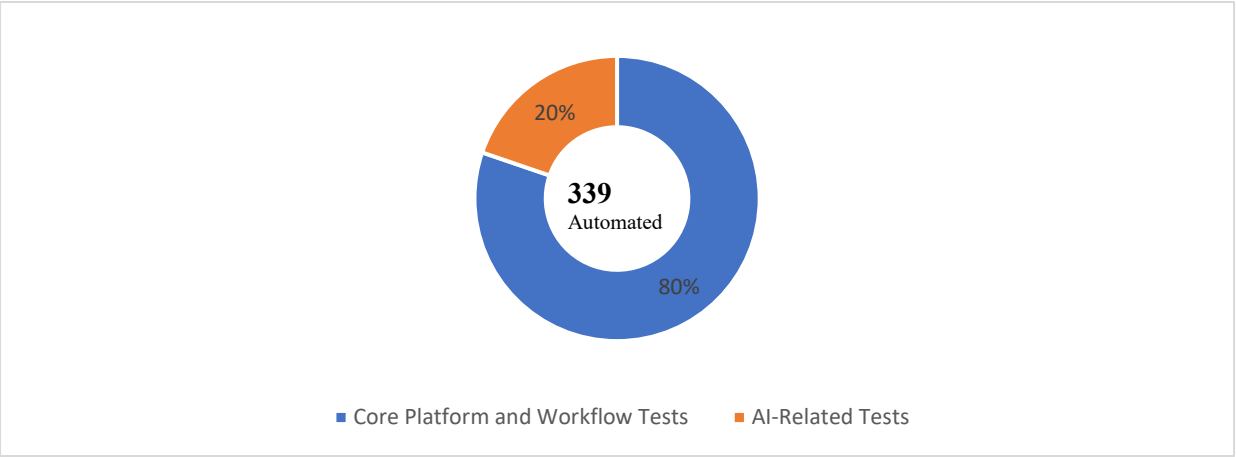


Figure 30 Distribution of the Automated Test Suite.

7.3.3 Automated Test-Result Diagram

The complete automated regression suite was executed after the final workflow and artificial intelligence changes. Figure 31 summarizes the final execution status.

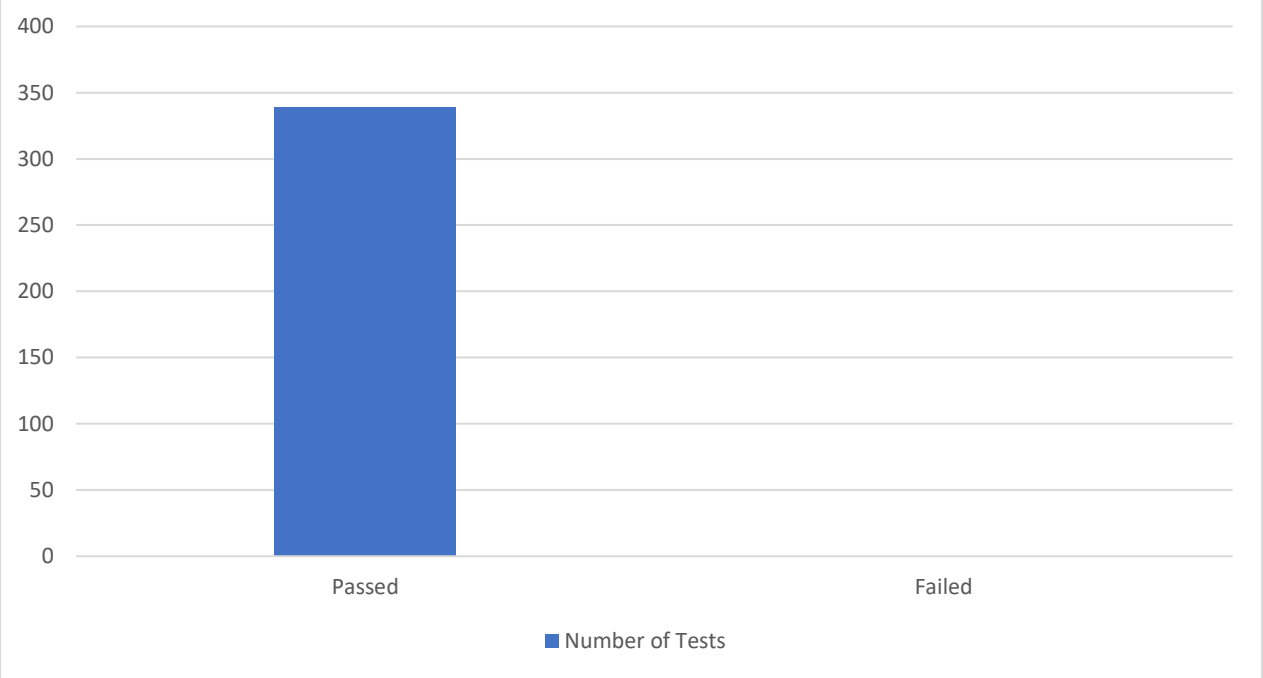


Figure 31 Final Automated Test Execution Results.

7.4 Test Results and Analysis

7.4.1 Final Automated Test Results

Table 26 Final Automated Test-Suite Results

Result Metric	Value
Total automated tests	339
Passed tests	339
Failed tests	0
Pass rate	100%
Failed-test rate	0%
Total assertions	1370
Execution time	65.78s
Execution command	php artisan test

7.4.2 Automated Results by Test Area

Table 27 Analysis of Automated Testing by System Area

Test Area	Principal Verified Behavior	Final Result
Authentication and Sessions	Login, logout, password recovery, password reset, session lifecycle, and role routing	Passed
Account and Access Control	Account activation, email completion, role separation, and unauthorized-access rejection	Passed
Administrator Operations	Student and Supervisor account provisioning, account-status management, password reset, and oversight access	Passed

Test Area	Principal Verified Behavior	Final Result
Project Applications	Join-request creation, idea submission, team validation, duplicate prevention, and decision processing	Passed
Enrollment and Membership	Acceptance transactions, official membership creation, one-active-project rule, and lifecycle consistency	Passed
Project Submissions	Upload validation, private storage, authorized download, review status, and feedback	Passed
Communication and Notifications	Messages, replies, announcements, notification ownership, and read-state updates	Passed
Activity Logging	Recording and sanitization of selected significant workflow events	Passed
Proposal Assistant	Validation, model response, normalized proposal output, Apply workflow support, and fallback behavior	Passed
Semantic Similarity	Embeddings, cosine calculation, ranking, privacy-safe responses, and unavailable-service handling	Passed
Similarity Snapshot	Server-side recalculation, anti-tampering behavior, persistence, and Supervisor presentation data	Passed

7.4.3 Artificial Intelligence Test Results

Table 28 Automated Test Results for the Artificial Intelligence Functions

AI Test Group	Number of Tests	Result
AI Proposal Assistant	17	Passed
Similarity Route and Interface	11	Passed
Similarity Service	11	Passed

Ollama Embedding Service	6	Passed
Cosine-Similarity Utility	5	Passed
Proposal-Description Workflow	7	Passed
Similarity-Snapshot Workflow	10	Passed
Total AI-related tests	**67**	**Passed**

All 67 artificial intelligence-related tests passed in the final automated baseline. These tests verify both isolated algorithmic behavior and integrated Laravel workflows.

Proposal-assistance tests cover input validation, structured output, response normalization, fallback operation, and the separation between generation and final idea submission. Similarity tests cover embedding requests, invalid vectors, cosine calculation, score ranking, privacy-safe response fields, final snapshot persistence, and rejection of browser-supplied similarity values.

Failure-oriented tests also verify that an unavailable or invalid Ollama response does not prevent the Student from continuing the primary project-idea workflow.

7.4.4 Representative Test-Case Results

Table 29 Representative Test-Case Execution Results

Test Case	Tested Scenario	Expected Result	Actual Result	Status
TC-AUTH-01	Login using a valid university number and password	User is authenticated and redirected to the correct role portal	Expected portal was displayed	Passed
TC-SEC-01	Student attempts to access a Supervisor route	Access is rejected	Request was rejected	Passed
TC-ADMIN-01	Administrator creates a Supervisor account	User, role, and linked Supervisor profile are created	All required records were created	Passed

Test Case	Tested Scenario	Expected Result	Actual Result	Status
TC-REQ-01	Student submits a valid project join request	Pending request and proposed members are stored	Request was stored correctly	Passed
TC-REQ-02	Student submits a conflicting pending application	Operation is rejected without inconsistent records	Application was rejected	Passed
TC-SUP-01	Responsible Supervisor accepts a pending request	Membership is synchronized and request becomes accepted	Transaction completed successfully	Passed
TC-FILE-01	Unrelated user attempts to download a private submission	File access is denied	Download was rejected	Passed
TC-SUB-01	Enrolled Student uploads a valid milestone file	Private file and submission record are created	Submission was stored correctly	Passed
TC-AI-01	Student requests an AI proposal while Ollama is available	Structured editable suggestion is returned	Suggestion was returned	Passed
TC-AI-02	Proposal service is unavailable	Controlled editable fallback is returned	Fallback was returned	Passed
TC-AI-03	Student checks semantic similarity	Ranked privacy-safe advisory results are returned	Expected result structure was returned	Passed
TC-AI-04	Client submits manipulated similarity fields	Server ignores client values and generates its own snapshot	Server-generated snapshot was stored	Passed

Chapter 8 Conclusion and Recommendations

8.1 System Application Results

The implemented platform provides an integrated environment for managing graduation-project activities among Students, Supervisors, and Administrators. The final application supports controlled account access, project discovery, project applications, Student-proposed ideas, Supervisor decisions, official team enrollment, milestone submissions, private file access, communication, notifications, administrative oversight, and local artificial intelligence assistance.

The system was implemented as a functional Laravel web application rather than as a static interface prototype. Operational results are persisted in the relational database and are reflected in the role-specific dashboards according to each user's identity, role, project relationship, and workflow state.

8.1.1 Role-Specific Results

Student Results

The Student portal adapts to the Student's current academic state. Before enrollment, the Student can browse available projects, submit a join request, or propose a new idea. While an application is pending, the Student can track its status and view the decision when issued. After acceptance, the portal provides the enrolled-project workspace, team information, milestone timeline, submissions, messages, announcements, progress information, and notifications.

This state-aware behavior reduces the possibility of presenting invalid actions. For example, a Student who is already enrolled does not continue to operate through the project-discovery workflow as though no project had been assigned.

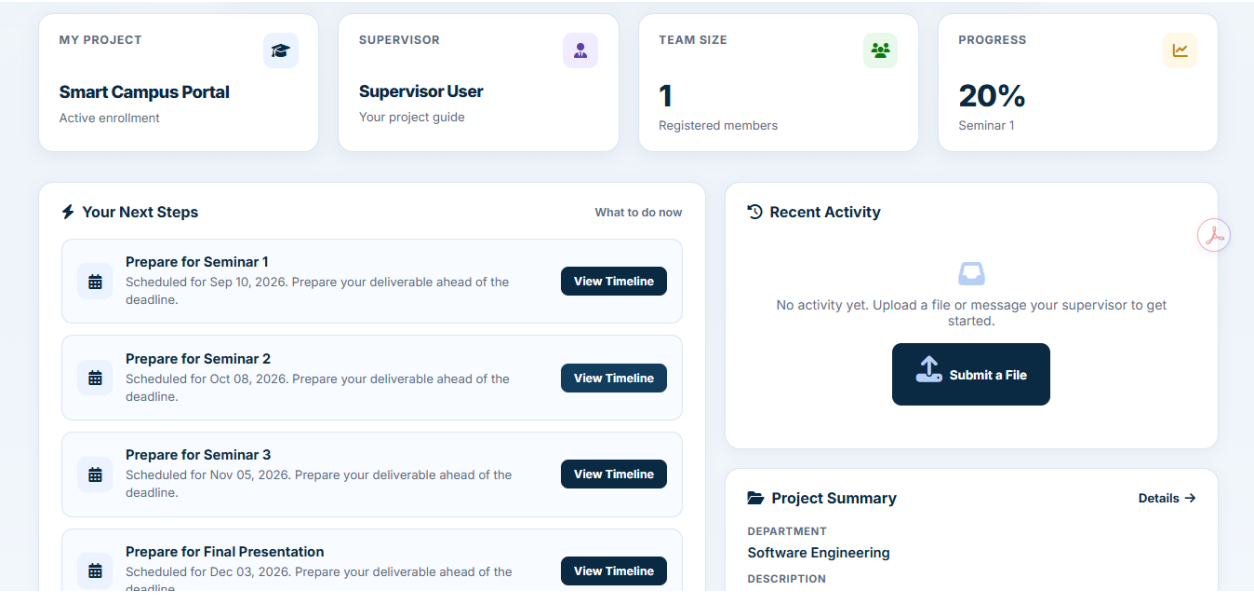


Figure 28 Student Workspace After Successful Project Enrollment.

Figure 30 shows the operational result of an accepted enrollment workflow. The Student interface displays the assigned project, team information, milestone structure, and the functions available after official membership has been created.

Supervisor Results

The Supervisor portal consolidates the projects and academic workflows assigned to the authenticated Supervisor. Pending join requests and project ideas can be evaluated, while processed records remain available through their corresponding history views.

Supervisors can inspect proposed team information, read the Student's proposal description, view the stored similarity snapshot, review project submissions, provide feedback, respond to messages, and publish project announcements.

Ownership restrictions ensure that a Supervisor receives and processes only resources associated with the linked Supervisor profile.

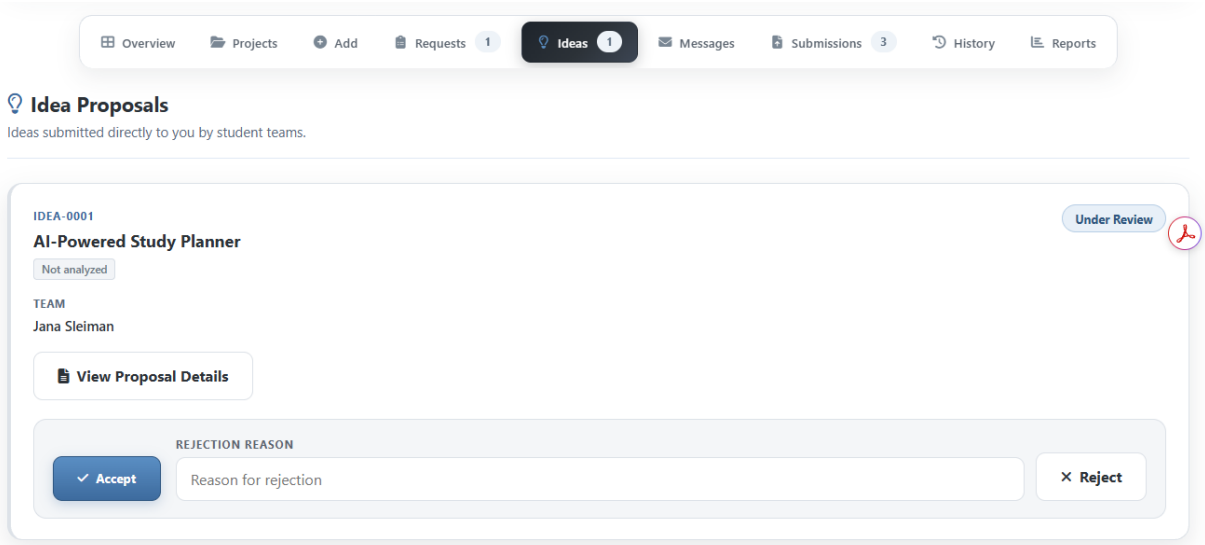


Figure 29 Supervisor Review of a Submitted Project Idea

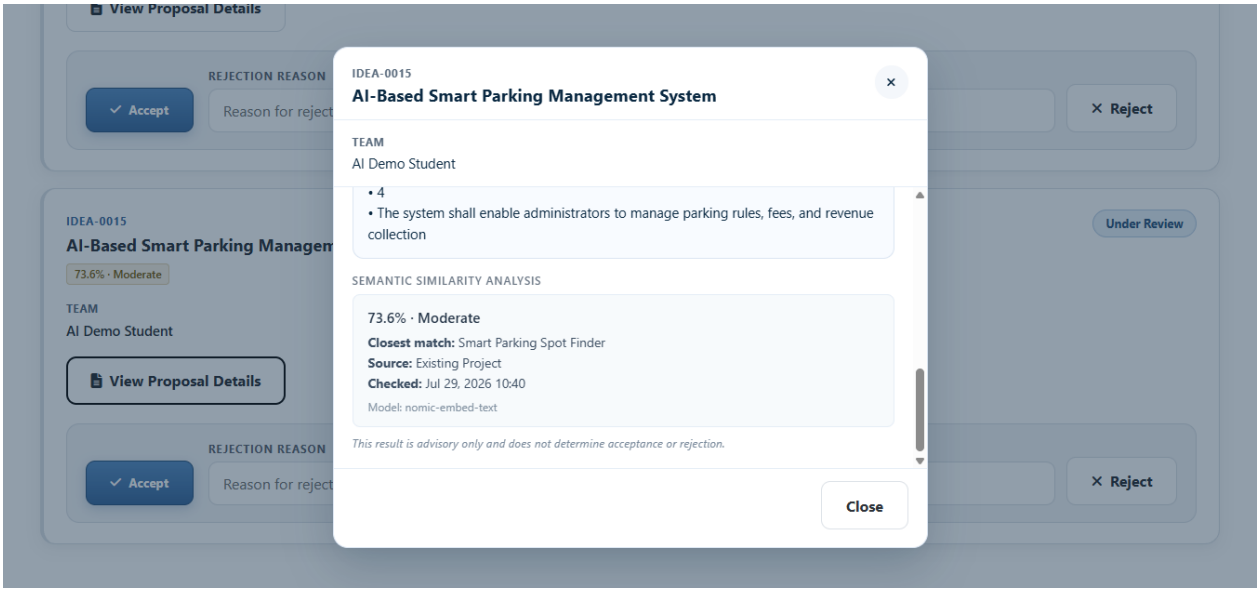


Figure 30 Supervisor View of the Stored Semantic-Similarity Snapshot

Figure 31 and Figure 32 presents the information available during the Supervisor's review of a project idea. The proposal description and similarity snapshot support the review process, while acceptance or rejection remains an explicit human decision.

Administrator Results

The Administrator portal provides account lifecycle management and read-only institutional oversight. Summary indicators present the current state of accounts, projects, applications, ideas, submissions, and Supervisor activity.

Administrators can create operational accounts, activate or deactivate users, reset another user's password, and inspect system records and activity history. Academic acceptance and submission-review operations remain under Supervisor responsibility and are not transferred to the Administrator role.

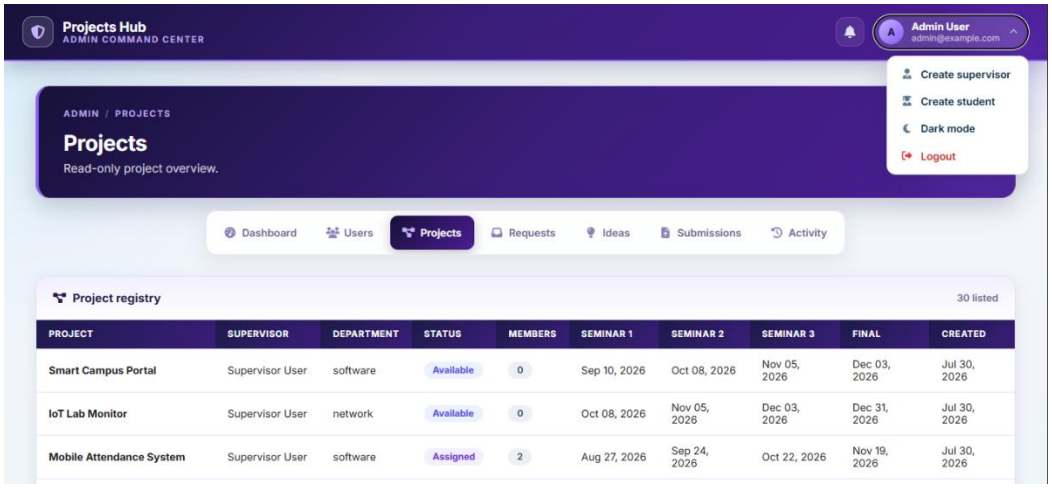


Figure 31 Administrator Platform Overview and Account Management Results

8.1.2 Application and Enrollment Results

The platform supports two distinct application paths. A Student may request to join an existing project or may propose a new project idea to a selected Supervisor.

Before submission, the platform validates the Student and proposed team members against the current enrollment and pending-application state. This prevents duplicated members, conflicting pending applications, invalid Student accounts, and multiple active project assignments.

When the responsible Supervisor accepts a valid application, the platform synchronizes the approved team with the selected or newly created project and changes the application state within a database transaction. This produces official project-membership records rather than leaving the accepted team as duplicated textual data.

Rejected applications remain available with their final state and optional rejection reason, allowing the Student to understand the outcome without modifying official project membership.

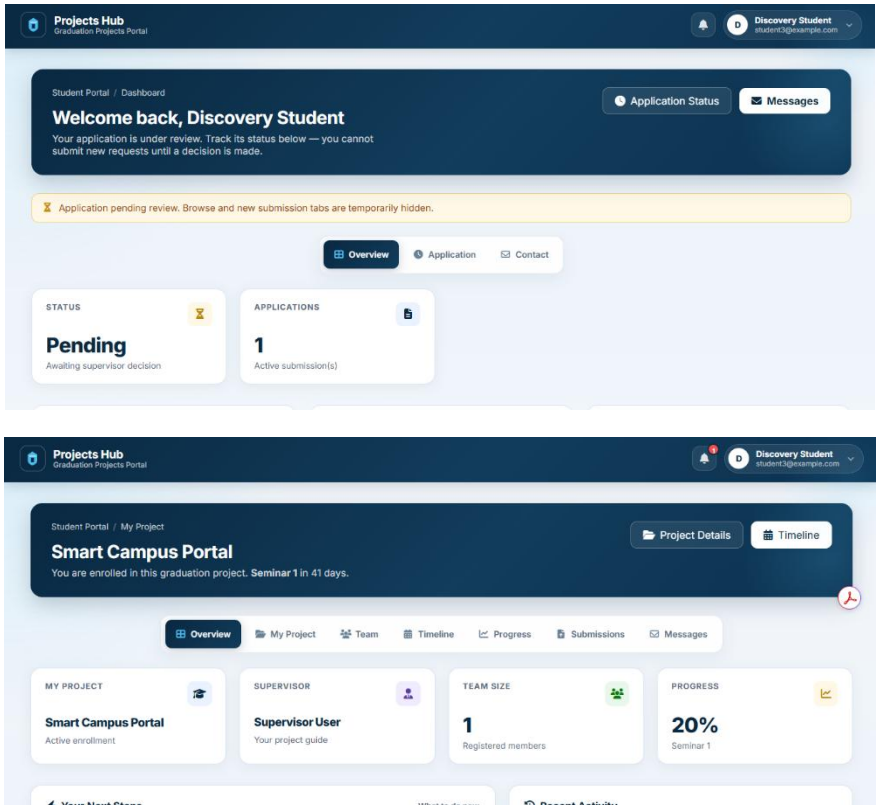


Figure 32 Accepted Project Request and Resulting Team Enrollment

8.1.3 Milestone Submission and Review Results

The implemented submission workflow allows an enrolled Student to upload a deliverable for an allowed project milestone. The application records the associated project, uploader, milestone, private file location, submission state, review information, and Supervisor feedback.

Submission files are maintained in private storage and are returned only after the application verifies the requester's project relationship. This prevents unrelated authenticated users from accessing a file by changing its identifier or URL.

The responsible Supervisor can review the deliverable, mark it as approved or requiring revision, and provide feedback. The resulting state and feedback are then displayed in the Student workspace.

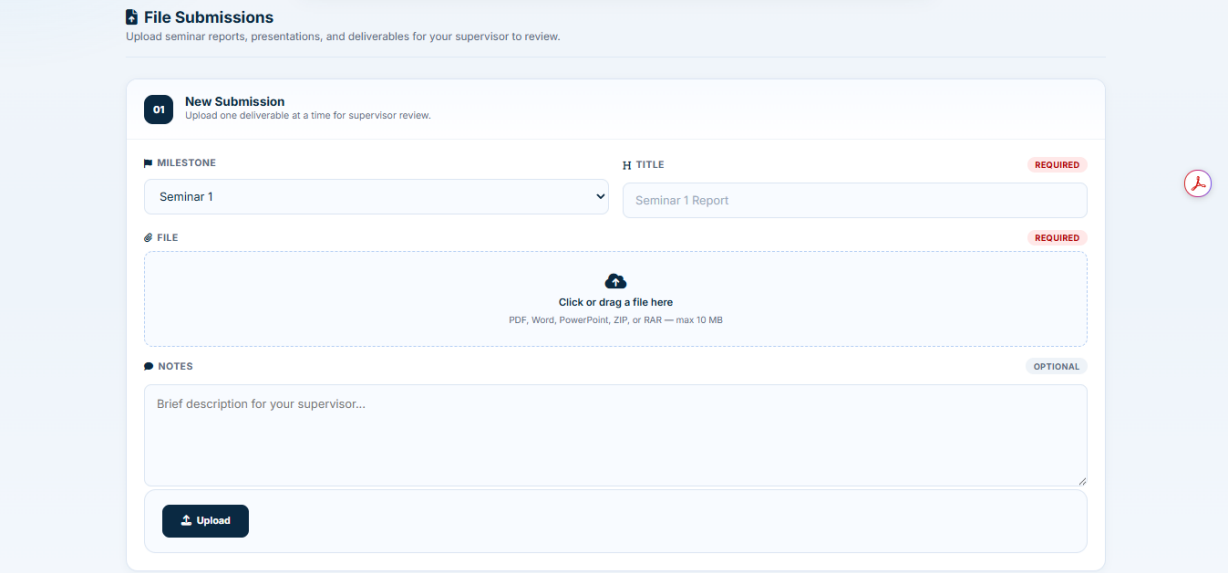


Figure 33 Student Milestone Submission Result

8.1.4 Communication and Notification Results

The platform provides structured communication between Students and Supervisors. Student messages are associated with the responsible Supervisor and may receive a stored reply. Supervisor announcements are associated with graduation projects and become visible to the corresponding enrolled teams.

Important workflow transitions generate user-specific notifications. These include application decisions, submission-related activity, reviews, and communication events. Notification ownership is verified before records are displayed or marked as read.

Activity logging provides an additional administrative result by preserving selected significant events with the acting user, affected resource, timestamp, and sanitized contextual information.

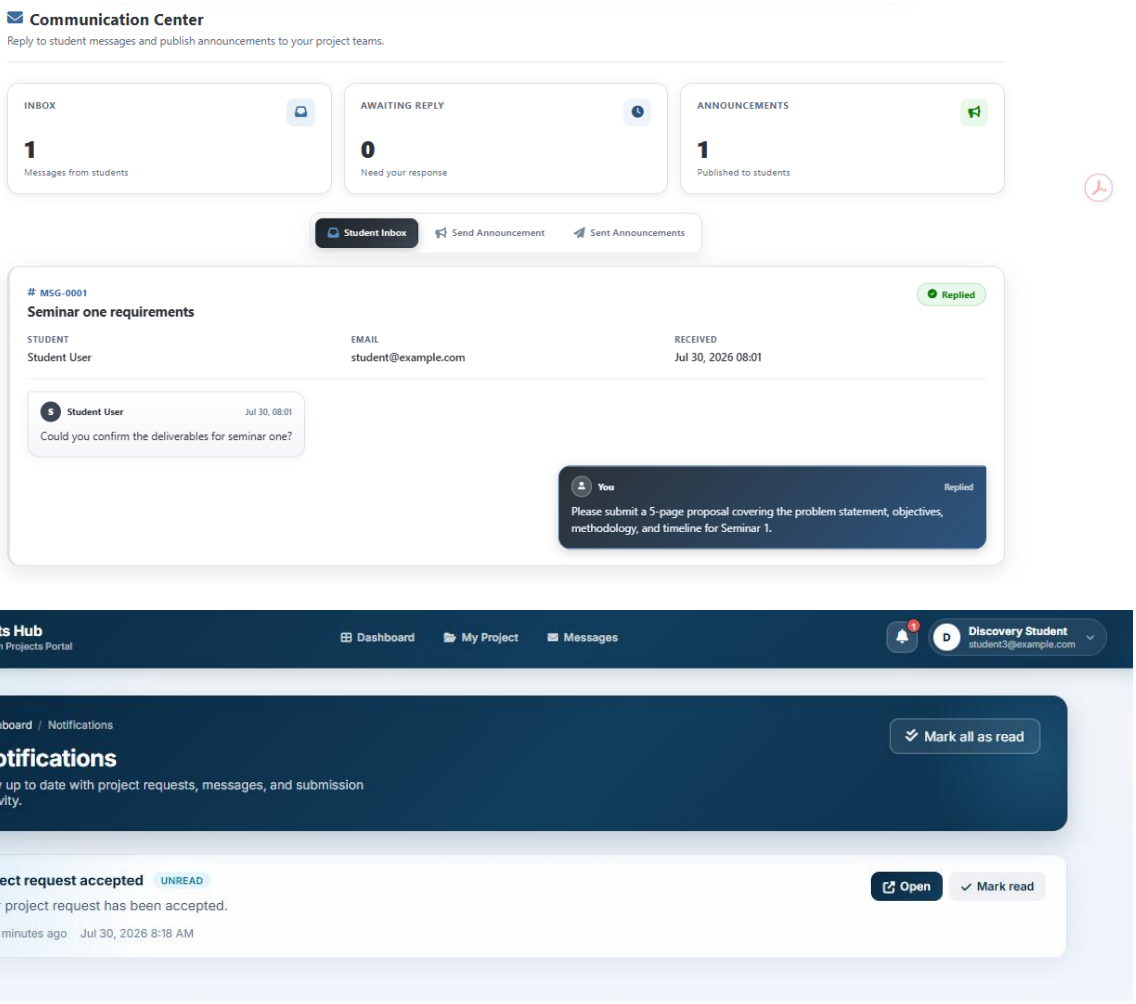


Figure 34 Communication and Notification

8.1.5 Artificial Intelligence Results

The implemented artificial intelligence contribution consists of two complementary functions. The first assists the Student in transforming an initial description into an editable structured project proposal. The second provides semantic comparison against existing graduation projects and accepted project ideas.

The Proposal Assistant returns a suggested title, problem statement, objectives, scope, and functional requirements. The Student may review, apply, modify, or discard the generated content. The operation does not create or submit the project idea automatically.

The Semantic Similarity Assistant uses locally generated embeddings and cosine similarity to rank conceptually related records. The result is presented as advisory information and is not treated as a plagiarism score or automatic rejection mechanism.

During final idea submission, the server recalculates the similarity result using the final submitted content and stores only a limited privacy-safe snapshot of the highest-ranking match. Embedding vectors and full foreign proposals are not persisted.

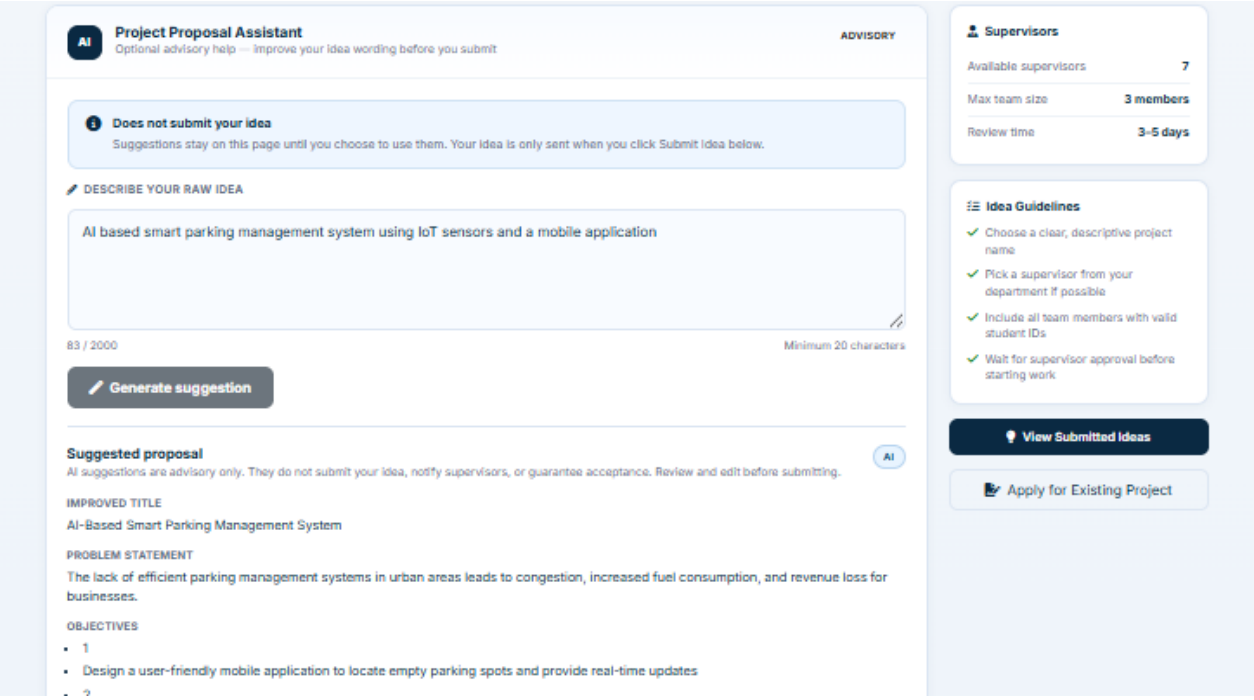


Figure 35 Generated and Editable Project Proposal Suggestion

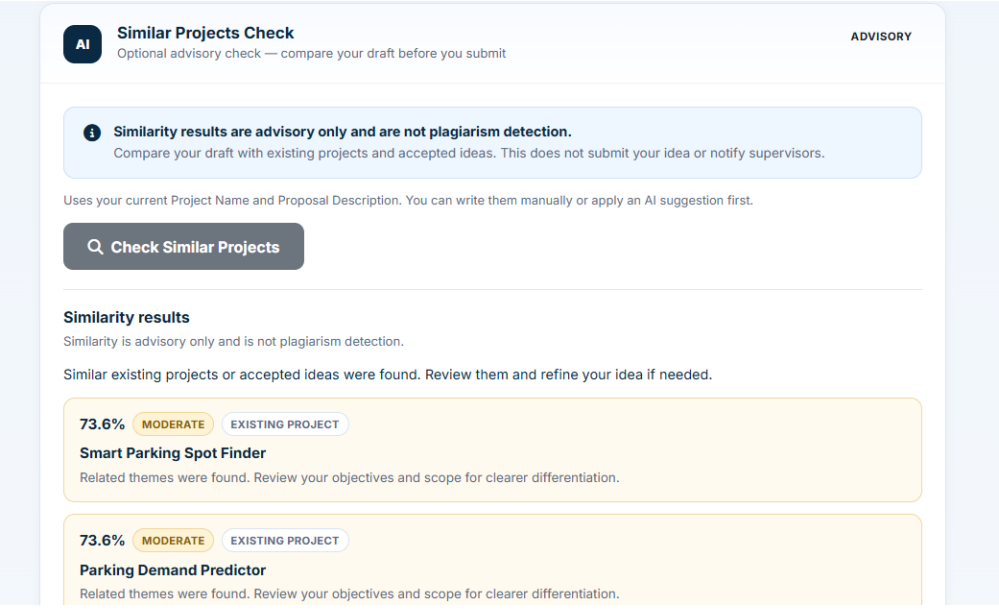


Figure 36 Student Advisory Semantic-Similarity Result

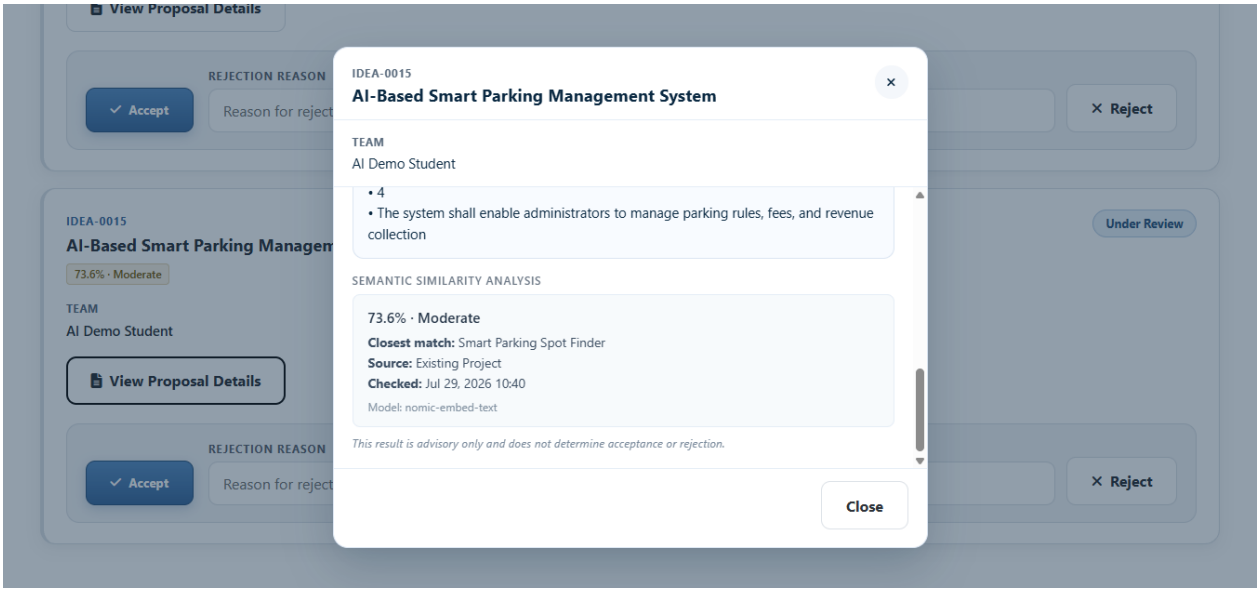


Figure 37 Supervisor View of the Stored Similarity Snapshot

8.2 Comparative Analysis with Existing Systems

This section compares the implemented Graduation Project Management Platform with two common alternatives: traditional management through email, paper records, and separate files; and general-purpose project-management platforms.

Unlike the comparative study presented in Chapter 3, which examined related systems and the research gap before implementation, the present comparison is based on the actual functions delivered by the completed platform.

Table 30 Comparative Analysis of Graduation-Project Management Approaches

Comparison Criterion	Proposed Graduation Project Management Platform	Traditional Email, Paper, and Separate Files	General-Purpose Project-Management Platforms
Workflow specialization	Designed specifically for graduation-project workflows involving Students, Supervisors, and Administrators	Does not provide a unified academic workflow	Provides general task and project management but normally requires extensive customization for graduation-project procedures
Account and role management	Provides controlled university-number authentication and separate Student, Supervisor, and Administrator responsibilities	Depends on unrelated email accounts, paper records, or manually managed contact lists	Provides workspace roles, but they are not inherently aligned with academic responsibilities
Project discovery	Allows eligible Students to browse available graduation projects and related Supervisors	Requires announcements, messages, spreadsheets, or direct communication	Can publish tasks or project boards, but does not normally provide an academic project-selection workflow

Comparison Criterion	Proposed Graduation Project Management Platform	Traditional Email, Paper, and Separate Files	General-Purpose Project-Management Platforms
Project applications	Supports structured requests to join existing projects and submission of new project ideas	Applications are commonly exchanged through email, paper forms, or unstructured messages	Forms and tasks may be customized, but acceptance and academic enrollment are not built-in domain workflows
Team validation	Validates Student accounts, duplicated members, pending conflicts, and active enrollment before accepting applications	Relies mainly on manual verification	May support team assignment but does not normally verify university enrollment constraints
Supervisor decisions	Allows the responsible Supervisor to accept or reject requests and ideas with controlled workflow states	Decisions may be recorded in separate messages or documents	Approval workflows may exist, but responsibility and academic project-enrollment effects require configuration
Official project membership	Creates relational membership records after acceptance	Membership may be represented in spreadsheets, documents, or message history	Users may be assigned to workspaces or boards, but this does not necessarily represent official academic enrollment
Milestone submissions	Supports milestone-based upload, private storage, review status, and Supervisor feedback	Files are distributed across email attachments, storage folders, or removable media	File attachment and comments are usually available, but academic milestones and review states require customization
File protection	Files are accessed through server-side authorization based on project membership and Supervisor ownership	Files may be forwarded or shared without consistent authorization controls	Commercial platforms generally provide access controls, depending on configuration and subscription plan
Communication	Provides Student messages, Supervisor replies, and project announcements within the academic context	Communication is fragmented across email and messaging applications	Provides comments and notifications, but communication is organized around generic tasks or boards
Notifications	Generates user-specific notifications for important workflow events	Depends on manual follow-up and external communication tools	Usually provides configurable notifications, although they are not inherently linked to academic workflow rules
Administrative oversight	Provides account management, summary indicators, read-only workflow monitoring, and activity records	Requires manual aggregation from multiple files and communication sources	Provides workspace reporting, but institutional academic indicators may require configuration or paid features
Proposal assistance	Generates an editable structured proposal through a locally executed language model with deterministic fallback	No integrated proposal-generation support	Some platforms may provide cloud AI assistance, depending on product and subscription, but not necessarily tailored to academic proposal structure
Semantic similarity	Compares proposals with existing projects and accepted ideas using local embeddings and cosine similarity	Comparison depends on manual memory, document search, or informal review	General-purpose platforms do not normally provide graduation-project semantic comparison as a native workflow
AI decision authority	AI output is advisory and does not automatically submit, accept, reject, or block an idea	Decisions are entirely manual	Behavior depends on the selected platform and configured AI features
Data organization	Uses structured relational records, controlled statuses, foreign keys, and official membership relations	Data may be duplicated, inconsistent, or difficult to retrieve across files	Provides structured cloud records, although the structure is based on generic project-management concepts

Comparison Criterion	Proposed Graduation Project Management Platform	Traditional Email, Paper, and Separate Files	General-Purpose Project-Management Platforms
Traceability	Preserves application status, submission reviews, notifications, and selected activity records	Traceability depends on retained messages and manually organized documents	Usually provides activity history, but not necessarily the specific academic traceability required by the project
Cost	The evaluated local implementation uses open-source software and does not require a recurring platform subscription; deployment and maintenance still have costs	Direct software cost may be low, but administrative time and fragmented management create indirect costs	May require recurring subscription fees, especially for advanced administration, storage, reporting, or AI features
Ease of use	Provides role-focused interfaces and state-aware Student workflows; formal external usability results were not measured	Familiar tools may be individually simple, but the complete workflow is fragmented	Usually provides polished interfaces, but generic terminology and features may require training and configuration
Performance	Core operations completed successfully in the evaluated local environment; no formal comparative response-time benchmark was conducted	Retrieval and coordination may be slow because information is distributed across channels	Mature hosted platforms generally provide optimized cloud operation, but performance depends on service and network conditions
Customization	Source code and workflows can be modified according to institutional academic requirements	Processes can be changed manually but lack automated enforcement	Customization is limited by the features, integrations, and subscription model of the selected platform
Internet dependency	Core Laravel functions can operate on a locally deployed network; the implemented AI models also run locally	Email and cloud-file workflows usually require internet access, while paper processes do not	Hosted platforms normally depend on continuous internet and provider availability
Institutional deployment readiness	Provides a tested academic implementation but still requires production infrastructure, formal user evaluation, backups, monitoring, and deployment hardening	Can operate immediately but provides limited automation and consistency	Provides mature infrastructure, but adaptation to institutional rules, data governance, and academic workflows may be required

8.3 Achievement of Project Objectives

This section evaluates the extent to which the objectives defined in Chapter 1 were achieved by the final implementation of the Graduation Project Management Platform.

Table 31 Evaluation of the Achievement of the Project Objectives

No.	Project Objective	Implementation Evidence	Achievement Status
1	Develop a centralized platform for managing graduation projects.	The platform integrates user accounts, projects, applications, ideas, project membership, milestone submissions, communication, notifications, and administrative oversight within one Laravel application and relational database.	Fully Achieved

2	Improve communication between Students, Supervisors, and Administrators.	Students can send messages to Supervisors and view their replies. Supervisors can publish project announcements, while supported workflow events generate role-specific notifications. Administrators can monitor selected system activities.	Functionally Achieved
3	Enable tracking of project progress, milestones, and submissions.	Enrolled Students can view project information, team membership, milestone dates, uploaded submissions, review statuses, feedback, and project-progress information through the enrolled-project workspace.	Fully Achieved
4	Automate the graduation-project workflow from project selection to final submission.	The system supports project discovery, join requests, project-idea submission, Supervisor decisions, official team enrollment, milestone upload, private file access, submission review, feedback, and workflow notifications.	Fully Achieved Within the Defined Scope
5	Provide secure role-based access for different user types.	Students, Supervisors, and Administrators authenticate through a shared university-number login. Middleware, Laratrust roles, project ownership, project membership, account activation, and resource-level checks restrict access to authorized operations and records.	Fully Achieved
6	Support project-request submission and project-idea evaluation.	Students can submit requests to join existing projects or propose new ideas with team information and proposal descriptions. Responsible Supervisors can review, accept, or reject each application, while accepted workflows create official project membership.	Fully Achieved
7	Reduce administrative workload and manual errors.	Account provisioning, application validation, workflow-state tracking, membership creation, notifications, database constraints, and activity logging replace several repetitive manual operations and reduce inconsistent records. However, no formal before-and-after workload measurement was conducted.	Partially Achieved and Not Quantitatively Measured

The platform successfully centralizes the principal graduation-project records and workflows. Instead of distributing project information across messages, spreadsheets, paper documents, and unrelated files, the system stores the relevant records in a structured relational environment and presents them through role-specific interfaces.

8.4 Strengths and Weaknesses Analysis

Table 32 Strengths and Weaknesses of the Implemented Platform

Category	Analysis
Strength	The platform integrates the principal graduation-project workflows within one role-controlled web application.
Strength	Student, Supervisor, and Administrator responsibilities are separated through authentication, roles, middleware, and resource-level authorization.
Strength	Project requests, proposed ideas, accepted membership, submissions, feedback, communication, and notifications are stored as structured relational records.

Category	Analysis
Strength	Reusable services centralize enrollment-state resolution, workflow validation, activity logging, proposal assistance, and semantic-similarity processing.
Strength	Database transactions, foreign keys, junction tables, validation rules, and workflow guards reduce inconsistent academic states.
Strength	Submission files are stored privately and returned only after project membership or Supervisor ownership is verified.
Strength	The Student dashboard adapts to discovery, pending, and enrolled states.
Strength	The system supports two application paths: joining an existing project and proposing a new project idea.
Strength	Local artificial intelligence functions provide editable proposal assistance and advisory semantic-similarity analysis.
Strength	AI results do not automatically submit, block, accept, or reject project ideas.
Strength	Embedding vectors and complete similarity-result lists are not persisted.
Strength	AI failures do not prevent the principal project-idea workflow from continuing.
Strength	The final automated baseline contains 339 passing tests and no failed tests.
Weakness	The evaluated system operates in a local development environment and has not been deployed to institutional production infrastructure.
Weakness	SQLite is suitable for the current local academic implementation but has not been evaluated under large-scale concurrent institutional use.
Weakness	Formal load, stress, scalability, and controlled response-time testing were not conducted.

Category	Analysis
Weakness	Formal testing with independent Students, Supervisors, and administrative users was not completed.
Weakness	No formal code-coverage percentage or defect-density measurement was generated.
Weakness	Compatibility testing was limited to the inspected Windows and desktop-browser environment.
Weakness	The project does not currently provide a dedicated mobile application.
Weakness	The system does not currently integrate with an institutional Student Information System or university identity provider.
Weakness	AI quality and latency depend on the local hardware, installed models, input quality, and Ollama availability.
Weakness	Semantic similarity provides advisory conceptual comparison and is not a specialized plagiarism-detection mechanism.
Weakness	Production backup automation, monitoring, disaster recovery, and deployment hardening are outside the evaluated implementation.

8.5 Principal Strengths

8.5.1 Domain-Specific Workflow Integration

One of the platform's principal strengths is its specialization for graduation-project management. The system does not represent graduation projects as generic tasks or independent uploaded files. Instead, it models Students, Supervisors, projects, applications, proposed ideas, team membership, milestones, submissions, reviews, communication, and administrative oversight as connected academic workflows.

This specialization allows the platform to enforce rules that would otherwise require manual verification. Examples include one active project per Student, prevention of conflicting pending applications, validation of proposed team members, project-availability checks, and restriction of academic decisions to the responsible Supervisor.

8.5.2 Role and Resource Separation

The system separates the responsibilities of Students, Supervisors, and Administrators through one authenticated identity structure and role-specific portals.

Role middleware controls access to the principal portal areas, while resource-level checks apply more detailed restrictions. A Supervisor cannot process another Supervisor's project applications, and a Student cannot download a submission associated with an unrelated project.

This combination is stronger than relying only on interface visibility because authorization is enforced by the server even when a user attempts to modify a route or resource identifier manually.

8.5.3 Relational Data Consistency

The database design uses relational identities, foreign keys, and junction tables instead of repeatedly storing Student names or university numbers in operational records.

Temporary proposed membership is maintained separately from official accepted project membership. This distinction allows the platform to preserve pending application information without treating proposed members as enrolled Students.

Database transactions are used where several related records must change together. For example, project membership synchronization and request-decision updates are performed as one atomic operation during acceptance.

8.5.4 Advisory Artificial Intelligence Integration

The platform integrates artificial intelligence directly into the project-idea workflow without transferring academic authority to an automated model.

The Proposal Assistant produces editable structured content, while the Semantic Similarity Assistant provides early awareness of conceptual overlap. The Student remains responsible for the final submitted proposal, and the Supervisor remains responsible for acceptance or rejection.

Local Ollama execution reduces dependence on an external cloud AI provider. The system also avoids persisting embedding vectors and stores only a limited privacy-safe similarity snapshot after final submission.

8.6 Principal Weaknesses

8.6.1 Local Evaluation Environment

The platform was developed and evaluated in a local Windows environment using Laravel Herd, SQLite, and a locally running Ollama service. This environment is suitable for development, testing, academic demonstration, and controlled evaluation.

However, the system has not been deployed to an institutional production server. Production networking, domain configuration, HTTPS termination, server hardening, monitoring, centralized logging, backup scheduling, and disaster recovery were not evaluated as operational components.

8.6.2 Database Scalability

SQLite provides a lightweight and reproducible relational environment for the evaluated implementation. It requires no separate database server and integrates effectively with Laravel's development and testing workflows.

Nevertheless, the current database configuration has not been evaluated under large-scale concurrent institutional activity. A production deployment may require migration to a managed relational database system such as MySQL or PostgreSQL, followed by compatibility, migration, concurrency, and performance testing.

This does not indicate that the current SQLite implementation is incorrect; it indicates that production-scale suitability has not been formally demonstrated.

8.6.3 Absence of Institutional Integration

The current platform maintains its own user accounts and university numbers. It does not integrate with an institutional Student Information System, university directory, Single Sign-On service, or official academic records platform.

As a result, Administrators must create and maintain Student and Supervisor accounts through the platform. Institution-wide deployment would benefit from controlled integration with authoritative university identity and enrollment data.

Conclusion and Recommendations

Project Summary and Main Achievements

This project presented the design, implementation, testing, and evaluation of a web-based Graduation Project Management Platform intended to organize the principal academic workflows connecting Students, Supervisors, and Administrators.

The platform was developed to address the fragmentation associated with managing graduation projects through separate messages, spreadsheets, documents, and file-storage locations. It provides a centralized relational environment for user accounts, graduation projects, project applications, proposed ideas, project membership, milestone submissions, reviews, communication, notifications, and selected activity records.

Students can browse available projects, request to join an existing project, or submit a new project idea to a selected Supervisor. After acceptance, the approved team becomes officially associated with the corresponding project and gains access to the enrolled-project workspace, milestone submissions, progress information, communication functions, announcements, and notifications.

Supervisors can create and update their own projects, evaluate assigned requests and ideas, review milestone submissions, provide feedback, reply to Student messages, and publish announcements. Administrators can provision Student and Supervisor accounts, manage account status, reset user passwords, inspect platform records, and review selected activity logs.

The platform was implemented using Laravel 13, PHP, Blade, Eloquent ORM, SQLite, Laratrust, Pest, and a service-oriented extension of the Laravel Model–View–Controller structure. The final documented scope contains 56 implemented functional requirements. The verified automated baseline contains 339 passing tests and 1,370 assertions.

Challenges and Difficulties Encountered

Framework and Runtime Upgrades

The application was progressively upgraded through newer Laravel versions and finally validated using Laravel 13 and PHP 8.5. This process introduced dependency, testing-framework, configuration, and runtime-compatibility challenges.

Isolated Git branches and regression testing were used to evaluate each upgrade before incorporating it into the stable implementation.

Artificial Intelligence Reliability

Local artificial intelligence integration introduced additional challenges related to service availability, model installation, variable response time, malformed generated output, embedding dimensions, and limited machine resources.

These challenges were addressed through configurable timeouts, input limits, response normalization, deterministic fallback behavior, soft-unavailable similarity handling, and separation of artificial intelligence functions from the essential idea-submission workflow.

Documentation Consistency

Maintaining consistency between the evolving implementation and the academic documentation was also a significant challenge. Requirements, diagrams, database models, interface descriptions, testing results, and artificial intelligence behavior had to be revised whenever the implementation changed.

Particular care was required to distinguish implemented capabilities from future work and to avoid retaining outdated assumptions about registration, project membership, similarity persistence, test counts, or artificial intelligence authority.

Future Work

Production Deployment and Infrastructure

Future work should prepare the platform for controlled institutional deployment. This includes deploying the Laravel application on a secured production server, enabling HTTPS, configuring production session and cookie policies, establishing centralized logging, and implementing monitoring, backup, and disaster-recovery procedures.

A staging environment should also be created so that database migrations, dependency upgrades, and workflow changes can be validated before production release.

University-System Integration

Future versions could integrate with the university's authoritative Student and staff records. Controlled integration with a Student Information System, institutional directory, or Single Sign-On provider could reduce manual account provisioning and improve identity consistency.

Such integration must define ownership of academic data, synchronization rules, account deactivation procedures, and privacy responsibilities.

Mobile and External Interfaces

A responsive mobile experience or dedicated mobile application could be developed after defining a secured API layer. The current system is primarily a server-rendered Laravel web application and does not provide a public general-purpose REST API.

API development would require versioning, token management, rate limiting, documentation, authorization policies, and dedicated security testing.

Artificial Intelligence Enhancement

The Proposal Assistant could be enhanced through configurable model selection, improved prompt templates, Arabic and English proposal support, and institution-specific proposal examples.

Semantic-similarity processing could be improved through embedding caching, corpus indexing, model comparison, threshold calibration, and clearer explanation of why two proposals are considered similar.

Future versions could also integrate with broader academic repositories or specialized plagiarism-analysis services. Such functionality should remain advisory and should not automatically accuse a Student of plagiarism or determine the Supervisor's final decision.

An administrative configuration interface could allow authorized staff to manage AI availability, model selection, thresholds, result limits, and advisory wording without changing application source code.

Final Conclusion

The Graduation Project Management Platform achieved its principal objective of providing an integrated and role-controlled environment for managing the main graduation-project lifecycle.

The completed platform centralizes account management, project discovery, applications, proposed ideas, Supervisor decisions, official team membership, milestone submissions, academic review, communication, notifications, activity oversight, and advisory artificial intelligence assistance.

The project demonstrates that graduation-project workflows can be represented through structured relational records, protected resources, reusable business rules, transactional processing, and state-aware interfaces. It also demonstrates that local artificial intelligence can support proposal preparation and semantic comparison without replacing Student responsibility or Supervisor judgment.

The final implementation provides a strong functional and tested academic baseline. Its next stage should focus on production infrastructure, institutional integration, formal user evaluation,

performance measurement, and controlled expansion of academic and artificial intelligence capabilities.

References

Abinaya, G., & Meenatchi, K. (2024). *Cloud based system for streamlined final year project allocation and tracking with deduplication*. *International Journal of Innovative Research in Computing and Technology (IJIRCT), 10*(4), 1–3.

Arumugam, S., & Ishak, S. A. (2025). *A web-based FSKTM final year project management system*. *Applied Information Technology and Computer Science, 6*(2), 338–355.

Isa, R., Othman, S., Ali, A. S., Azizan, N., & Ferguson, J. (2024). *Prototype development of final year project management system to monitor student performance*. *Journal of Advanced Research in Applied Sciences and Engineering Technology, 40*(1), 164–173.

Li, Q., Liu, Y., & Wu, W. (2021). *Design of university graduation project management system based on microservice architecture*. In *Proceedings of the 2021 International Conference on Social Sciences and Big Data Application* (pp. 21–27). Atlantis Press.

Rahman, E. S., Burhan, M. I., & Mangesa, R. T. (2025). *Development of sentence similarity detection application with semantic similarity and machine learning approaches: Case study of student thesis titles*. *Jurnal Media Elektrik, 23*(1), 19–36.

Su, Y., Zhao, S., & Fu, Y. (2024). *Design and research of university students' graduation project management system based on outcomes-based education*. In *Proceedings of the 3rd International Conference on New Media Development and Modernized Education*. EAI.

Laravel. (2025). *Laravel Documentation*. https://laravel.com/docs

Ollama. (2025). *Ollama Documentation*. https://ollama.com

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.

Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence embeddings using Siamese BERT-networks*. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*.

Sandhu, R., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). *Role-based access control models*. *IEEE Computer, 29*(2), 38–47.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.